

# Package ‘rwf’

June 17, 2026

**Title** Statistical reporting and visualization for common methods

**Version** 0.1.0

**Author** Dimitrios Zacharatos [aut, cre]

**Maintainer** Dimitrios Zacharatos <dzacharatos@yahoo.com>

**Description** Utilities for psychological data analysis, including automated statistical reporting and visualization for common inferential and modeling workflows. Helps users run analyses faster with less repetitive code and more standardized output.

**Depends** R (>= 3.5), ggplot2

**Imports** MASS, future, future.apply, gtools, irr, openxlsx, pROC, plyr, psych, reshape2, xgboost, DescTools, Rmisc, ggfortify, ggrepel, car, sjstats, ggpubr, gridExtra, scales, pwr, mirt, nlme, NLP, openNLP, tm, spelling, lavaan, ggExtra, QuantPsyc, purrr, emmeans, ez, htmlwidgets, mixlm, rstatix, rstudioapi, igraph, stopwords, gtable, stringr

**Suggests** e1071, forecast, lsr, rcompanion, thurstonianIRT

**URL** <https://sedzinfo.github.io/rwf/>

**BugReports** <https://github.com/sedzinfo/rwf/issues>

**License** GPL (>= 2)

**Encoding** UTF-8

**LazyData** true

**Config/roxygen2/version** 8.0.0

**RoxygenNote** 7.3.3

## Contents

|                             |    |
|-----------------------------|----|
| alpha_diagnostics . . . . . | 6  |
| call_to_string . . . . .    | 7  |
| cdf . . . . .               | 7  |
| cdff . . . . .              | 9  |
| cfa_icc_index . . . . .     | 10 |
| change_data_type . . . . .  | 11 |
| check_heywood . . . . .     | 12 |

|                                          |    |
|------------------------------------------|----|
| clear_stopwords . . . . .                | 13 |
| clear_text . . . . .                     | 14 |
| comparison_combinations . . . . .        | 14 |
| compute_ability . . . . .                | 15 |
| compute_adjustment . . . . .             | 16 |
| compute_aggregate . . . . .              | 16 |
| compute_aov_es . . . . .                 | 18 |
| compute_confidence_interval . . . . .    | 18 |
| compute_crosstable . . . . .             | 19 |
| compute_descriptives . . . . .           | 20 |
| compute_dissatenuation . . . . .         | 21 |
| compute_dummy_comparisons . . . . .      | 22 |
| compute_frequencies . . . . .            | 22 |
| compute_icc_thurstonian . . . . .        | 23 |
| compute_info_1pl . . . . .               | 24 |
| compute_info_2pl . . . . .               | 24 |
| compute_info_3pl . . . . .               | 25 |
| compute_kruskal_wallis_test . . . . .    | 26 |
| compute_kurtosis . . . . .               | 27 |
| compute_map . . . . .                    | 28 |
| compute_moving_average . . . . .         | 28 |
| compute_one_way_test . . . . .           | 29 |
| compute_posthoc . . . . .                | 30 |
| compute_power_r . . . . .                | 30 |
| compute_power_r_matrix . . . . .         | 31 |
| compute_residual_stats . . . . .         | 32 |
| compute_scores . . . . .                 | 32 |
| compute_se_theta . . . . .               | 33 |
| compute_skewness . . . . .               | 34 |
| compute_solve . . . . .                  | 34 |
| compute_standard . . . . .               | 36 |
| compute_standard_error . . . . .         | 37 |
| compute_tversky_index . . . . .          | 37 |
| compute_unidimensional_ability . . . . . | 39 |
| compute_unidimensional_theta . . . . .   | 40 |
| compute_y_logistic . . . . .             | 41 |
| confusion . . . . .                      | 41 |
| confusion_matrix_percent . . . . .       | 42 |
| convert_excel_unix_timestamp . . . . .   | 43 |
| c_bind . . . . .                         | 44 |
| data_frame_index . . . . .               | 44 |
| decompose_datetime . . . . .             | 45 |
| deg2rad . . . . .                        | 46 |
| detach_package . . . . .                 | 46 |
| df_admission . . . . .                   | 47 |
| df_automotive_data . . . . .             | 48 |
| df_blood_pressure . . . . .              | 49 |
| df_co2 . . . . .                         | 50 |

|                                             |    |
|---------------------------------------------|----|
| df_crop_yield . . . . .                     | 51 |
| df_difficile . . . . .                      | 51 |
| df_insurance . . . . .                      | 52 |
| df_ocean . . . . .                          | 53 |
| df_personality . . . . .                    | 56 |
| df_responses . . . . .                      | 58 |
| df_responses_state . . . . .                | 63 |
| df_sexual_comp . . . . .                    | 63 |
| df_titanic . . . . .                        | 64 |
| display_upper_lower_triangle . . . . .      | 65 |
| dotnames . . . . .                          | 66 |
| drop_levels . . . . .                       | 67 |
| dummy_arrange . . . . .                     | 67 |
| duplicate_y_axis . . . . .                  | 68 |
| environment_options . . . . .               | 69 |
| excel_confusion_matrix . . . . .            | 69 |
| excel_critical_value . . . . .              | 70 |
| excel_generic_format . . . . .              | 71 |
| excel_matrix . . . . .                      | 72 |
| extract_components . . . . .                | 74 |
| extract_tirt_params . . . . .               | 75 |
| fixed . . . . .                             | 76 |
| flatten_list . . . . .                      | 77 |
| generate_comparisons_matrix . . . . .       | 77 |
| generate_correlation_matrix . . . . .       | 78 |
| generate_data . . . . .                     | 78 |
| generate_factor . . . . .                   | 79 |
| generate_matrix_A . . . . .                 | 80 |
| generate_matrix_lambda_hat . . . . .        | 80 |
| generate_missing . . . . .                  | 81 |
| generate_multiple_responce_vector . . . . . | 81 |
| generate_string . . . . .                   | 82 |
| generate_unique_comparisons_index . . . . . | 82 |
| getfwp . . . . .                            | 83 |
| get_script_directory . . . . .              | 84 |
| hinvert_title_grob . . . . .                | 84 |
| icc_cfa . . . . .                           | 85 |
| increase_index . . . . .                    | 85 |
| install_all_packages . . . . .              | 86 |
| install_load . . . . .                      | 86 |
| key_to_cfa_model . . . . .                  | 87 |
| k_fold . . . . .                            | 87 |
| k_sample . . . . .                          | 88 |
| matrix_triangle . . . . .                   | 89 |
| mean_sd_alpha . . . . .                     | 90 |
| mgsb . . . . .                              | 90 |
| min_max_index . . . . .                     | 91 |
| model_loadings . . . . .                    | 92 |

|                                          |     |
|------------------------------------------|-----|
| name_triplet_pairs . . . . .             | 92  |
| off_diagonal_index . . . . .             | 94  |
| outlier_summary . . . . .                | 94  |
| output_compare_model_logistic . . . . .  | 95  |
| output_separator . . . . .               | 95  |
| padNA . . . . .                          | 96  |
| plot_acf . . . . .                       | 97  |
| plot_boxplot . . . . .                   | 98  |
| plot_cfa . . . . .                       | 98  |
| plot_cfa_gg . . . . .                    | 99  |
| plot_confusion . . . . .                 | 100 |
| plot_corrplot . . . . .                  | 101 |
| plot_crosstable . . . . .                | 101 |
| plot_histogram . . . . .                 | 102 |
| plot_icc_thurstonian . . . . .           | 103 |
| plot_interaction . . . . .               | 104 |
| plot_irt_onefactor . . . . .             | 105 |
| plot_loadings . . . . .                  | 106 |
| plot_logistic_model . . . . .            | 107 |
| plot_mosaic . . . . .                    | 108 |
| plot_mtmm . . . . .                      | 109 |
| plot_multiplot . . . . .                 | 110 |
| plot_normality_diagnostics . . . . .     | 111 |
| plot_oneway . . . . .                    | 112 |
| plot_oneway_diagnostics . . . . .        | 113 |
| plot_outlier . . . . .                   | 114 |
| plot_qq . . . . .                        | 115 |
| plot_response_frequencies . . . . .      | 116 |
| plot_roc . . . . .                       | 117 |
| plot_scatterplot . . . . .               | 118 |
| plot_scee . . . . .                      | 119 |
| plot_separability . . . . .              | 120 |
| plot_trees_xgboost . . . . .             | 121 |
| plot_ts . . . . .                        | 122 |
| proper . . . . .                         | 123 |
| proportion_accurate . . . . .            | 123 |
| questions_by_keys . . . . .              | 124 |
| questions_dimensions_dataframe . . . . . | 124 |
| rad2deg . . . . .                        | 125 |
| rank3_to_triplets . . . . .              | 126 |
| rank_df_to_binary . . . . .              | 126 |
| rank_to_binary . . . . .                 | 127 |
| raw_alpha . . . . .                      | 128 |
| rbind_all . . . . .                      | 128 |
| recode_scale_dummy . . . . .             | 129 |
| remove_nc . . . . .                      | 130 |
| remove_outliers . . . . .                | 131 |
| remove_user_packages . . . . .           | 132 |

|                                            |     |
|--------------------------------------------|-----|
| replace_na_with_previous . . . . .         | 132 |
| report_alpha . . . . .                     | 133 |
| report_cfa . . . . .                       | 134 |
| report_choric_serial . . . . .             | 135 |
| report_correlation . . . . .               | 136 |
| report_dataframe . . . . .                 | 137 |
| report_efa . . . . .                       | 138 |
| report_factorial_anova . . . . .           | 139 |
| report_hlr . . . . .                       | 141 |
| report_irt . . . . .                       | 142 |
| report_lda . . . . .                       | 143 |
| report_logistic . . . . .                  | 143 |
| report_manova . . . . .                    | 145 |
| report_normality_tests . . . . .           | 146 |
| report_oneway . . . . .                    | 147 |
| report_pdf . . . . .                       | 148 |
| report_regression . . . . .                | 149 |
| report_ttests . . . . .                    | 151 |
| report_wtests . . . . .                    | 154 |
| report_xgboost . . . . .                   | 157 |
| response_dimension . . . . .               | 158 |
| response_frequency . . . . .               | 159 |
| result_confusion_performance . . . . .     | 160 |
| round_dataframe . . . . .                  | 161 |
| score_tirt . . . . .                       | 162 |
| score_tirt_pattern . . . . .               | 164 |
| shrout . . . . .                           | 165 |
| simulate_cfa_fit . . . . .                 | 166 |
| simulate_correlation_from_sample . . . . . | 168 |
| split_str . . . . .                        | 169 |
| split_str_df . . . . .                     | 169 |
| stat_word_char . . . . .                   | 170 |
| string_aes . . . . .                       | 170 |
| str_count . . . . .                        | 171 |
| str_pad . . . . .                          | 172 |
| str_replace . . . . .                      | 173 |
| str_replace_all . . . . .                  | 174 |
| str_split_fixed . . . . .                  | 175 |
| str_squish . . . . .                       | 176 |
| str_wrap . . . . .                         | 177 |
| sub_str . . . . .                          | 178 |
| swap . . . . .                             | 178 |
| symmetric_matrix . . . . .                 | 179 |
| tag_pos . . . . .                          | 179 |
| text_similarity . . . . .                  | 180 |
| trim_df . . . . .                          | 181 |
| ts_smoothing . . . . .                     | 181 |
| wrapper . . . . .                          | 183 |

write\_txt . . . . . 183

Index 184

---

|                   |                                                                      |
|-------------------|----------------------------------------------------------------------|
| alpha_diagnostics | <i>Item-total correlations and alpha-if-item-removed diagnostics</i> |
|-------------------|----------------------------------------------------------------------|

---

Description

Computes item-level reliability diagnostics for a unidimensional scale. For each item the function returns the corrected and uncorrected item-total correlations and Cronbach’s alpha recalculated with that item excluded. Use this alongside [raw\\_alpha](#) to identify items that reduce scale reliability.

Usage

```
alpha_diagnostics(df)
```

Arguments

df                    A data frame whose columns are the items of a single scale. All columns must be numeric.

Value

A data frame with one row per item and the following columns:

- item** Item name.
- alpha.if.item.removed** Cronbach’s alpha computed on the remaining items after excluding this item.
- item.total.correlation** Pearson correlation between this item and the sum of all items (uncorrected).
- item.total.correlation.r.drop** Pearson correlation between this item and the sum of all other items (corrected item-total correlation, also known as r-drop). Values below 0.30 typically indicate a problematic item.

Examples

```
set.seed(12345)
df<-data.frame(matrix(.5,ncol=6,nrow=6))
correlation_martix<-as.matrix(df)
diag(correlation_martix)<-1
df<-round(generate_correlation_matrix(correlation_martix,nrows=1000),0)+5
psych::alpha(df)
alpha_diagnostics(df=df)
```

---

|                |                             |
|----------------|-----------------------------|
| call_to_string | <i>Model call to string</i> |
|----------------|-----------------------------|

---

**Description**

Takes a call object and convert it to string

**Usage**

```
call_to_string(model)
```

**Arguments**

|       |              |
|-------|--------------|
| model | Model object |
|-------|--------------|

**Examples**

```
df<-generate_correlation_matrix()
model<-lm(df$X1~df$X2)
call_to_string(model)
```

---

|     |                        |
|-----|------------------------|
| cdf | <i>Check dataframe</i> |
|-----|------------------------|

---

**Description**

Produces a column-level diagnostic summary of a dataframe, reporting missing values, data types, range statistics, and optionally unique values and factor levels. Returns a named list with a per-column table and a whole-dataframe summary. Can optionally export results to an .xlsx file.

**Usage**

```
cdf(
  df,
  name_length = (getOption("width")/3),
  digits = 2,
  nunique = 0,
  parallel = FALSE,
  file = NULL
)
```

## Arguments

|                          |                                                                                                                                                                                                                                         |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>df</code>          | A <code>data.frame</code> to inspect. Accepts any column types: numeric, integer, character, factor, logical, Date, POSIXct.                                                                                                            |
| <code>name_length</code> | Integer. Maximum number of characters displayed for column names and MIN/MAX values in the printed output. Longer strings are truncated. Defaults to <code>getOption("width") / 3</code> .                                              |
| <code>digits</code>      | Integer. Number of decimal places used when rounding MEAN, MEDIAN, and SD for numeric columns. Defaults to 2.                                                                                                                           |
| <code>nuniques</code>    | Integer. If $> 0$ , appends UNiques and LEVELS columns to the output. Columns with more distinct entries than <code>nuniques</code> are summarised as "N Uniques" / "N Levels". Set to 0 to skip (faster). Defaults to 0.               |
| <code>parallel</code>    | Logical. If TRUE, uses <code>future.apply</code> with a multisession plan across all available cores. Recommended for wide dataframes ( $> 100$ columns) or very large <code>n</code> . Defaults to FALSE.                              |
| <code>file</code>        | Character or NULL. If a string is provided, exports results to <code>&lt;file&gt;.xlsx</code> with two sheets: <code>variables</code> and <code>summary</code> . Any existing file with the same name is overwritten. Defaults to NULL. |

## Value

A named list with two elements:

`$summary` A single-row `data.frame` with whole-dataframe counts: `COLUMNS`, `ROWS`, `TOTAL`, `EMPTY`, `null`, `NAN`, `na`, `INF`, `FIN`, `FACTOR`.

`$check` A per-column `data.frame` with the following fields:

**NAMES** Column name (truncated to `name_length`).

**EMPTY** Count of "" empty strings.

**null** Count of NULL values (always 0 for dataframe columns).

**na** Count of NA values.

**NOT\_NA** Count of non-NA values.

**NAN** Count of NaN values.

**INF** Count of Inf and -Inf values.

**FIN** Count of finite values.

**RANGE** Number of distinct values.

**MEAN** Arithmetic mean, rounded to `digits`. NA for non-numeric columns.

**MEDIAN** Median, rounded to `digits`. NA for non-numeric columns.

**SD** Standard deviation, rounded to `digits`. NA for non-numeric columns.

**MIN** Minimum value or first label in sorted order.

**MAX** Maximum value or last label in sorted order.

**MODE** Storage mode as returned by `mode()`.

**TYPE** Type as returned by `typeof()`.

**CLASS** Class as returned by `class()`.

**FACTOR** Logical; TRUE if the column is a factor.



**Note**

MEAN, MEDIAN, and SD are NA for non-numeric columns. MIN and MAX for non-double columns are derived from `sort()` on character representations — natural sort ordering is not guaranteed for mixed alphanumeric strings.

**Examples**

```
cdff(df=mtcars,parallel=TRUE)
cdff(df=change_data_type(mtcars,"factor"),nuniques=3)
cdff(df=data.frame(t(mtcars)),file="mtcars",nuniques=10)
cdff(df=mtcars)
cdff(df=generate_missing(mtcars))
cdff(df=infert,nuniques=10)
cdff(df=infert)
df<-data.frame(infert,
               date=seq(as.Date("2010-1-1"),
                       as.Date("2020-1-1"),
                       length.out=nrow(infert)))
cdff(df=df)
```

cdff

*Check dataframe (optimised)***Description**

A faster equivalent of `cdff`. Produces an identical column-level diagnostic summary but avoids repeated passes over each column, eliminates row-by-row `rbind` calls, and removes the `gtools` and `plyr` dependencies. Recommended for large dataframes (> 100k rows or > 50 columns).

**Usage**

```
cdff(
  df,
  name_length = (getOption("width")/3),
  digits = 2,
  uniques = 0,
  parallel = FALSE,
  file = NULL
)
```

**Arguments**

|                          |                                                                                                                                                                                            |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>df</code>          | A <code>data.frame</code> to inspect. Accepts any column types: numeric, integer, character, factor, logical, Date, POSIXct.                                                               |
| <code>name_length</code> | Integer. Maximum number of characters displayed for column names and MIN/MAX values in the printed output. Longer strings are truncated. Defaults to <code>getOption("width") / 3</code> . |

|          |                                                                                                                                                                                                           |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| digits   | Integer. Number of decimal places used when rounding MEAN, MEDIAN, and SD for numeric columns. Defaults to 2.                                                                                             |
| nuniques | Integer. If > 0, appends UNIQUES and LEVELS columns to the output. Columns with more distinct entries than nuniques are summarised as "N Uniques" / "N Levels". Set to 0 to skip (faster). Defaults to 0. |
| parallel | Logical. If TRUE, uses future.apply with a multisession plan across all available cores. Recommended for wide dataframes (> 100 columns) or very large n. Defaults to FALSE.                              |
| file     | Character or NULL. If a string is provided, exports results to <file>.xlsx with two sheets: variables and summary. Any existing file with the same name is overwritten. Defaults to NULL.                 |

### Value

Identical structure to `cdf`: a named list with elements `$summary` and `$check`. See `cdf` for full field descriptions.

### Note

MIN and MAX for non-double columns use base `min()` / `max()` on character representations. Unlike `cdf`, mixed alphanumeric ordering (e.g. "V1" < "V10" < "V2") is *not* guaranteed — lexicographic order is used instead.

### Examples

```
cdff(df=mtcars,parallel=TRUE)
cdff(df=change_data_type(mtcars,"factor"),nuniques=3)
cdff(df=data.frame(t(mtcars)),file="mtcars",nuniques=10)
cdff(df=mtcars)
cdff(df=generate_missing(mtcars))
cdff(df=infert,nuniques=10)
cdff(df=infert)
df<-data.frame(infert,
               date=seq(as.Date("2010-1-1"),
                       as.Date("2020-1-1"),
                       length.out=nrow(infert)))
cdff(df=df)
```

---

cfa\_icc\_index

*index of items to convert from lavaan to thurstonian order for analysis*

---

### Description

index of items to convert from lavaan to thurstonian order for analysis

### Usage

```
cfa_icc_index(nitems, nfactors = 3)
```

**Arguments**

|          |                                      |
|----------|--------------------------------------|
| nitems   | number of items in the questionnaire |
| nfactors | number of factors                    |

**Examples**

```
cfa_icc_index(nitems=18,nfactors=3)
```

---

|                  |                                                  |
|------------------|--------------------------------------------------|
| change_data_type | <i>Convert column data types in a data frame</i> |
|------------------|--------------------------------------------------|

---

**Description**

Converts all or selected columns in a data frame to a specified data type. Whitespace (tabs, carriage returns, newlines) is trimmed automatically when converting to "character" or "numeric".

**Usage**

```
change_data_type(df, type)
```

**Arguments**

|      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| df   | A data frame whose columns will be converted.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| type | Character string specifying the conversion to apply:<br>"character" Converts all columns to character, trimming leading and trailing whitespace.<br>"numeric" Converts all columns to numeric (via character with whitespace trimming). Non-numeric strings become NA.<br>"factor" Converts all columns to factor.<br>"factor_character" Converts only factor columns to character; all other columns are left unchanged.<br>"character_factor" Converts only character columns to factor; all other columns are left unchanged. |

**Value**

A data frame with the same dimensions as df with column types converted as specified.

**Examples**

```
cdf(df=change_data_type(df=mtcars,"character"))
cdf(df=change_data_type(df=mtcars,"numeric"))
cdf(df=change_data_type(df=mtcars,"factor"))
df<-change_data_type(df=mtcars,"factor")
cdf(df=change_data_type(df=df,"factor_character"))
```

check\_heywood

*Check for Heywood Cases and Related SEM Estimation Problems***Description**

Screens a fitted lavaan model for common warning signs such as negative variances, impossible standardized values, unusually large standard errors, and convergence failure.

In simple terms: this is a quick model health check. It tells you whether your solution contains suspicious estimates that often indicate misspecification, weak identification, or numerical instability.

**Usage**

```
check_heywood(fit_model, verbose = TRUE)
```

**Arguments**

|                        |                                                                                                                                 |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <code>fit_model</code> | A fitted lavaan model object (for example, from <code>lavaan::cfa()</code> , <code>lavaan::sem()</code> , or related wrappers). |
| <code>verbose</code>   | Logical. If TRUE (default), prints diagnostic sections and a summary to the console. If FALSE, only returns results.            |

**Details**

The function checks:

- Negative variances (`~~` with `lhs == rhs` and `estimate < 0`).
- Negative residual variances in Thurstonian style parameters (`~*~` with `estimate < 0`).
- Standardized loadings outside `[-1, 1]`.
- Standardized correlations outside `[-1, 1]`.
- Extremely large standard errors (`se > 10`).
- Non-convergence.

A Heywood case usually refers to impossible estimates like negative variances or standardized loadings greater than 1 in absolute value.

**Value**

An invisible list with:

- `has_issues`: Logical, TRUE if any issue was detected.
- `issues`: Named list of detected issue tables/messages.
- `converged`: Logical convergence flag from `lavaan::lavInspect(fit_model, "converged")`.

**Examples**

```
library(lavaan)

# Example model
HS.model <- '
  visual =~ x1 + x2 + x3
  textual =~ x4 + x5 + x6
  speed   =~ x7 + x8 + x9
'

fit <- cfa(HS.model, data = HolzingerSwineford1939)

# Verbose diagnostic output
chk <- check_heywood(fit, verbose = TRUE)

# Programmatic use
check_heywood(fit, verbose = TRUE)
```

---

|                 |                         |
|-----------------|-------------------------|
| clear_stopwords | <i>Remove stopwords</i> |
|-----------------|-------------------------|

---

**Description**

Remove stopwords

**Usage**

```
clear_stopwords(text, stopwords = stopwords::stopwords("english"))
```

**Arguments**

|           |                           |
|-----------|---------------------------|
| text      | character vector          |
| stopwords | character words to remove |

**Examples**

```
text1<-"word_one word_two word_three"
text2<-"word_three word_four word_six"
text3<-"All the Lorem Ipsum generators on the Internet tend to repeat predefined
chunks as necessary, making this the first true generator on the Internet."
text4<-"It uses a dictionary of over 200 Latin words, combined with a handful of
model sentence structures, to generate Lorem Ipsum which looks reasonable."
text5<-"The generated Lorem Ipsum is therefore always free from repetition,
injected humour, or non-characteristic words etc."
stopwords<-stopwords::stopwords("english")
text<-c(text1,text2,text3,text4,text5)
clear_stopwords(text,stopwords=stopwords)
```

---

clear\_text

*Clear text*


---

### Description

Clear text

### Usage

```
clear_text(text)
```

### Arguments

text                      character vector

### Examples

```
text1<-"word_one word_two word_three"
text2<-"word_three word_four word_six"
text3<-"All the Lorem Ipsum generators on the Internet tend to repeat predefined
chunks as necessary, making this the first true generator on the Internet."
text4<-"It uses a dictionary of over 200 Latin words, combined with a handful of
model sentence structures, to generate Lorem Ipsum which looks reasonable."
text5<-"The generated Lorem Ipsum is therefore always free from repetition,
injected humour, or non-characteristic words etc."
text<-c(text1,text2,text3,text4,text5)
clear_text(text)
```

---

comparison\_combinations

*All pairwise column name combinations*


---

### Description

Generates a data frame of all pairwise combinations of column names from a data frame. Useful for programmatically specifying variable pairs to pass to functions like [compute\\_crosstable](#) or [plot\\_crosstable](#).

### Usage

```
comparison_combinations(df, all_orders = TRUE)
```

### Arguments

df                      A data frame whose column names will be combined.

all\_orders              Logical. When TRUE (default) both orderings of each pair are included (e.g. (X1, X2) and (X2, X1)), producing  $n(n - 1)$  rows for  $n$  columns. When FALSE only unique unordered pairs are returned, producing  $n(n - 1)/2$  rows.

**Value**

A data frame with two character columns X1 and X2, each row representing one variable pair.

**Examples**

```
comparison_combinations(generate_correlation_matrix(n=10)[,1:4])
```

---

|                 |                                                       |
|-----------------|-------------------------------------------------------|
| compute_ability | <i>Compute subject ability for thurstonian models</i> |
|-----------------|-------------------------------------------------------|

---

**Description**

Computes person ability for binary thurstonian coded items for a single dimension

**Usage**

```
compute_ability(
  response,
  eta,
  gamma,
  lambda,
  psi,
  plot = FALSE,
  map = compute_map(eta = eta, mean = 0, sd = 1)
)
```

**Arguments**

|          |                                                                  |
|----------|------------------------------------------------------------------|
| response | item responses                                                   |
| eta      | eta or ability                                                   |
| gamma    | gamma or threshold                                               |
| lambda   | lambda or loading                                                |
| psi      | psi or error                                                     |
| plot     | if TRUE plots icc curves using the plot_icc_thurstonian function |
| map      | vector from compute_map                                          |

**Examples**

```
gamma<-c(0.556,-1.253,-1.729,0.618,0.937,0.295,-0.672,-1.127,-0.446,0.632,1.147,0.498)
psi<-c(2.172,1.883,2.055,1.869,2.231,2.100,1.762,1.803,1.565,1.892,1.794,1.686)
lambda<-c(1.082,1.082,-1.297,-1.297,0.802,0.802,1.083,1.083)
gamma<-gamma[response_dimension(c(1:12),3,c(1,2))]
psi<-psi[response_dimension(c(1:12),3,c(1,2))]
eta<-seq(-6,6,by=0.1)
response1<-c(0,0,0,0,0,0,0,0,0,0,0,0)
response2<-c(1,1,1,1,1,1,1,1,1,1,1,1)
```

```

response3<-c(1,0,1,0,1,0,1,0)
response4<-c(0,1,0,1,0,1,0,1)
map<-compute_map(eta=eta,mean=0,sd=1)
compute_ability(response1,eta,gamma,lambda,psi,map=map,plot=FALSE)
compute_ability(response2,eta,gamma,lambda,psi,map=map,plot=FALSE)
compute_ability(response3,eta,gamma,lambda,psi,map=map,plot=FALSE)
compute_ability(response4,eta,gamma,lambda,psi,map=map,plot=FALSE)

```

---

|                    |                            |
|--------------------|----------------------------|
| compute_adjustment | <i>Compute adjustments</i> |
|--------------------|----------------------------|

---

### Description

Compute adjustments

### Usage

```
compute_adjustment(a, ntests)
```

### Arguments

|        |                 |
|--------|-----------------|
| a      | alpha criterion |
| ntests | number of tests |

### Examples

```
compute_adjustment(0.05,100)
```

---

|                   |                                                  |
|-------------------|--------------------------------------------------|
| compute_aggregate | <i>Aggregate descriptive statistics by group</i> |
|-------------------|--------------------------------------------------|

---

### Description

Computes a comprehensive set of descriptive statistics for all numeric columns in a data frame, stratified by one or more grouping variables. Unlike [compute\\_descriptives](#), this function operates on every numeric column simultaneously and returns results in a long format where each row corresponds to one statistic for one group combination. Results can be exported to an Excel file.

### Usage

```
compute_aggregate(df, iv, file = NULL)
```



**Arguments**

|             |                                                                                                                         |
|-------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>df</b>   | A data frame containing the variables of interest. All numeric columns that are not listed in <b>iv</b> are summarised. |
| <b>iv</b>   | Integer vector of column indices identifying the grouping variables used to stratify the output.                        |
| <b>file</b> | Character string naming the output Excel file (without extension). When NULL (default) no file is written.              |

**Value**

A data frame in long format with one row per statistic per group combination. The first column is **statistic** (see below), followed by the grouping variable columns, and then one column per numeric variable in **df**. The **statistic** column takes the following values:

**mean** Arithmetic mean.

**SD** Standard deviation.

**median** Median.

**mad** Median absolute deviation.

**trimmed mean** 50% trimmed mean.

**N** Number of non-missing observations.

**min** Minimum observed value.

**max** Maximum observed value.

**range** Difference between maximum and minimum.

**skewness** Skewness; deviations from 0 indicate departure from symmetry.

**kurtosis** Excess kurtosis; deviations from 0 indicate departure from normality.

**IQR** Interquartile range (Q0.75 - Q0.25).

**SE** Standard error of the mean.

**Examples**

```
compute_aggregate(df=mtcars, iv=9)
compute_aggregate(df=mtcars, iv=9:10)
compute_aggregate(df=mtcars, iv=9:11)
compute_aggregate(df=mtcars, iv=9:11, file="descriptives")
```

---

|                |                              |
|----------------|------------------------------|
| compute_aov_es | <i>Compute eta and omega</i> |
|----------------|------------------------------|

---

**Description**

Computes omega using aov object. Based on <http://stats.stackexchange.com/a/126520>

**Usage**

```
compute_aov_es(model, ss = "I")
```

**Arguments**

|       |                                                  |
|-------|--------------------------------------------------|
| model | object aov                                       |
| ss    | Character type of sums of squares "I" "II" "III" |

**Examples**

```
form<-formula(uptake~Treatment)
one_way_between<-aov(form,C02)
factorial_between<-aov(uptake~Treatment*Type,C02)
compute_aov_es(model=one_way_between,ss="I")
sjstats::anova_stats(one_way_between,digits=10)
compute_aov_es(model=one_way_between,ss="II")
sjstats::anova_stats(one_way_between,digits=10)
compute_aov_es(model=one_way_between,ss="III")
sjstats::anova_stats(one_way_between,digits=10)
compute_aov_es(model=factorial_between,ss="I")
sjstats::anova_stats(factorial_between,digits=10)
compute_aov_es(model=factorial_between,ss="II")
sjstats::anova_stats(factorial_between,digits=10)
compute_aov_es(model=factorial_between,ss="III")
sjstats::anova_stats(car::Anova(factorial_between,Type=3),digits=10)
```

---

|                             |                                    |
|-----------------------------|------------------------------------|
| compute_confidence_interval | <i>Compute confidence interval</i> |
|-----------------------------|------------------------------------|

---

**Description**

Compute confidence interval

**Usage**

```
compute_confidence_interval(vector)
```

**Arguments**

vector                      vector

**Examples**

```
set.seed(1)
vector<-rnorm(1000)
compute_confidence_inteval(vector)
```

---

compute\_crosstable      *Pairwise cross-tabulation of categorical variables*

---

**Description**

Computes contingency tables (frequency counts and percentages) for pairs of categorical variables. Variable pairs can be supplied explicitly via `combinations`, or all unique pairs within a set of columns can be generated automatically via `factor_index`. A progress bar is displayed during computation.

**Usage**

```
compute_crosstable(df, factor_index = NULL, combinations = NULL)
```

**Arguments**

**df**                      A data frame containing the variables to cross-tabulate.

**factor\_index**          Integer vector of column indices. When provided and `combinations` is `NULL`, all unique pairwise combinations of the selected columns are computed (self-pairs and duplicate pairs are excluded).

**combinations**          A data frame with two character columns named `index1` and `index2`, each row specifying one variable pair to cross-tabulate. Takes precedence over `factor_index`.

**Value**

A data frame with one row per combination of variable-pair levels, containing the following columns:

**f1** Name of the first variable.

**f2** Name of the second variable.

**l1** Level of the first variable.

**l2** Level of the second variable.

**Frequency** Observed count for the  $l1 \times l2$  cell.

**Percent** Cell count as a percentage of all observations in that variable pair ( $\text{Frequency} / \text{total} * 100$ ).

Variable pairs with zero total observations are silently dropped.

## Examples

```
combinations<-data.frame(index1=c("vs","am","gear"),index2=c("cyl","cyl","cyl"))
compute_crosstable(df=mtcars,combinations=combinations)
combinations<-data.frame(index1=c("vs","am"),index2=c("cyl","cyl"))
compute_crosstable(df=mtcars,combinations=combinations)
compute_crosstable(df=mtcars,factor_index=8:10)
```

---

compute\_descriptives    *Descriptive statistics*

---

## Description

Computes a comprehensive set of descriptive statistics for one or more continuous variables, optionally stratified by one or more grouping (independent) variables. When `iv` is supplied the statistics are computed separately for each combination of dependent and independent variable levels. Results can be exported to an Excel file.

## Usage

```
compute_descriptives(df, dv, iv = NULL, file = NULL)
```

## Arguments

|                   |                                                                                                                                                                                                 |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>df</code>   | A data frame containing the variables of interest.                                                                                                                                              |
| <code>dv</code>   | Integer vector of column indices identifying the dependent (continuous) variables to summarise.                                                                                                 |
| <code>iv</code>   | Integer vector of column indices identifying the independent (grouping) variables used to stratify the output. When <code>NULL</code> (default) the statistics are computed on the full sample. |
| <code>file</code> | Character string naming the output Excel file (without extension). When <code>NULL</code> (default) no file is written.                                                                         |

## Value

A data frame with one row per variable (and per group level when `iv` is supplied) containing the following columns:

**factor** Name of the grouping variable (`iv` only).

**levels** Level of the grouping variable (`iv` only).

**variable** Name of the dependent variable.

**n** Sample size (non-missing observations).

**mean** Arithmetic mean.

**sd** Standard deviation.

**median** Median.

**trimmed** 10% trimmed mean.  
**mad** Median absolute deviation.  
**min** Minimum observed value.  
**max** Maximum observed value.  
**range** Difference between maximum and minimum.  
**skew** Skewness; deviations from 0 indicate departure from symmetry.  
**kurtosis** Excess kurtosis; deviations from 0 indicate departure from normality.  
**se** Standard error of the mean.  
**IQR** Interquartile range (Q0.75 - Q0.25).  
**Q0.1, Q0.25, Q0.5, Q0.75, Q0.9** Percentiles at 10, 25, 50, 75, and 90.

### Examples

```
compute_descriptives(df=mtcars,dv=1:5)
compute_descriptives(df=mtcars,dv=1:2,iv=9:10)
compute_descriptives(df=mtcars,dv=1:2,file="descriptives_no_factor")
compute_descriptives(df=mtcars,dv=1:2,iv=9:10,file="descriptives_factor")
```

---

```
compute_dissatenuation
```

*Compute dissatenuation*

---

### Description

Compute dissatenuation

### Usage

```
compute_dissatenuation(variable1, error1, variable2, error2)
```

### Arguments

|           |                                        |
|-----------|----------------------------------------|
| variable1 | vector                                 |
| error1    | vector error measurement for variable1 |
| variable2 | vector                                 |
| error2    | vector error measurement for variable2 |

### Examples

```
set.seed(1)
compute_dissatenuation(rnorm(10),rnorm(10),rnorm(10),rnorm(10))
```

---

```
compute_dummy_comparisons
```

*Compute number of dummy comparisons*

---

### Description

Compute number of dummy comparisons

### Usage

```
compute_dummy_comparisons(items)
```

### Arguments

|       |                           |
|-------|---------------------------|
| items | number of items per block |
|-------|---------------------------|

### Examples

```
compute_dummy_comparisons(1)
compute_dummy_comparisons(2)
compute_dummy_comparisons(3)
compute_dummy_comparisons(4)
compute_dummy_comparisons(5)
compute_dummy_comparisons(6)
```

---

```
compute_frequencies
```

*Frequency table for categorical variables*

---

### Description

Computes frequency counts, proportions, and percentages for every column in a data frame. All columns are processed together and the results are stacked into a single long-format table. Missing values are excluded from the frequency counts via `table()`. Results can be exported to an Excel file.

### Usage

```
compute_frequencies(df, ordered = TRUE, file = NULL)
```

### Arguments

|         |                                                                                                                      |
|---------|----------------------------------------------------------------------------------------------------------------------|
| df      | A data frame whose columns are the categorical variables to tabulate. All columns are processed regardless of class. |
| ordered | Logical. When TRUE (default) the rows within each variable are sorted by frequency in descending order.              |
| file    | Character string naming the output Excel file (without extension). When NULL (default) no file is written.           |

**Value**

A data frame in long format with one row per observed level per variable, containing the following columns:

**variable** Name of the column from df.

**Observation** Observed level (category label).

**Frequency** Count of observations at that level.

**Proportion** Relative frequency (Frequency / total for that variable).

**Percent** Proportion multiplied by 100.

**Examples**

```
compute_frequencies(df=generate_missing(generate_factor(nrows=10,ncols=10),missing=5))
compute_frequencies(df=generate_factor())
compute_frequencies(df=generate_factor(),file="descriptives")
```

---

```
compute_icc_thurstonian
```

*Compute item characteristic curves for thurstonian models*

---

**Description**

Computes icc curves for binary thurstonian coded items for a single dimension

**Usage**

```
compute_icc_thurstonian(eta, gamma, lambda, psi, plot = FALSE)
```

**Arguments**

|        |                                                                  |
|--------|------------------------------------------------------------------|
| eta    | eta or ability                                                   |
| gamma  | gamma or threshold                                               |
| lambda | lambda or loading                                                |
| psi    | psi or error                                                     |
| plot   | if TRUE plots icc curves using the plot_icc_thurstonian function |

**Examples**

```
gamma<-c(0.556,-1.253,-1.729,0.618,0.937,0.295,-0.672,-1.127,-0.446,0.632,1.147,0.498)
psi<-c(2.172,1.883,2.055,1.869,2.231,2.100,1.762,1.803,1.565,1.892,1.794,1.686)
lambda<-c(1.082,1.082,-1.297,-1.297,0.802,0.802,1.083,1.083)
gamma<-gamma[response_dimension(c(1:12),3,c(1,2))]
psi<-psi[response_dimension(c(1:12),3,c(1,2))]
eta<-seq(-6,6,by=0.01)
compute_icc_thurstonian(eta=eta,gamma=gamma,lambda=lambda,psi=psi,plot=FALSE)
```

---

|                  |                                               |
|------------------|-----------------------------------------------|
| compute_info_1pl | <i>Compute item information for 1PL model</i> |
|------------------|-----------------------------------------------|

---

**Description**

Compute item information for 1PL model

**Usage**

```
compute_info_1pl(b, theta)
```

**Arguments**

|       |                              |
|-------|------------------------------|
| b     | numeric difficulty parameter |
| theta | numeric theta                |

**Examples**

```
compute_info_1pl(b=1, theta=-3)
compute_info_1pl(b=1, theta=-2)
compute_info_1pl(b=1, theta=-1)
compute_info_1pl(b=1, theta=0)
compute_info_1pl(b=1, theta=1)
compute_info_1pl(b=1, theta=2)
compute_info_1pl(b=1, theta=3)
ti<-compute_info_1pl(b=1, theta=seq(-6,6,by=.01)) # test information
plot(ti, x=seq(-6,6,by=.01))
```

---

|                  |                                               |
|------------------|-----------------------------------------------|
| compute_info_2pl | <i>Compute item information for 2PL model</i> |
|------------------|-----------------------------------------------|

---

**Description**

Compute item information for 2PL model

**Usage**

```
compute_info_2pl(a, b, theta)
```

**Arguments**

|       |                                  |
|-------|----------------------------------|
| a     | numeric discrimination parameter |
| b     | numeric difficulty parameter     |
| theta | numeric theta                    |



**Examples**

```

compute_info_2pl(a=1.5,b=1,theta=-3)
compute_info_2pl(a=1.5,b=1,theta=-2)
compute_info_2pl(a=1.5,b=1,theta=-1)
compute_info_2pl(a=1.5,b=1,theta=0)
compute_info_2pl(a=1.5,b=1,theta=1)
compute_info_2pl(a=1.5,b=1,theta=2)
compute_info_2pl(a=1.5,b=1,theta=3)
ti<-compute_info_2pl(a=1,b=-2,theta=seq(-6,6,by=.01)) # test information
plot(ti,x=seq(-6,6,by=.01))
ti<-compute_info_2pl(a=2,b=0,theta=seq(-6,6,by=.01)) # test information
plot(ti,x=seq(-6,6,by=.01))
ti<-compute_info_2pl(a=3,b=2,theta=seq(-6,6,by=.01)) # test information
plot(ti,x=seq(-6,6,by=.01))

```

---

|                  |                                               |
|------------------|-----------------------------------------------|
| compute_info_3pl | <i>Compute item information for 3PL model</i> |
|------------------|-----------------------------------------------|

---

**Description**

Compute item information for 3PL model

**Usage**

```
compute_info_3pl(a, b, g, theta)
```

**Arguments**

|       |                                  |
|-------|----------------------------------|
| a     | numeric discrimination parameter |
| b     | numeric difficulty parameter     |
| g     | numeric guessing parameter       |
| theta | numeric theta                    |

**Examples**

```

compute_info_3pl(a=1.5,b=1,g=.2,theta=-3)
compute_info_3pl(a=1.5,b=1,g=.2,theta=-2)
compute_info_3pl(a=1.5,b=1,g=.2,theta=-1)
compute_info_3pl(a=1.5,b=1,g=.2,theta=0)
compute_info_3pl(a=1.5,b=1,g=.2,theta=1)
compute_info_3pl(a=1.5,b=1,g=.2,theta=2)
compute_info_3pl(a=1.5,b=1,g=.2,theta=3)
ti<-compute_info_3pl(a=1.5,b=1,g=.2,theta=seq(-6,6,by=.01)) # test information
plot(ti,x=seq(-6,6,by=.01))

```

---

```
compute_kruskal_wallis_test
```

*Kruskal-Wallis Test with Effect Sizes*

---

## Description

Runs a one-way Kruskal-Wallis rank-sum test and returns the test statistic, p-value, and two effect sizes:

- etasq: eta-squared for Kruskal-Wallis ( $\eta_H^2$ )
- epsilonsq: epsilon-squared ( $\epsilon^2$ )

In simple terms, this tests whether groups differ in their distributions, and quantifies how large that group effect is.

## Usage

```
compute_kruskal_wallis_test(formula, df)
```

## Arguments

|         |                                                       |
|---------|-------------------------------------------------------|
| formula | A one-way formula in the form $y \sim \text{group}$ . |
| df      | A data frame containing the variables in formula.     |

## Details

etasq and epsilonsq are both in  $[0, 1]$  in typical use. Multiplying by 100 gives an approximate percentage-style interpretation of explained rank variance.

## Value

A one-row data frame with:

- formula: model formula used
- method: test name
- etasq: Kruskal-Wallis eta-squared,  $(H - k + 1)/(n - k)$
- epsilonsq: epsilon-squared,  $H/(n - 1)$
- H: Kruskal-Wallis chi-squared statistic
- df: degrees of freedom ( $k - 1$ )
- p: p-value

**Examples**

```

form<-formula(bp_before~agegrp)
kruskal.test(formula=form,data=df_blood_pressure)
rcompanion::epsilonSquared(x=df_blood_pressure$bp_before,
                           g=df_blood_pressure$agegrp,
                           group="row",
                           ci=TRUE,
                           conf=0.95,
                           type="perc",
                           R=1000,
                           digits=3)
rstatix::kruskal_effsize(df_blood_pressure,form,ci=TRUE,conf.level=0.95,ci.type="perc",nboot=100)
compute_kruskal_wallis_test(formula=form,df=df_blood_pressure)

```

---

|                  |                         |
|------------------|-------------------------|
| compute_kurtosis | <i>Compute kurtosis</i> |
|------------------|-------------------------|

---

**Description**

Compute kurtosis

**Usage**

```
compute_kurtosis(vector)
```

**Arguments**

vector                      vector

**Note**

$b_2 = m_4 / s^4 - 3 = (g_2 + 3) (1 - 1/n)^2 - 3$ . Used in MINITAB and BMDP.

**Examples**

```

set.seed(1)
vector<-rnorm(1000)
compute_kurtosis(vector)
e1071::kurtosis(vector)

```

---

 compute\_map

*Simulate prior distribution*


---

### Description

Simulate prior distribution

### Usage

```
compute_map(eta, mean = 0, sd = 1)
```

### Arguments

|      |         |
|------|---------|
| eta  | vector  |
| mean | numeric |
| sd   | numeric |

### Examples

```
eta<-seq(-6,6,by=0.1)
compute_map(eta=eta,mean=0,sd=1)
```

---

 compute\_moving\_average

*Centered moving average*


---

### Description

Smooths every numeric column of a data frame by replacing each value with the mean of a symmetric window of  $2w + 1$  observations (the  $w$  rows before, the row itself, and the  $w$  rows after). At the edges of the series the window is clipped to the available rows, so the average is computed over fewer observations.

### Usage

```
compute_moving_average(df, w)
```

### Arguments

|    |                                                                                                                                               |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------|
| df | A data frame whose columns will be smoothed. All columns should be numeric.                                                                   |
| w  | Integer half-window size. The total window width is $2w + 1$ . For example, $w = 5$ averages 11 consecutive rows centred on each observation. |

**Value**

A data frame with the same dimensions and column names as df where each value has been replaced by its centred moving average.

**Examples**

```
compute_moving_average(df=mtcars,w=5)
```

---

```
compute_one_way_test  one way test
```

---

**Description**

one way test

**Usage**

```
compute_one_way_test(formula, df, var.equal = TRUE)
```

**Arguments**

|           |                                                       |
|-----------|-------------------------------------------------------|
| formula   | A one-way formula in the form $y \sim \text{group}$ . |
| df        | A data frame containing the variables in formula.     |
| var.equal | if TRUE it assumes equal variances                    |

**Note**

eta and omega for Welch statistics are not adequately tested and they should not be consulted

**Examples**

```
form<-formula(bp_before~agegrp)
compute_one_way_test(formula=form,df=df_blood_pressure,var.equal=TRUE)
compute_one_way_test(formula=form,df=df_blood_pressure,var.equal=FALSE)
oneway.test(formula=form,data=df_blood_pressure,var.equal=TRUE)
oneway.test(formula=form,data=df_blood_pressure,var.equal=FALSE)
car::Anova(aov(form,data=df_blood_pressure),type=2)
model<-lm(form,data=df_blood_pressure)
lsr::etaSquared(aov(form,data=df_blood_pressure),type=3,anova=TRUE)
sjstats::anova_stats(model,digits=22)
```

---

|                 |                                          |
|-----------------|------------------------------------------|
| compute_posthoc | <i>Games Howell Tukey post hoc tests</i> |
|-----------------|------------------------------------------|

---

**Description**

Based on [http://www.psych.yorku.ca/cribbie/6130/games\\_howell.R](http://www.psych.yorku.ca/cribbie/6130/games_howell.R)

**Usage**

```
compute_posthoc(y, x)
```

**Arguments**

|   |                           |
|---|---------------------------|
| y | Vector continous variable |
| x | Vector factor             |

**Examples**

```
compute_posthoc(y=df_blood_pressure$bp_before,x=df_blood_pressure$agegrp)
compute_posthoc(y=df_blood_pressure$bp_after,x=df_blood_pressure$agegrp)
```

---

|                 |                              |
|-----------------|------------------------------|
| compute_power_r | <i>Compute r power curve</i> |
|-----------------|------------------------------|

---

**Description**

Compute r power curve

**Usage**

```
compute_power_r(
  n = 100,
  r = NULL,
  sig.level = 0.05,
  alternative = c("two.sided", "less", "greater"),
  title = "",
  base_size = 10
)
```

**Arguments**

|             |                                                                                                                     |
|-------------|---------------------------------------------------------------------------------------------------------------------|
| n           | number of observations                                                                                              |
| r           | correlation coefficient                                                                                             |
| sig.level   | alpha (type I error probability)                                                                                    |
| alternative | a character string specifying the alternative hypothesis, must be one of "two.sided" (default), "greater" or "less" |
| title       | plot title                                                                                                          |
| base_size   | base font size                                                                                                      |

**Examples**

```
compute_power_r(n=100,r=.5,sig.level=.05,alternative=c("two.sided"))
```

---

```
compute_power_r_matrix
```

*Compute correlation matrix*

---

**Description**

Compute correlation matrix

**Usage**

```
compute_power_r_matrix(m, ...)
```

**Arguments**

|     |                                     |
|-----|-------------------------------------|
| m   | correlation matrix                  |
| ... | arguments passed to compute_power_r |

**Examples**

```
compute_power_r_matrix(m=stats::cor(mtcars,use="pairwise.complete.obs"),n=100)
```

---

compute\_residual\_stats

*Residuals for matrices*

---

### Description

Root Mean Squared Residual Number of absolute residuals > 0.05 Proportion of absolute residuals > 0.05. It can either accept a psych EFA model or it can compare two correlation or covariance matrices

### Usage

```
compute_residual_stats(model, data = NULL)
```

### Arguments

|       |                                                                                      |
|-------|--------------------------------------------------------------------------------------|
| model | psych EFA model. It has to be a correlation or covariance matrix if data is not NULL |
| data  | correlation or covariance matrix                                                     |

### Examples

```
model<-psych::fa(mtcars,nfactors=2,rotate="oblimin",fm="pa",oblique.scores=TRUE)
compute_residual_stats(model)
```

---

compute\_scores

*Compute subject ability for thurstonian models*

---

### Description

Computes person ability for binary thurstonian coded items for a single dimension

### Usage

```
compute_scores(mydata, ...)
```

### Arguments

|        |                                     |
|--------|-------------------------------------|
| mydata | item responses                      |
| ...    | arguments passed to compute_ability |



**Examples**

```

gamma<-c(0.556,-1.253,-1.729,0.618,0.937,0.295,
         -0.672,-1.127,-0.446,0.632,1.147,0.498)
psi<-c(2.172,1.883,2.055,1.869,2.231,2.100,
       1.762,1.803,1.565,1.892,1.794,1.686)
lambda<-c(1.082,1.082,-1.297,-1.297,0.802,0.802,1.083,1.083)
gamma<-gamma[response_dimension(c(1:12),3,c(1,2))]
psi<-psi[response_dimension(c(1:12),3,c(1,2))]
eta<-seq(-6,6,by=0.1)
map<-compute_map(eta=eta,mean=0,sd=1)
response_df<-data.frame(matrix(nrow=0,ncol=8))
response_df[1,]<-c(0,0,0,0,0,0,0,0)
response_df[2,]<-c(1,1,1,1,1,1,1,1)
response_df[3,]<-c(1,0,1,0,1,0,1,0)
response_df[4,]<-c(0,1,0,1,0,1,0,1)
compute_scores(response_df,eta,gamma,lambda,psi,map=map,plot=FALSE)

```

---

|                  |                                |
|------------------|--------------------------------|
| compute_se_theta | <i>Compute the SE of theta</i> |
|------------------|--------------------------------|

---

**Description**

Compute the SE of theta

**Usage**

```
compute_se_theta(info)
```

**Arguments**

|      |                     |
|------|---------------------|
| info | numeric information |
|------|---------------------|

**Examples**

```

compute_se_theta(1)
ti<-compute_info_2pl(a=10,b=0,theta=seq(-3,3,by=.01)) # test information
plot(compute_se_theta(ti),x=seq(-3,3,by=.01))

```

---

|                  |                         |
|------------------|-------------------------|
| compute_skewness | <i>Compute skewness</i> |
|------------------|-------------------------|

---

**Description**

Compute skewness

**Usage**

```
compute_skewness(vector)
```

**Arguments**

|        |        |
|--------|--------|
| vector | vector |
|--------|--------|

**Note**

$b_1 = m_3 / s^3 = g_1 ((n-1)/n)^{3/2}$ . Used in MINITAB and BMDP.

**Examples**

```
set.seed(1)
vector<-rnorm(1000)
compute_skewness(vector)
e1071::skewness(vector)
```

---

|               |                                                               |
|---------------|---------------------------------------------------------------|
| compute_solve | <i>Solve Linear Systems or Invert a Matrix (Gauss-Jordan)</i> |
|---------------|---------------------------------------------------------------|

---

**Description**

Solves matrix equations of the form  $A X = B$  using Gauss-Jordan elimination with partial pivoting.

In simple terms: this function takes a square matrix A and finds X so that multiplying A by X gives B.

If B is not provided, it uses the identity matrix and returns the inverse of A.

**Usage**

```
compute_solve(a, b)
```

**Arguments**

|   |                                                                                                                           |
|---|---------------------------------------------------------------------------------------------------------------------------|
| a | Numeric square matrix A.                                                                                                  |
| b | Optional numeric vector or matrix B. Must have the same number of rows as A. If omitted, B is set to the identity matrix. |

## Details

Mathematical meaning:

$$AX = B$$

where A is known, B is known, and X is unknown.

Algebra meaning: each column of X is the solution to one linear system with the same A.

Computational method: the function builds the augmented matrix  $[A \mid B]$ , then applies row operations until the left side becomes the identity matrix. At that point, the right side is X.

Partial pivoting is used for better numerical stability: in each column, it swaps in the row with the largest absolute pivot.

The function stops with an error if A is singular (non-invertible).

## Value

If b is a matrix, returns matrix X solving  $A X = B$ .

If b is a vector, returns vector x solving  $A x = b$ .

If b is missing, returns the inverse of A.

## Examples

```
# Example 1: solve A x = b
A <- matrix(c(2, 1,
1, 3), nrow = 2, byrow = TRUE)
b <- c(1, 2)
x <- compute_solve(A, b)
x
# Check: A %% x should equal b
A %% x

# Example 2: solve A X = B (multiple right-hand sides)
B <- cbind(c(1, 2), c(0, 1))
X <- compute_solve(A, B)
X
# Check: A %% X should equal B
A %% X

# Example 3: inverse of A (when b is omitted)
A_inv <- compute_solve(A)
A_inv
# Check: A %% A_inv should be identity
A %% A_inv
```

---

|                  |                                |
|------------------|--------------------------------|
| compute_standard | <i>compute standard scores</i> |
|------------------|--------------------------------|

---

**Description**

compute standard scores

**Usage**

compute\_standard(vector, mean = 0, sd = 1, type = "z", input = "non\_standard")

**Arguments**

|        |                                                                                                                                              |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------|
| vector | vector                                                                                                                                       |
| mean   | numeric applicable to "uz"                                                                                                                   |
| sd     | numeric applicable to "uz"                                                                                                                   |
| type   | "z" "uz" "sten" "t" "stanine" "center" "center_reversed" "percent" "percentile" "scale_zero_one" "normal_density" "cumulative_density" "all" |
| input  | "standard" "non_standard" standard inputs are z scores and non standard are raw scores                                                       |

**Examples**

```
vector<-c(rnorm(10),NA,rnorm(10))
compute_standard(vector,type="z")
compute_standard(vector,mean=0,sd=1,type="uz")
compute_standard(vector,type="sten")
compute_standard(vector,type="t")
compute_standard(vector,type="stanine")
compute_standard(vector,type="center")
compute_standard(vector,type="center_reversed")
compute_standard(vector,type="percent")
compute_standard(vector,type="scale_zero_one")
ndf<-compute_standard(seq(-6,6,.01),mean=0,sd=1,type="normal_density")
plot(ndf)
cdf<-compute_standard(ndf,mean=0,sd=1,type="cumulative_density")
plot(cdf)
compute_standard(vector,type="all")
compute_standard(seq(-6,6,.1),type="all",input="standard")
```

---

`compute_standard_error`*Compute standard error*

---

**Description**

Compute standard error

**Usage**

```
compute_standard_error(vector)
```

**Arguments**

vector                      vector

**Examples**

```
set.seed(1)
vector<-rnorm(1000)
compute_standard_error(vector)
```

---

`compute_tversky_index`    *Compute the Tversky index*

---

**Description**

Computes the Tversky index between two sets, a generalisation of the Jaccard and Sørensen–Dice similarity coefficients. The index measures the overlap between  $x$  and  $y$  relative to their differences, weighted by  $\alpha$  and  $\beta$ .

The Tversky index is defined as:

$$T(x, y) = \frac{|x \cap y|}{|x \cap y| + \alpha |x \setminus y| + \beta |y \setminus x|}$$

Special cases:

- $\alpha = \beta = 0.5$  — Sørensen–Dice coefficient
- $\alpha = \beta = 1.0$  — Jaccard index

**Usage**

```
compute_tversky_index(x, y, alpha = 0.5, beta = 0.5)
```

**Arguments**

|       |                                                                                      |
|-------|--------------------------------------------------------------------------------------|
| x     | A vector. Coerced to character before comparison.                                    |
| y     | A vector. Coerced to character before comparison.                                    |
| alpha | Non-negative numeric. Weight applied to elements in x but not in y. Defaults to 0.5. |
| beta  | Non-negative numeric. Weight applied to elements in y but not in x. Defaults to 0.5. |

**Value**

A single numeric value in the range  $[0, 1]$ , where 0 indicates no overlap and 1 indicates identical sets.

**Note**

Both x and y are treated as *sets* — duplicate elements within each vector are ignored. Inputs are coerced to character before comparison, so 1L and "1" are treated as equal.

**See Also**

[intersect](#), [setdiff](#), [cdf](#)

**Examples**

```
x <- c("a", "b", "c", "d")
y <- c("b", "c", "d", "e")

# default (Sorensen-Dice)
compute_tversky_index(x, y)

# Jaccard index
compute_tversky_index(x, y, alpha = 1, beta = 1)

# asymmetric: penalise x-only elements more heavily
compute_tversky_index(x, y, alpha = 0.9, beta = 0.1)

# identical sets → 1
compute_tversky_index(x, x)

# disjoint sets → 0
compute_tversky_index(c("a", "b"), c("c", "d"))
```

---

compute\_unidimensional\_ability

*Compute theta for unidimensional models*


---

## Description

Compute theta for unidimensional models

## Usage

```
compute_unidimensional_ability(
  a,
  b,
  g = NULL,
  d = 1.702,
  u,
  lim_theta = c(-6, 6)
)
```

## Arguments

|           |                                                                         |
|-----------|-------------------------------------------------------------------------|
| a         | numeric vector discrimination parameters                                |
| b         | numeric vector difficulty parameters                                    |
| g         | numeric vector guessing parameters                                      |
| d         | numeric scaling constant usually it is a value that approximating 1.749 |
| u         | numeric vector responses                                                |
| lim_theta | vector minimum and maximum value of theta                               |

## Examples

```
a<-c(0.39,0.45,0.52,0.3,0.35,0.43,0.42,0.44,0.34,0.42)
b<-c(-1.96,-1.9,-1.38,-0.58,0.48,-0.81,-0.35,1.59,1.33,2.93)
u<-c(1,1,1,1,0,0,1,0,1,0)
# SHOULD RETURN 0.48402574251176
compute_unidimensional_ability(a=a,b=b,u=u,d=1.7,g=NULL)
a<-c(1.27,0.9,0.94,0.95,0.55,0.6,0.44,0.4)
b<-c(-0.54,0.18,0.21,1.26,1.73,-0.87,1.72,2.67)
u<-c(1,1,1,1,0,0,0,0)
# SHOULD RETURN 1.04621621510192
compute_unidimensional_ability(a=a,b=b,u=u,d=1.7,g=NULL)
a<-c(0.41,0.32,0.33,1.2,0.63,0.62,0.7,0.61,0.38,0.53,0.6,1.16)
b<-c(-1.4,-1.3,-1.17,0.2,0.71,0.86,-0.12,0.12,2.06,1.38,1.18,-0.33)
u<-c(1,0,1,1,0,0,0,1,1,0,1,0)
# SHOULD RETURN 0.0860506282671103
compute_unidimensional_ability(a=a,b=b,u=u,d=1.7,g=NULL)
```

---

 compute\_unidimensional\_theta

*Compute theta for unidimensional models*


---

## Description

Compute theta for unidimensional models

## Usage

```
compute_unidimensional_theta(a, b = 0, g = 0, i = 1, d = 1.702, theta = 0)
```

## Arguments

|       |                                                         |
|-------|---------------------------------------------------------|
| a     | numeric discrimination parameter                        |
| b     | numeric difficulty parameter                            |
| g     | numeric guessing parameter                              |
| i     | numeric inattentiveness parameter                       |
| d     | numeric scaling constant usually a value 1.749 or 1.702 |
| theta | numeric or vector theta                                 |

## Note

when scaling constant=1 it has no effect in equation  
 when inattentiveness=1 and guessing=0 function computes a 2PL score  
 when inattentiveness=1 and guessing!=0 function computes a 3PL score  
 when inattentiveness!=1 and guessing!=0 function computes a 4PL score

## Examples

```
compute_unidimensional_theta(a=10,b=0)
x<-seq(-3,3,by=.01)
plot(compute_unidimensional_theta(a=5,b=0,theta=x),x=x)
plot(compute_unidimensional_theta(a=5,b=-1,theta=x),x=x)
plot(compute_unidimensional_theta(a=5,b=1,theta=x),x=x)
plot(compute_unidimensional_theta(a=.1,b=0,theta=x),x=x)
plot(compute_unidimensional_theta(a=1,b=0,theta=x),x=x)
plot(compute_unidimensional_theta(a=10,b=0,theta=x),x=x)
plot(compute_unidimensional_theta(a=10,b=0,g=0,theta=x),x=x)
plot(compute_unidimensional_theta(a=10,b=0,g=.1,theta=x),x=x)
plot(compute_unidimensional_theta(a=10,b=0,g=.5,theta=x),x=x)
plot(compute_unidimensional_theta(a=10,b=0,g=0,i=1,theta=x),x=x)
plot(compute_unidimensional_theta(a=10,b=0,g=0,i=.9,theta=x),x=x)
plot(compute_unidimensional_theta(a=10,b=0,g=0,i=.6,theta=x),x=x)
```



---

|                    |                                        |
|--------------------|----------------------------------------|
| compute_y_logistic | <i>Compute y for logistic function</i> |
|--------------------|----------------------------------------|

---

**Description**

This function requires x range to produce a vector with y values

**Usage**

```
compute_y_logistic(intercept, coefficient, x)
```

**Arguments**

|             |         |
|-------------|---------|
| intercept   | Numeric |
| coefficient | Numeric |
| x           | Numeric |

**Examples**

```
x<--10:10
compute_y_logistic(0,1,x)
compute_y_logistic(0,1,1)
plot(x,compute_y_logistic(0,1,x),type="l");grid();abline(b=0,a=.5)
```

---

|           |                                                                      |
|-----------|----------------------------------------------------------------------|
| confusion | <i>Create a confusion matrix from observed and predicted vectors</i> |
|-----------|----------------------------------------------------------------------|

---

**Description**

Generates a confusion matrix from observed and predicted values.

**Usage**

```
confusion(observed, predicted)
```

**Arguments**

|           |                                                                      |
|-----------|----------------------------------------------------------------------|
| observed  | Vector of observed variables. These are the true class labels.       |
| predicted | Vector of predicted variables. These are the predicted class labels. |

## Details

This function creates a confusion matrix by comparing the observed (true) class labels with the predicted class labels. The confusion matrix is a table that is often used to describe the performance of a classification model. The function performs the following steps: 1. Identifies the unique class labels from both the observed and predicted vectors. 2. Sorts the class labels in a mixed order (if they are character variables) using 'gtools::mixedsort'. 3. Constructs a table to represent the confusion matrix with the sorted class labels as levels. The output is a confusion matrix, where rows represent the predicted class labels and columns represent the observed class labels.

## Examples

```
# Example with numeric observed and predicted values
confusion(observed=c(1,2,3,4,5,10),predicted=c(1,2,3,4,5,11))
# Example with repeated observed and predicted values
confusion(observed=c(1,2,2,2,2),predicted=c(1,1,2,2,2))
```

---

confusion\_matrix\_percent

*Confusion matrix with row and column percent*

---

## Description

Generates a confusion matrix from observed and predicted values, including row and column percentages.

## Usage

```
confusion_matrix_percent(observed, predicted)
```

## Arguments

|           |                                                                      |
|-----------|----------------------------------------------------------------------|
| observed  | Vector of observed variables. These are the true class labels.       |
| predicted | Vector of predicted variables. These are the predicted class labels. |

## Details

This function creates a confusion matrix by comparing the observed (true) class labels with the predicted class labels. Additionally, it calculates row and column percentages to provide a more detailed performance analysis.

The function performs the following steps: 1. Computes the confusion matrix from the observed and predicted values. 2. Calculates the overall accuracy by dividing the sum of diagonal elements by the total number of observations. 3. Appends row and column sums to the confusion matrix. 4. Computes precision and recall for each class and appends these metrics to the matrix. 5. Returns a formatted data frame with the confusion matrix, row and column percentages, and overall accuracy.

**Note**

Total measures-Accuracy:  $(TP+TN)/total$   
 Total measures-Prevalence:  $(TP+FN)/total$   
 Total measures-Proportion Incorrectly Classified:  $(FN+FP)/total$   
 Horizontal measures-True Positive Rate-Sensitivity:  $TP/(TP+FN)$   
 Horizontal measures-True Negative Rate-Specificity:  $TN/(FP+TN)$   
 Horizontal measures-False Negative Rate-Miss Rate:  $FN/(TP+FN)$   
 Horizontal measures-False Positive Rate-Fall-out:  $FP/(FP+TN)$   
 Vertical measures-Positive Predictive value-Precision:  $TP/(TP+FP)$   
 Vertical measures-Negative Predictive value:  $TN/(FN+TN)$   
 Vertical measures-False Omission Rate:  $FN/(FN+TN)$   
 Vertical measures-False Discovery Rate:  $FP/(TP+FP)$

**Examples**

```

# Example with numeric observed and predicted values
confusion_matrix_percent(observed=c(1,2,3,4,5,10),predicted=c(1,2,3,4,5,11))

# Example with repeated observed and predicted values
confusion_matrix_percent(observed=c(1,2,2,2,2),predicted=c(1,1,2,2,2))

# Example with random observed and predicted values
observed<-factor(round(rnorm(10000,m=10,sd=1)))
predicted<-factor(round(rnorm(10000,m=10,sd=1)))
confusion_matrix_percent(observed,predicted)

```

---

```
convert_excel_unix_timestamp
```

*Convert UNIX EXCEL timestamp*

---

**Description**

Convert UNIX EXCEL timestamp

**Usage**

```
convert_excel_unix_timestamp(timestamp)
```

**Arguments**

timestamp          unix or excel timestamp

**Examples**

```
convert_excel_unix_timestamp(1)
```

---

c\_bind

Column-bind data frames or vectors of unequal lengths

---

### Description

Combines any number of data frames or vectors side by side, padding shorter inputs with NA rows so all columns reach the same length. Each input's columns are prefixed with the object's name to avoid duplicate column names. Vectors are coerced to single-column data frames before binding.

### Usage

```
c_bind(..., first = TRUE)
```

### Arguments

|       |                                                                                                                                      |
|-------|--------------------------------------------------------------------------------------------------------------------------------------|
| ...   | Data frames or vectors to column-bind. Names are taken from the unevaluated expressions passed (e.g. variable names).                |
| first | Logical. When TRUE (default) NA padding rows are appended at the bottom of shorter inputs; when FALSE they are prepended at the top. |

### Value

A data frame with one column per column across all inputs, padded with NA rows to the length of the longest input. Column names follow the pattern <object\_name> for single-column inputs and <object\_name>\_<original\_colname> for multi-column inputs.

### Author(s)

Ananda Mahto

### Examples

```
c_bind(rnorm(10), rnorm(11), rnorm(12), rnorm(13))
```

---

data\_frame\_index

dataframe index

---

### Description

dataframe index

### Usage

```
data_frame_index(nrow, ncol)
```

Arguments

|      |                    |
|------|--------------------|
| nrow | number of rows     |
| ncol | number of collumns |

Examples

```
data_frame_index(5,5)
```

---

|                    |                                                         |
|--------------------|---------------------------------------------------------|
| decompose_datetime | <i>Decompose datetime objects to dataframe collumns</i> |
|--------------------|---------------------------------------------------------|

---

Description

Decompose datetime objects to dataframe collumns

Usage

```
decompose_datetime(  
  x,  
  format = "",  
  origin = "1970-01-01",  
  tz = "GMT",  
  extended = FALSE,  
  breaks = c(-1, 5, 13, 16, 20, 23),  
  ...  
)
```

Arguments

|          |                                                                                                                       |
|----------|-----------------------------------------------------------------------------------------------------------------------|
| x        | datetime object                                                                                                       |
| format   | date time format                                                                                                      |
| origin   | Starting date. The default is the unix time origin "1970-01-01"                                                       |
| tz       | Timezone                                                                                                              |
| extended | if TRUE it will display additional day time categories<br>WEEKDAY MONTH JULIAN QUARTER DAY_PERIOD                     |
| breaks   | Numeric vector Breaks define hour of day for classifiying into<br>"Night", "Morning", "Noon", "Afternoon", "Evening". |
| ...      | arguments passed to as.POSIXct This argument is used if extended=TRUE                                                 |

Examples

```
timestamp1<-as.numeric(as.POSIXct(Sys.Date()))
timestamp2<-as.numeric(as.POSIXct(Sys.time()))
d1<-Sys.Date()
d2<-Sys.time()
decompose_datetime(x=d1)
decompose_datetime(x=d2)
decompose_datetime(x=d1,extended=TRUE)
decompose_datetime(x=d2,extended=TRUE)
decompose_datetime(x="01/15/1900",format="%m/%e/%Y")
decompose_datetime(x="01/15/1900",format="%m/%e/%Y",extended=TRUE)
decompose_datetime(x=as.Date(as.POSIXct(10000,origin="1970-01-01")))
decompose_datetime(x=as.Date(as.POSIXct(timestamp1,origin="1970-01-01")),
                    format="%m/%e/%Y")
decompose_datetime(x=as.Date(as.POSIXct(timestamp2,origin="1970-01-01")),
                    format="%m/%e/%Y")
```

---

|         |                                   |
|---------|-----------------------------------|
| deg2rad | <i>Convert degrees to radians</i> |
|---------|-----------------------------------|

---

Description

Convert degrees to radians

Usage

```
deg2rad(degrees)
```

Arguments

degrees                      degrees

Examples

```
deg2rad(180)
```

---

|                |                                    |
|----------------|------------------------------------|
| detach_package | <i>Detach and unload a package</i> |
|----------------|------------------------------------|

---

Description

Removes a package from the R search path and unloads its namespace. If the package was attached more than once, all instances are removed. Does nothing if the package is not currently attached.

Usage

```
detach_package(package)
```

**Arguments**

**package** Character string giving the name of the package to detach (without the "package:" prefix), e.g. "ggplot2".

**Value**

Invisibly returns NULL. Called for its side effect of detaching the package.

---

|              |                       |
|--------------|-----------------------|
| df_admission | <i>Admission Data</i> |
|--------------|-----------------------|

---

**Description**

This data set contains information about graduate admission, including GRE scores, GPA, and the ranking of the undergraduate institution.

**Usage**

```
df_admission
```

**Format**

A data frame with 8 rows and 4 variables:

**admit** Binary variable indicating admission (0=No, 1=Yes)

**gre** GRE (Graduate Record Examination) score

**gpa** Grade Point Average

**rank** Ranking of the undergraduate institution (1=highest, 4=lowest)

**Source**

researchpy repo

**Examples**

```
data(df_admission)
head(df_admission)
```

---

df\_automotive\_data      *Automotive Data*


---

### Description

This data set contains various automotive information including engine location, dimensions, weight, engine type, number of cylinders, and other specifications.

### Usage

```
df_automotive_data
```

### Format

A data frame with 38 rows and 26 variables:

**symboling** Symboling code for the vehicle  
**normalized-losses** Normalized losses in the vehicle  
**make** Make of the vehicle  
**fuel-type** Type of fuel used (e.g., gas, diesel)  
**aspiration** Aspiration type (e.g., std, turbo)  
**num-of-doors** Number of doors (e.g., two, four)  
**body-style** Body style (e.g., sedan, hatchback, wagon)  
**drive-wheels** Drive wheels (e.g., fwd, rwd, 4wd)  
**engine-location** Location of the engine (e.g., front, rear)  
**wheel-base** Wheelbase of the vehicle in inches  
**length** Length of the vehicle in inches  
**width** Width of the vehicle in inches  
**height** Height of the vehicle in inches  
**curb-weight** Curb weight of the vehicle in pounds  
**engine-type** Type of engine (e.g., dohc, ohcv, ohc, l)  
**num-of-cylinders** Number of cylinders in the engine  
**engine-size** Size of the engine in cubic inches  
**fuel-system** Fuel system used (e.g., mpfi, 2bbl, mfi, 1bbl)  
**bore** Diameter of the cylinders in the engine  
**stroke** Stroke length of the engine  
**compression-ratio** Compression ratio of the engine  
**horsepower** Horsepower generated by the engine  
**peak-rpm** Peak RPM of the engine  
**city-mpg** Miles per gallon in the city  
**highway-mpg** Miles per gallon on the highway  
**price** Price of the vehicle



**Source**

Downloaded from Kaggle.com by the user Ramakrishnan Srinivasan. see <https://www.kaggle.com/toramky/automobile-dataset>

**Examples**

```
data(df_automotive_data)
head(df_automotive_data)
```

---

|                   |                            |
|-------------------|----------------------------|
| df_blood_pressure | <i>Blood Pressure Data</i> |
|-------------------|----------------------------|

---

**Description**

This data set contains blood pressure readings for patients before and after a certain treatment or intervention.

**Usage**

```
df_blood_pressure
```

**Format**

A data frame with 30 rows and 5 variables:

**patient** Unique identifier for each patient

**sex** Sex of the patient (e.g., Male, Female)

**agegrp** Age group of the patient (e.g., 30-45, 46-59)

**bp\_before** Blood pressure reading before the intervention

**bp\_after** Blood pressure reading after the intervention

**Source**

researchpy repo

**Examples**

```
data(df_blood_pressure)
head(df_blood_pressure)
```

df\_co2

*Carbon Dioxide Uptake in Grass Plants***Description**

The CO2 data frame has 84 rows and 5 columns of data from an experiment on the cold tolerance of the grass species *Echinochloa crus-galli*.

**Usage**

df\_co2

**Format**

A data frame with 84 rows and 5 variables:

**Plant** an ordered factor with levels Qn1 < Qn2 < Qn3 < ... < Mc1 giving a unique identifier for each plant). Used as a grouping factor.

**Type** a factor with levels Quebec Mississippi giving the origin of the plant

**Treatment** a factor with levels nonchilled chilled

**conc** a numeric vector of ambient carbon dioxide concentrations (mL/L)

**uptake** a numeric vector of carbon dioxide uptake rates (in  $\mu\text{mol}/\text{m}^2/\text{sec}$ )

**Details**

Grouped formulas like `uptake ~ conc | Plant` are useful in lattice graphics and mixed-effect models. The vertical bar ('|') separates the grouping variable. This allows modeling or plotting the response (uptake) versus the predictor (conc) within each level of Plant.

**Source**

Potvin, C., Lechowicz, M. J. and Tardif, S. (1990) "The statistical analysis of ecophysiological response curves obtained from experiments involving repeated measures", *Ecology*, 71, 1389–1400.  
Pinheiro, J. C. and Bates, D. M. (2000) *Mixed-effects Models in S and S-PLUS*, Springer.

**Examples**

```
data(df_co2)
head(df_co2)
```

---

|               |                        |
|---------------|------------------------|
| df_crop_yield | <i>Crop Yield Data</i> |
|---------------|------------------------|

---

**Description**

This data set contains information about crop yields based on different fertilizer types and water conditions.

**Usage**

```
df_crop_yield
```

**Format**

A data frame with 20 rows and 3 variables:

**Fert** Type of fertilizer used (A or B)

**Water** Watering condition (High or Low)

**Yield** Crop yield (in unspecified units)

**Source**

researchpy repo (simulated data, not real)

**Examples**

```
data(df_crop_yield)
head(df_crop_yield)
```

---

|              |                       |
|--------------|-----------------------|
| df_difficile | <i>Difficile Data</i> |
|--------------|-----------------------|

---

**Description**

This data set contains information about the impact of different doses on libido.

**Usage**

```
df_difficile
```

**Format**

A data frame with 15 rows and 3 variables:

**person** Unique identifier for each person

**dose** Dose received (e.g., 1, 2, 3)

**libido** Libido level of the person

**Source**

researchpy repo

**Examples**

```
data(df_difficile)
head(df_difficile)
```

---

df\_insurance

*Insurance Data*

---

**Description**

This data set contains information about insurance charges based on various factors such as age, sex, BMI, number of children, smoking status, and region.

**Usage**

```
df_insurance
```

**Format**

A data frame with 19 rows and 7 variables:

**age** Age of the individual

**sex** Sex of the individual (e.g., male, female)

**bmi** Body Mass Index of the individual

**children** Number of children covered by the insurance

**smoker** Smoking status (yes or no)

**region** Region where the individual resides (e.g., southwest, southeast, northwest, northeast)

**charges** Insurance charges

**Source**

researchpy repo

**Examples**

```
data(df_insurance)
head(df_insurance)
```

---

df\_ocean

*Big Five Personality Test Dataset*

---

**Description**

This dataset contains responses to an interactive online Big Five personality test conducted around 2012. Participants rated themselves on 50 personality statements, and also provided demographic and technical metadata. Responses were collected with informed consent, and missing data is coded as 0.

**Usage**

df\_ocean

**Format**

A data frame with 19719 rows and 57 variables:

**race** Race/ethnic background (1–13, 0=missing)

**age** Age (integer; only responses from participants 13 and older included)

**engnat** Is English your native language? (1=Yes, 2=No, 0=missing)

**gender** 1=Male, 2=Female, 3=Other, 0=missing

**hand** Dominant writing hand: 1=Right, 2=Left, 3=Both, 0=missing

**source** How participant came to the test site: 1=Internal link, 2=Google, 3=Facebook, 4=.edu site, 6=Other/unknown

**country** Two-letter ISO country code (e.g., "US", "GB")

**E1** I am the life of the party.

**E2** I don't talk a lot.

**E3** I feel comfortable around people.

**E4** I keep in the background.

**E5** I start conversations.

**E6** I have little to say.

**E7** I talk to a lot of different people at parties.

**E8** I don't like to draw attention to myself.

**E9** I don't mind being the center of attention.

**E10** I am quiet around strangers.

**N1** I get stressed out easily.

**N2** I am relaxed most of the time.

**N3** I worry about things.

**N4** I seldom feel blue.

**N5** I am easily disturbed.

- N6** I get upset easily.
- N7** I change my mood a lot.
- N8** I have frequent mood swings.
- N9** I get irritated easily.
- N10** I often feel blue.
- A1** I feel little concern for others.
- A2** I am interested in people.
- A3** I insult people.
- A4** I sympathize with others' feelings.
- A5** I am not interested in other people's problems.
- A6** I have a soft heart.
- A7** I am not really interested in others.
- A8** I take time out for others.
- A9** I feel others' emotions.
- A10** I make people feel at ease.
- C1** I am always prepared.
- C2** I leave my belongings around.
- C3** I pay attention to details.
- C4** I make a mess of things.
- C5** I get chores done right away.
- C6** I often forget to put things back in their proper place.
- C7** I like order.
- C8** I shirk my duties.
- C9** I follow a schedule.
- C10** I am exacting in my work.
- O1** I have a rich vocabulary.
- O2** I have difficulty understanding abstract ideas.
- O3** I have a vivid imagination.
- O4** I am not interested in abstract ideas.
- O5** I have excellent ideas.
- O6** I do not have a good imagination.
- O7** I am quick to understand things.
- O8** I use difficult words.
- O9** I spend time reflecting on things.
- O10** I am full of ideas.

## Details

Personality items were rated on a five-point Likert scale: #' 1=Disagree, 3=Neutral, 5=Agree. Missed=0.

**race** Chosen from a drop down menu. 1=Mixed Race, 2=Arctic (Siberian, Eskimo), 3=Caucasian (European), 4=Caucasian (Indian), 5=Caucasian (Middle East), 6=Caucasian (North African, Other), 7=Indigenous Australian, 8=Native American, 9=North East Asian (Mongol, Tibetan, Korean Japanese, etc), 10=Pacific (Polynesian, Micronesian, etc), 11=South East Asian (Chinese, Thai, Malay, Filipino, etc), 12=West African, Bushmen, Ethiopian, 13=Other (0=missed) **age** Entered as text (individuals reporting age < 13 were not recorded) **engnat** Response to "is English your native language?". 1=Yes, 2=No (0=missed) **gender** Chosen from a drop down menu. 1=Male, 2=Female, 3=Other (0=missed) **hand** "What hand do you use to write with?". 1=Right, 2=Left, 3=Both (0=missed)

On this page users were also asked to confirm that their answers were accurate and could be used for research. Participants who did not were not recorded). Some values were calculated from technical information.

**country** The participant's technical location. ISO country code. **source** How the participant came to the test. Based on HTTP Referrer. 1=from another page on the test website, 2=from google, 3=from facebook, 4=from any url with ".edu" in its domain name (e.g. xxx.edu, xxx.edu.au), 6=other source, or HTTP Referer not provided.

In psychological trait theory, the Big Five personality traits, also known as the five-factor model (FFM) and the OCEAN model, is a suggested taxonomy, or grouping, for personality traits, developed from the 1980s onwards. When factor analysis (a statistical technique) is applied to personality survey data, some words used to describe aspects of personality are often applied to the same person. For example, someone described as conscientious is more likely to be described as "always prepared" rather than "messy". This theory is based therefore on semantic associations between words and not on neuropsychological experiments. This theory uses descriptors of common language and suggests five broad dimensions commonly used to describe the human personality and psyche.

The theory identifies five factors:

- **Openness to experience (O)** (inventive/curious vs. consistent/cautious)
- **Conscientiousness (C)** (efficient/organized vs. extravagant/careless)
- **Extraversion (E)** (outgoing/energetic vs. solitary/reserved)
- **Agreeableness (A)** (friendly/compassionate vs. challenging/callous)
- **Neuroticism (N)** (sensitive/nervous vs. resilient/confident)

The five factors are represented using the acronyms OCEAN or CANOE. Beneath each proposed global factor, there are a number of correlated and more specific primary factors. For example, extroversion is typically associated with qualities such as gregariousness, assertiveness, excitement-seeking, warmth, activity, and positive emotions. Family life and the way someone was raised will affect these traits. Twin studies and other research have shown that about half of the variation between individuals results from their genetics and half from their environments. Researchers have found conscientiousness, extroversion, openness to experience, and neuroticism to be relatively stable from childhood through adulthood.

Items are grouped by Big Five traits:

- **Extraversion (E):** E1 to E10
- **Neuroticism (N):** N1 to N10
- **Agreeableness (A):** A1 to A10
- **Conscientiousness (C):** C1 to C10
- **Openness (O):** O1 to O10

Negatively keyed items are: E2, E4, E6, E8, E10, N2, N4, A1, A3, A5, A7, C2, C4, C6, C8, O2, O4, O6. These should be reverse-coded prior to scoring.

### Source

Collected via an online personality test with informed consent (~2012). Downloaded from Kaggle.com by the user Lucas Greenwell. see <https://www.kaggle.com/datasets/lucasgreenwell/ocean-five-factor-personality-test-responses>

### Examples

```
data(df_ocean)
head(df_ocean)

# Compute Big Five average scores (after reverse scoring)
# library(dplyr)
# df_ocean <- df_ocean %>% mutate(E=rowMeans(select(., E1:E10), na.rm=TRUE))
data(df_ocean)
head(df_ocean)
```

---

|                |                                                        |
|----------------|--------------------------------------------------------|
| df_personality | <i>Big Five Inventory (BFI-44) Personality Dataset</i> |
|----------------|--------------------------------------------------------|

---

### Description

Responses from 433 participants to the Big Five Inventory (BFI-44), a widely used measure of the five major dimensions of personality: Extraversion (E), Agreeableness (A), Conscientiousness (C), Neuroticism (N), and Openness to Experience (O). Items are rated on a 1-6 Likert scale. Reverse-scored items are marked with (R).

### Usage

```
df_personality
```

### Format

A data frame with 433 rows and 44 variables:

**pers01** [E] Is talkative

**pers02** [A-R] Tends to find fault with others

**pers03** [C] Does a thorough job



- pers04** [N] Is depressed, blue
- pers05** [O] Is original, comes up with new ideas
- pers06** [E-R] Is reserved
- pers07** [A] Is helpful and unselfish with others
- pers08** [C-R] Can be somewhat careless
- pers09** [N-R] Is relaxed, handles stress well
- pers10** [O] Is curious about many different things
- pers11** [E] Is full of energy
- pers12** [A-R] Starts quarrels with others
- pers13** [C] Is a reliable worker
- pers14** [N] Can be tense
- pers15** [O] Is ingenious, a deep thinker
- pers16** [E] Generates a lot of enthusiasm
- pers17** [A] Has a forgiving nature
- pers18** [C-R] Tends to be disorganized
- pers19** [N] Worries a lot
- pers20** [O] Has an active imagination
- pers21** [E-R] Tends to be quiet
- pers22** [A] Is generally trusting
- pers23** [C-R] Tends to be lazy
- pers24** [N-R] Is emotionally stable, not easily upset
- pers25** [O] Is inventive
- pers26** [E] Has an assertive personality
- pers27** [A-R] Can be cold and aloof
- pers28** [C] Perseveres until the task is finished
- pers29** [N] Can be moody
- pers30** [O] Values artistic, aesthetic experiences
- pers31** [E-R] Is sometimes shy, inhibited
- pers32** [A] Is considerate and kind to almost everyone
- pers33** [C] Does things efficiently
- pers34** [N-R] Remains calm in tense situations
- pers35** [O-R] Prefers work that is routine
- pers36** [E] Is outgoing, sociable
- pers37** [A-R] Is sometimes rude to others
- pers38** [C] Makes plans and follows through with them
- pers39** [N] Gets nervous easily
- pers40** [O] Likes to reflect, play with ideas
- pers41** [O-R] Has few artistic interests
- pers42** [A] Likes to cooperate with others
- pers43** [C-R] Is easily distracted
- pers44** [O] Is sophisticated in art, music, or literature

## References

John, O. P., & Srivastava, S. (1999). The Big Five trait taxonomy: History, measurement, and theoretical perspectives. In L. A. Pervin & O. P. John (Eds.), *Handbook of personality: Theory and research* (2nd ed., pp. 102-138). Guilford Press.

## Examples

```
data(df_personality)
head(df_personality)
```

---

|              |                                      |
|--------------|--------------------------------------|
| df_responses | <i>Young People Survey Responses</i> |
|--------------|--------------------------------------|

---

## Description

Survey responses from 1010 young people (15-30 years old) on music preferences, movie preferences, hobbies, phobias, health habits, personality traits, spending habits, and demographics.

## Usage

```
df_responses
```

## Format

A data frame with 1010 rows and 151 variables. All preference and personality items are rated on a 1-5 Likert scale (1=strongly disagree / not interested, 5=strongly agree / very interested) unless otherwise noted.

- **Participant Number:** Unique participant identifier
- **Music:** Interest in music generally
- **Slow songs or fast songs:** Preference for slow vs fast songs
- **Dance:** Interest in dance music
- **Folk:** Interest in folk music
- **Country:** Interest in country music
- **Classical music:** Interest in classical music
- **Musical:** Interest in musicals
- **Pop:** Interest in pop music
- **Rock:** Interest in rock music
- **Metal or Hardrock:** Interest in metal/hardrock
- **Punk:** Interest in punk music
- **Hiphop, Rap:** Interest in hiphop/rap
- **Reggae, Ska:** Interest in reggae/ska
- **Swing, Jazz:** Interest in swing/jazz

- **Rock n roll:** Interest in rock n roll
- **Alternative:** Interest in alternative music
- **Latino:** Interest in Latino music
- **Techno, Trance:** Interest in techno/trance
- **Opera:** Interest in opera
- **Movies:** Interest in movies generally
- **Horror:** Interest in horror films
- **Thriller:** Interest in thrillers
- **Comedy:** Interest in comedy films
- **Romantic:** Interest in romantic films
- **Sci-fi:** Interest in sci-fi films
- **War:** Interest in war films
- **Fantasy/Fairy tales:** Interest in fantasy films
- **Animated:** Interest in animated films
- **Documentary:** Interest in documentaries
- **Western:** Interest in westerns
- **Action:** Interest in action films
- **History:** Interest in history
- **Psychology:** Interest in psychology
- **Politics:** Interest in politics
- **Mathematics:** Interest in mathematics
- **Physics:** Interest in physics
- **Internet:** Interest in internet
- **PC:** Interest in computers
- **Economy Management:** Interest in economy/management
- **Biology:** Interest in biology
- **Chemistry:** Interest in chemistry
- **Reading:** Interest in reading
- **Geography:** Interest in geography
- **Foreign languages:** Interest in foreign languages
- **Medicine:** Interest in medicine
- **Law:** Interest in law
- **Cars:** Interest in cars
- **Art exhibitions:** Interest in art exhibitions
- **Religion:** Interest in religion
- **Countryside, outdoors:** Interest in countryside/outdoors
- **Dancing:** Interest in dancing

- **Musical instruments:** Interest in playing musical instruments
- **Writing:** Interest in writing
- **Passive sport:** Interest in watching sports
- **Active sport:** Interest in playing sports
- **Gardening:** Interest in gardening
- **Celebrities:** Interest in celebrities
- **Shopping:** Interest in shopping
- **Science and technology:** Interest in science and technology
- **Theatre:** Interest in theatre
- **Fun with friends:** Enjoyment of socialising with friends
- **Adrenaline sports:** Interest in adrenaline sports
- **Pets:** Interest in pets
- **Flying:** Fear of flying
- **Storm:** Fear of storms
- **Darkness:** Fear of darkness
- **Heights:** Fear of heights
- **Spiders:** Fear of spiders
- **Snakes:** Fear of snakes
- **Rats:** Fear of rats
- **Ageing:** Fear of ageing
- **Dangerous dogs:** Fear of dangerous dogs
- **Fear of public speaking:** Fear of public speaking
- **Smoking:** Smoking behaviour/attitude
- **Alcohol:** Alcohol consumption/attitude
- **Healthy eating:** Attitude toward healthy eating
- **Daily events:** Attitude toward planning daily events
- **Prioritising workload:** Ability to prioritise workload
- **Writing notes:** Habit of writing notes
- **Workaholism:** Tendency toward workaholism
- **Thinking ahead:** Tendency to think ahead
- **Final judgement:** Tendency to make final judgements
- **Reliability:** Self-rated reliability
- **Keeping promises:** Tendency to keep promises
- **Loss of interest:** Tendency to lose interest
- **Friends versus money:** Attitude toward friends vs money
- **Funniness:** Self-rated funniness
- **Fake:** Tendency to be fake

- **Criminal damage:** Attitude toward criminal damage
- **Decision making:** Decision-making style
- **Elections:** Attitude toward elections
- **Self-criticism:** Tendency toward self-criticism
- **Judgment calls:** Tendency to make judgment calls
- **Hypochondria:** Tendency toward hypochondria
- **Empathy:** Level of empathy
- **Eating to survive:** Attitude toward eating
- **Giving:** Tendency to give to others
- **Compassion to animals:** Compassion toward animals
- **Borrowed stuff:** Attitude toward returning borrowed items
- **Loneliness:** Tendency to feel lonely
- **Cheating in school:** Attitude toward cheating
- **Health:** Self-rated health
- **Changing the past:** Desire to change the past
- **God:** Belief in God
- **Dreams:** Belief in dreams
- **Charity:** Attitude toward charity
- **Number of friends:** Number of friends
- **Punctuality:** Self-rated punctuality
- **Lying:** Tendency to lie
- **Waiting:** Attitude toward waiting
- **New environment:** Comfort in new environments
- **Mood swings:** Tendency to have mood swings
- **Appearance and gestures:** Attention to appearance and gestures
- **Socializing:** Enjoyment of socializing
- **Achievements:** Drive for achievements
- **Responding to a serious letter:** Tendency to respond to serious letters
- **Children:** Attitude toward children
- **Assertiveness:** Level of assertiveness
- **Getting angry:** Tendency to get angry
- **Knowing the right people:** Value placed on knowing the right people
- **Public speaking:** Comfort with public speaking
- **Unpopularity:** Attitude toward unpopularity
- **Life struggles:** Attitude toward life struggles
- **Happiness in life:** Self-rated happiness
- **Energy levels:** Self-rated energy levels

- **Small - big dogs:** Preference for small vs big dogs
- **Personality:** Self-rated personality
- **Finding lost valuables:** Tendency to return lost valuables
- **Getting up:** Ease of getting up in the morning
- **Interests or hobbies:** Number of interests or hobbies
- **Parents' advice:** Attitude toward parents' advice
- **Questionnaires or polls:** Attitude toward questionnaires
- **Internet usage:** Hours of internet usage
- **Finances:** Self-rated financial management
- **Shopping centres:** Attitude toward shopping centres
- **Branded clothing:** Attitude toward branded clothing
- **Entertainment spending:** Spending on entertainment
- **Spending on looks:** Spending on looks
- **Spending on gadgets:** Spending on gadgets
- **Spending on healthy eating:** Spending on healthy eating
- **Age:** Age in years
- **Height:** Height in cm
- **Weight:** Weight in kg
- **Number of siblings:** Number of siblings
- **Gender:** 1=Female, 2=Male
- **Left - right handed:** 1=Right, 2=Left
- **Education:** 1=Primary, 2=Secondary, 3=College/University, 4=Masters, 5=Doctorate
- **Only child:** 1=Yes, 2=No
- **Village - town:** 1=Village, 2=City
- **House - block of flats:** 1=House, 2=Block of flats

### Source

Collected from students of Statistics at FSEV UK. Available on Kaggle.

### Examples

```
data(df_responses)
head(df_responses)
```

---

|                    |                             |
|--------------------|-----------------------------|
| df_responses_state | <i>Responses State Data</i> |
|--------------------|-----------------------------|

---

**Description**

This data set contains simulated state information paired with participant numbers from the responses data set.

**Usage**

```
df_responses_state
```

**Format**

A data frame with 28 rows and 2 variables:

**Participant Number** Unique identifier for each participant

**State** State code where the participant resides (e.g., MI, OH, CO, CA, MA, WA)

**Source**

researchpy repo (simulated data, not real)

**Examples**

```
data(df_responses_state)
head(df_responses_state)
```

---

|                |                                  |
|----------------|----------------------------------|
| df_sexual_comp | <i>Sexual Compatibility Data</i> |
|----------------|----------------------------------|

---

**Description**

Responses from 3,376 participants to a 10-item sexual compatibility questionnaire. The dataset is used as a teaching example in the researchpy Python statistics tutorials. The exact item wording is not publicly documented by the original source.

**Usage**

```
df_sexual_comp
```

**Format**

A data frame with 3376 rows and 13 variables. Each of Q1-Q10 is rated on a 0-4 scale; score is the unweighted sum (range 0-40). Note: age value 999 is a missing-data code; gender codes 0 and 3 are rare and likely represent missing or other responses.

**Q1** Sexual compatibility item 1 (0-4)

**Q2** Sexual compatibility item 2 (0-4)

**Q3** Sexual compatibility item 3 (0-4)

**Q4** Sexual compatibility item 4 (0-4)

**Q5** Sexual compatibility item 5 (0-4)

**Q6** Sexual compatibility item 6 (0-4)

**Q7** Sexual compatibility item 7 (0-4)

**Q8** Sexual compatibility item 8 (0-4)

**Q9** Sexual compatibility item 9 (0-4)

**Q10** Sexual compatibility item 10 (0-4)

**score** Total scale score (sum of Q1-Q10, range 0-40)

**gender** Gender (1=Male, 2=Female; 0 and 3 = missing/other)

**age** Age in years (999 = missing)

**Source**

researchpy Data-sets repository (<https://github.com/researchpy/Data-sets>)

**Examples**

```
data(df_sexual_comp)
head(df_sexual_comp)
```

---

df\_titanic

*Titanic Dataset*


---

**Description**

A dataset containing information about passengers on the Titanic.

**Usage**

```
df_titanic
```



**Format**

A data frame with the following variables:

**PassengerId** Unique identifier for each passenger  
**survived** Survival status (0=No, 1=Yes)  
**pclass** Passenger class (1=1st, 2=2nd, 3=3rd)  
**name** Name of the passenger  
**sex** Gender of the passenger  
**age** Age of the passenger  
**sibsp** Number of siblings/spouses aboard the Titanic  
**parch** Number of parents/children aboard the Titanic  
**ticket** Ticket number  
**fare** Passenger fare  
**cabin** Cabin number  
**embarked** Port of embarkation (C=Cherbourg; Q=Queenstown; S=Southampton)  
**boat** Lifeboat number  
**body** Body number  
**home.dest** Home destination

**Examples**

```
data(df_titanic)
head(df_titanic)
```

---

```
display_upper_lower_triangle
```

*Return upper diagonal from one matrix and lower diagonal from another matrix*

---

**Description**

Return upper diagonal from one matrix and lower diagonal from another matrix

**Usage**

```
display_upper_lower_triangle(m_upper, m_lower, diagonal = NA)
```

**Arguments**

|          |                                                                                                                                                 |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| m_upper  | matrix                                                                                                                                          |
| m_lower  | matrix                                                                                                                                          |
| diagonal | if "upper" it returns upper diagonal if "lower" it returns lower diagonal if NA returns NA in diagonal otherwise it returns any value specified |

## Examples

```
m1<-matrix(1:9,nrow=3,ncol=3)
m2<-matrix(11:19,nrow=3,ncol=3)
display_upper_lower_triangle(m_upper=m1,m_lower=m2,diagonal="upper")
display_upper_lower_triangle(m_upper=m1,m_lower=m2,diagonal="lower")
display_upper_lower_triangle(m_upper=m1,m_lower=m2,diagonal=NA)
display_upper_lower_triangle(m_upper=m1,m_lower=m2,diagonal=1)
display_upper_lower_triangle(m_upper=m1,m_lower=m2,diagonal=c("X1","X2","X3"))
display_upper_lower_triangle(m_upper=m1,m_lower=m2,diagonal=c(1,2,3))
display_upper_lower_triangle(m_upper=m1,m2)
```

---

dotnames

*Pad a data frame to a target number of rows with NAs*

---

## Description

Extends a data frame to rowsneeded rows by appending (or prepending) NA-filled rows. Internal helper used by [c\\_bind](#).

## Usage

```
dotnames(...)
```

## Arguments

|            |                                                                                                            |
|------------|------------------------------------------------------------------------------------------------------------|
| df         | A data frame to pad.                                                                                       |
| rowsneeded | Integer target row count. Must be greater than or equal to nrow(df).                                       |
| first      | Logical. When TRUE (default) NA rows are appended at the bottom; when FALSE they are prepended at the top. |

## Value

A data frame with rowsneeded rows and the same columns as df.

## Author(s)

Ananda Mahto

---

|             |                                   |
|-------------|-----------------------------------|
| drop_levels | <i>Drops unused factor levels</i> |
|-------------|-----------------------------------|

---

**Description**

Drops unused factor levels

**Usage**

```
drop_levels(df, factor_index = NULL, minimum_frequency = 5)
```

**Arguments**

|                   |                                                                                                                                     |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| df                | dataframe                                                                                                                           |
| factor_index      | numeric index of factors. If NULL the function uses is.factor() to discriminate factors                                             |
| minimum_frequency | the minimum frequency each factor will have, levels with frequency bellow or equal to the defined frequency will be renamed "Other" |

**Examples**

```
factor1<-factor(c(rep("A",10),rep("B",10)),levels=c("A","B","C","D"))
factor2<-factor(c(rep("A",10),rep("B",10)),levels=c("A","B","C","D"))
numeric1<-c(1:20)
df<-data.frame(numeric1,factor1,factor2)
df$factor1
drop_levels(df=df,minimum_frequency=9)
drop_levels(df=df,minimum_frequency=10)
```

---

|               |                                                                                    |
|---------------|------------------------------------------------------------------------------------|
| dummy_arrange | <i>Takes a vector with multiple responses and dummy arranges it in a dataframe</i> |
|---------------|------------------------------------------------------------------------------------|

---

**Description**

Takes a vector with multiple responses and dummy arranges it in a dataframe

**Usage**

```
dummy_arrange(vector)
```

**Arguments**

|        |        |
|--------|--------|
| vector | Vector |
|--------|--------|

**Examples**

```

vector1<-gsub(" ", "",
              generate_multiple_responce_vector(responces=c("Agree", "Hi", "All"),
              responded=1:3, length=10), fixed=TRUE)
vector2<-gsub(" ", "",
              generate_multiple_responce_vector(responces=1:4, responded=1:4, length=10),
              fixed=TRUE)
vector3<-sample(1:4, 10, replace=TRUE)
vector4<-sample(LETTERS[1:3], 10, replace=TRUE)
dummy_arrange(vector1)
dummy_arrange(vector2)
dummy_arrange(vector3)
dummy_arrange(vector4)

```

---

|                  |                                                           |
|------------------|-----------------------------------------------------------|
| duplicate_y_axis | <i>Duplicate the y axis on the right side of a ggplot</i> |
|------------------|-----------------------------------------------------------|

---

**Description**

Takes two ggplot objects and renders p1 with the y axis of p2 mirrored onto the right side. Useful when overlaying two series with different scales or simply to frame the plot with matching axes on both sides.

**Usage**

```
duplicate_y_axis(p1, p2)
```

**Arguments**

|    |                                                                                             |
|----|---------------------------------------------------------------------------------------------|
| p1 | A ggplot object. This plot is drawn with the duplicated right axis.                         |
| p2 | A ggplot object whose left y axis is mirrored to the right of p1. Typically the same as p1. |

**Value**

Invisibly returns NULL. The combined plot is drawn to the current graphics device.

**Examples**

```

p1 <- ggplot(ChickWeight, aes(x=Time, y=weight, colour=Diet, group=Chick)) +
  geom_line() +
  ggtitle("Growth curve for individual chicks")
duplicate_y_axis(p1=p1, p2=p1)

```

---

|                     |                                 |
|---------------------|---------------------------------|
| environment_options | <i>Load environment options</i> |
|---------------------|---------------------------------|

---

**Description**

Load environment options

**Usage**

```
environment_options()
```

**Examples**

```
environment_options()
```

---

|                        |
|------------------------|
| excel_confusion_matrix |
|------------------------|

*Write matrix or dataframe to excel sheet*

---

**Description**

Usefull for correlation matrices since it uses conditional formatting for matrices

**Usage**

```
excel_confusion_matrix(
  df,
  workbook,
  title = "Rows: Expected Collumns: Observed"
)
```

**Arguments**

|          |                     |
|----------|---------------------|
| df       | dataframe or matrix |
| workbook | workbook            |
| title    | comment             |

**Examples**

```
filename<-"excel_confusion_matrix.xlsx"
if (file.exists(filename)) file.remove(filename)
observed<-factor(round(rnorm(10000,m=10,sd=1)))
predicted<-factor(round(rnorm(10000,m=10,sd=1)))
confusion(observed,predicted)
cm<-confusion_matrix_percent(observed,predicted)
wb<-openxlsx::createWorkbook()
excel_confusion_matrix(cm,wb)
openxlsx::saveWorkbook(wb,invisible(paste(filename)),TRUE)
```

---

excel\_critical\_value    *Write matrix or dataframe to excel sheet*

---

### Description

Usefull for generic data where conditional formating of a spesific collumn is required

### Usage

```
excel_critical_value(
  df,
  workbook,
  sheet = "output",
  title = NULL,
  comment = NULL,
  numFmt = "#0.00",
  critical = NULL
)
```

### Arguments

|          |                                                                            |
|----------|----------------------------------------------------------------------------|
| df       | dataframe or matrix                                                        |
| workbook | workbook                                                                   |
| sheet    | sheet                                                                      |
| title    | title                                                                      |
| comment  | comment                                                                    |
| numFmt   | number formatting                                                          |
| critical | list in the form of (collumn1=critical_value1,collumn2=critical_value2...) |

### Examples

```
comment<-list(mpg="Miles/(US) gallon",
              cyl="Number of cylinders",
              disp="Displacement (cu.in.)",
              hp="Gross horsepower",
              drat="Rear axle ratio",
              wt="Weight (1000 lbs)",
              qsec="1/4 mile time",
              vs="Engine (0=V-shaped,1=straight)",
              am="Transmission (0=automatic,1>manual)",
              gear="Number of forward gears",
              carb="Number of carburetors",
              extra_comment1="test1",
              extra_comment2="test2")
filename<-"excel_critical_value.xlsx"
if (file.exists(filename)) file.remove(filename)
wb<-openxlsx::createWorkbook()
```

```

df<-generate_missing(generate_correlation_matrix())
critical<-list(X1="<0.05",X5="<0")
excel_critical_value(df=df,workbook=wb,sheet="critical",comment=list(X1="test"),
                    numFmt="#0.00",critical=critical)
openxlsx::saveWorkbook(wb,invisible(paste(filename)),TRUE)
filename<-"excel_critical_value_comment.xlsx"
if (file.exists(filename)) file.remove(filename)
wb<-openxlsx::createWorkbook()
df<-generate_missing(mtcars)
critical<-list(mpg=">20",am="=0")
excel_critical_value(df=df,workbook=wb,sheet="critical",comment=comment,
                    numFmt="#0.00",critical=critical)
openxlsx::saveWorkbook(wb,invisible(paste(filename)),TRUE)
filename<-"excel_critical_value_comment_min_max.xlsx"
if (file.exists(filename)) file.remove(filename)
wb<-openxlsx::createWorkbook()
df<-generate_missing(mtcars)
critical<-list(mpg=c(">20","<11"),am="=0")
excel_critical_value(df=df,workbook=wb,sheet="critical",comment=comment,
                    numFmt="#0.00",critical=critical)
openxlsx::saveWorkbook(wb,invisible(paste(filename)),TRUE)

```

---

excel\_generic\_format    *Generic function for creating workbooks and worksheets*

---

## Description

This function is used by excel\_matrix and excel\_critical\_value functions

## Usage

```

excel_generic_format(
  df,
  workbook,
  sheet = "output",
  title = NULL,
  comment = NULL,
  numFmt = "#0.00"
)

```

## Arguments

|          |                     |
|----------|---------------------|
| df       | dataframe or matrix |
| workbook | workbook            |
| sheet    | sheet               |
| title    | title               |
| comment  | comment             |
| numFmt   | number formatting   |

## Examples

```
comment<-list(mpg="Miles/(US) gallon",
              cyl="Number of cylinders",
              disp="Displacement (cu.in.)",
              hp="Gross horsepower",
              drat="Rear axle ratio",
              wt="Weight (1000 lbs)",
              qsec="1/4 mile time",
              vs="Engine (0=V-shaped,1=straight)",
              am="Transmission (0=automatic,1>manual)",
              gear="Number of forward gears",
              carb="Number of carburetors",
              extra_comment1="test1",
              extra_comment2="test2")
mtcor<-data.frame(cor(mtcars))
filename<-"excel_generic.xlsx"
if (file.exists(filename)) file.remove(filename)
wb<-openxlsx::createWorkbook()
openxlsx::addWorksheet(wb,"sheet")
openxlsx::addWorksheet(wb,"correlation")
openxlsx::writeData(wb,sheet="sheet",x=mtcars,colNames=TRUE,rowNames=TRUE)
openxlsx::writeData(wb,sheet="correlation",x=mtcor,colNames=TRUE,rowNames=TRUE)
excel_generic_format(df=mtcars,workbook=wb,sheet="sheet",title="test",
                    comment=comment,numFmt="#0.00")
excel_generic_format(df=mtcor,workbook=wb,sheet="correlation",title="correlation",
                    comment=comment,numFmt="#0.00")
openxlsx::saveWorkbook(wb,invisible(paste(filename)),TRUE)
```

---

excel\_matrix

*Write matrix or dataframe to excel sheet*

---

## Description

Usefull for corellation matrices. It uses conditional formatting for matrices,which outlines high and low values using background color

## Usage

```
excel_matrix(
  df,
  workbook,
  sheet = "output",
  title = NULL,
  comment = NULL,
  numFmt = "#0.00",
  conditional_formatting = FALSE,
  diagonal = FALSE,
  diagonal_length = nrow(df)
)
```



**Arguments**

|                        |                                                 |
|------------------------|-------------------------------------------------|
| df                     | dataframe or matrix                             |
| workbook               | workbook                                        |
| sheet                  | sheet                                           |
| title                  | title                                           |
| comment                | comment                                         |
| numFmt                 | number formatting                               |
| conditional_formatting | if TRUE it will use conditional formatting      |
| diagonal               | if TRUE it will add background fill to diagonal |
| diagonal_length        | length of diagonal for background fill          |

**Examples**

```
comment<-list(mpg="Miles/(US) gallon",
              cyl="Number of cylinders",
              disp="Displacement (cu.in.)",
              hp="Gross horsepower",
              drat="Rear axle ratio",
              wt="Weight (1000 lbs)",
              qsec="1/4 mile time",
              vs="Engine (0=V-shaped,1=straight)",
              am="Transmission (0=automatic,1=manual)",
              gear="Number of forward gears",
              carb="Number of carburetors",
              extra_comment1="test1",
              extra_comment2="test2")

mtcor<-data.frame(cor(mtcars))
filename<-"excel_matrix.xlsx"
if (file.exists(filename)) file.remove(filename)
wb<-openxlsx::createWorkbook()
excel_matrix(mtcars,wb,sheet="matrix",comment=comment,
             conditional_formatting=TRUE,diagonal=FALSE)
excel_matrix(mtcars,wb,sheet="diagonal_non_square",comment=comment,
             conditional_formatting=FALSE,diagonal=TRUE)
excel_matrix(mtcars[1:10,1:10],wb,sheet="diagonal_square",comment=comment[1:10],
             conditional_formatting=FALSE,diagonal=TRUE)
excel_matrix(mtcars,wb,sheet="matrix_diagonal_non_square",comment=comment,
             conditional_formatting=TRUE,diagonal=TRUE)
excel_matrix(mtcars[1:10,1:10],wb,sheet="matrix_diagonal_square",comment=comment[1:10],
             conditional_formatting=TRUE,diagonal=TRUE)
excel_matrix(mtcors,wb,sheet="r",comment=comment,
             conditional_formatting=FALSE,diagonal=FALSE)
excel_matrix(mtcors,wb,sheet="conditional_formatting_r",comment=comment,
             conditional_formatting=TRUE,diagonal=TRUE)
openxlsx::saveWorkbook(wb,invisible(paste(filename)),TRUE)
```

---

|                    |                                                                |
|--------------------|----------------------------------------------------------------|
| extract_components | <i>Extract and plot variance components from a mixed model</i> |
|--------------------|----------------------------------------------------------------|

---

## Description

Extracts variance components from a mixed linear model fitted with `mixlm::lm()` and computes each component's percentage contribution to total variance. Results are returned as both a summary data frame and a horizontal bar chart, making it easy to identify which sources of variance (e.g. person, item, time, interactions) dominate the measurement design.

## Usage

```
extract_components(model, title = "")
```

## Arguments

|                    |                                                                                                                                                                                                               |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>model</code> | A mixed model object returned by <code>mixlm::lm()</code> . The model must include random effects specified with <code>r()</code> so that variance components are available via <code>mixlm::Anova()</code> . |
| <code>title</code> | Character string used as the plot title. Default is <code>""</code> .                                                                                                                                         |

## Value

A named list with two elements:

**components** A data frame with one row per variance component containing columns `component` (effect name), `VC` (estimated variance component), and `vc_percent` (percentage of total absolute variance explained by that component).

**plot** A ggplot horizontal bar chart displaying `vc_percent` for each component, with a line and points overlaid to show the profile across components.

## Examples

```
design<-expand.grid(time=1:3,item=1:3,person=1:10)
design<-change_data_type(design,type="factor")
design$response<-rowSums(change_data_type(design[,1:2],type="numeric"))+rnorm(90,0,0.1)
model<-mixlm::lm(response~r(time)*r(person)+r(item)*r(person),data=design)
extract_components(model)
```

---

|                     |                                                                      |
|---------------------|----------------------------------------------------------------------|
| extract_tirt_params | <i>Extract Thurstonian IRT Parameters from a lavaan-Fitted Model</i> |
|---------------------|----------------------------------------------------------------------|

---

## Description

Extracts and aligns the core parameter blocks needed for Thurstonian IRT scoring from a fitted lavaan object (as stored in `fit_lavaan_obj$fit`). The returned pieces are aligned to the row order of Lambda, which is critical for correct downstream scoring.

Specifically, this function returns:

- Lambda: factor loading matrix.
- theta\_diag: residual variances (diagonal of theta), reordered to `rownames(Lambda)` when names are available.
- tau: thresholds from tau, with row suffixes like "t1" removed and reordered to `rownames(Lambda)`.
- nu: indicator intercepts; defaults to zero when unavailable.
- Psi: latent covariance matrix (psi).

## Usage

```
extract_tirt_params(fit_lavaan_obj)
```

## Arguments

`fit_lavaan_obj` A fitted object containing a lavaan fit in `$fit`. For example, an object produced by a wrapper that stores a lavaan model under `fit_lavaan_obj$fit`.

## Details

Threshold rows in lavaan are often named like "item|t1". This function strips everything after "|" before matching thresholds to indicators.

## Value

A named list with elements: Lambda, theta\_diag, tau, nu, and Psi.

## Examples

```
library(thurstonianIRT)
data("triplets")
# define the blocks of items
blocks <-
  set_block(c("i1", "i2", "i3"), traits = c("t1", "t2", "t3"),
            signs = c(1, 1, 1)) +
  set_block(c("i4", "i5", "i6"), traits = c("t1", "t2", "t3"),
            signs = c(-1, 1, 1)) +
  set_block(c("i7", "i8", "i9"), traits = c("t1", "t2", "t3"),
            signs = c(1, 1, -1)) +
```

```

    set_block(c("i10", "i11", "i12"), traits = c("t1", "t2", "t3"),
              signs = c(1, -1, 1))
# generate the data to be understood by 'thurstonianIRT'
triplets_long <- make_TIRT_data(
  data = triplets, blocks = blocks, direction = "larger",
  format = "pairwise", family = "bernoulli", range = c(0, 1)
)
# fit the data using lavaan
fit <- fit_TIRT_lavaan(triplets_long)
pars <- extract_tirt_params(fit)

```

fixed

*Mark a pattern as a fixed string*

## Description

Flags a pattern to be interpreted as a literal string rather than a regular expression. Pass the result to `str_replace`, `str_replace_all`, `str_count`, or `str_split_fixed` wherever you want exact character matching instead of regex matching.

Flags a pattern to be interpreted as a literal string rather than a regular expression. Pass the result to `str_replace`, `str_replace_all`, `str_count`, or `str_split_fixed` wherever you want exact character matching instead of regex matching.

## Usage

```
fixed(pattern)
```

```
fixed(pattern)
```

## Arguments

`pattern`                      A character string to match literally.

## Value

The same character string with class `"fixed_pattern"`.

The same character string with class `"fixed_pattern"`.

## Examples

```

# Without fixed(), "." matches any character (regex)
str_replace_all("a.b.c", ".", "-")

# With fixed(), "." matches only a literal dot
str_replace_all("a.b.c", fixed("."), "-")
# Without fixed(), "." matches any character (regex)
str_replace_all("a.b.c", ".", "-")

# With fixed(), "." matches only a literal dot
str_replace_all("a.b.c", fixed("."), "-")

```

---

|              |                                     |
|--------------|-------------------------------------|
| flatten_list | <i>Flatten two dimensional list</i> |
|--------------|-------------------------------------|

---

**Description**

Flatten two dimensional list

**Usage**

```
flatten_list(mydata)
```

**Arguments**

|        |                          |
|--------|--------------------------|
| mydata | list with two dimensions |
|--------|--------------------------|

---

|                             |                                    |
|-----------------------------|------------------------------------|
| generate_comparisons_matrix | <i>Generate comparisons matrix</i> |
|-----------------------------|------------------------------------|

---

**Description**

Generate comparisons matrix

**Usage**

```
generate_comparisons_matrix(items)
```

**Arguments**

|       |                 |
|-------|-----------------|
| items | number of items |
|-------|-----------------|

**Examples**

```
generate_comparisons_matrix(2)
generate_comparisons_matrix(3)
generate_comparisons_matrix(4)
generate_comparisons_matrix(5)
generate_comparisons_matrix(6)
```

---

`generate_correlation_matrix`*Generate dataframe which outputs a predetermined correlation matrix*

---

**Description**

Generate dataframe which outputs a predetermined correlation matrix

**Usage**

```
generate_correlation_matrix(correlation_matrix, nrow = 10)
```

**Arguments**

|                                 |                                           |
|---------------------------------|-------------------------------------------|
| <code>correlation_matrix</code> | correlation matrix of resulting dataframe |
| <code>nrow</code>               | number of rows to generate                |

**Examples**

```
df<-data.frame(matrix(.999,ncol=2,nrow=2))
correlation_matrix<-as.matrix(df)
diag(correlation_matrix)<-1
df<-generate_correlation_matrix(correlation_matrix,nrow=100)
stats::cor(df)
```

---

`generate_data`*Generate dataframe with random numbers*

---

**Description**

Generate dataframe with random numbers

**Usage**

```
generate_data(
  nrow = 10,
  ncol = 5,
  mean = 0,
  sd = 1,
  min = 1,
  max = 5,
  type = "normal"
)
```

**Arguments**

|       |                                         |
|-------|-----------------------------------------|
| nrows | number of rows to generate              |
| ncols | number of columns to generate           |
| mean  | mean of generated vectors               |
| sd    | standard deviation of generated vectors |
| min   | minimum value in generated vector       |
| max   | maximum value in generated vector       |
| type  | character "normal" "uniform"            |

**Examples**

```
generate_data(nrows=10,ncols=5,mean=0,sd=1,type="normal")
generate_data(nrows=10,ncols=5,min=1,max=5,type="uniform")
```

---

|                 |                                      |
|-----------------|--------------------------------------|
| generate_factor | <i>Generate dataframe of factors</i> |
|-----------------|--------------------------------------|

---

**Description**

Generate dataframe of factors

**Usage**

```
generate_factor(vector = LETTERS[1:5], nrows = 2, ncols = 10, type = "random")
```

**Arguments**

|        |                                                                                                               |
|--------|---------------------------------------------------------------------------------------------------------------|
| vector | factor pool                                                                                                   |
| nrows  | number of rows to generate                                                                                    |
| ncols  | number of columns to generate                                                                                 |
| type   | "balanced" or "random" "balanced" generates balanced factor vectors, "random" generates random factor vectors |

**Examples**

```
generate_factor(vector=LETTERS[1:5],ncols=5,nrows=10,type="random")
generate_factor(vector=LETTERS[1:5],ncols=5,nrows=10,type="balanced")
generate_factor(vector=LETTERS[1:5],ncols=1,nrows=10,type="balanced")
generate_factor(vector=LETTERS[1:5],ncols=1,nrows=10,type="random")
```

---

|                   |                          |
|-------------------|--------------------------|
| generate_matrix_A | <i>Generate Matrix A</i> |
|-------------------|--------------------------|

---

**Description**

Generate Matrix A

**Usage**

generate\_matrix\_A(blocks = 3, items = 3)

**Arguments**

- |        |                           |
|--------|---------------------------|
| blocks | number of blocks          |
| items  | number of items per block |

**Examples**

generate\_matrix\_A(blocks=3,items=3)

---

|                            |                                                                   |
|----------------------------|-------------------------------------------------------------------|
| generate_matrix_lambda_hat | <i>Generate matrix lambda for spesified number of comparisons</i> |
|----------------------------|-------------------------------------------------------------------|

---

**Description**

Generate matrix lambda for spesified number of comparisons

**Usage**

generate\_matrix\_lambda\_hat(blocks = 3, items = 3)

**Arguments**

- |        |                           |
|--------|---------------------------|
| blocks | number of blocks          |
| items  | number of items per block |

**Examples**

generate\_matrix\_lambda\_hat(blocks=3,items=4)



---

|                  |                              |
|------------------|------------------------------|
| generate_missing | <i>Generate missing data</i> |
|------------------|------------------------------|

---

**Description**

Generate missing data

**Usage**

```
generate_missing(df, missing = 5)
```

**Arguments**

|         |                                   |
|---------|-----------------------------------|
| df      | vector or dataframe               |
| missing | number of missing data per vector |

**Examples**

```
generate_missing(rnorm(10),missing=5)
generate_missing(generate_data(nrow=10,ncol=2),missing=5)
```

---

|                                   |                                          |
|-----------------------------------|------------------------------------------|
| generate_multiple_responce_vector | <i>Generate multiple responce vector</i> |
|-----------------------------------|------------------------------------------|

---

**Description**

Generate multiple responce vector

**Usage**

```
generate_multiple_responce_vector(
  responce = 1:4,
  responded = 1:4,
  length = 10
)
```

**Arguments**

|           |                                            |
|-----------|--------------------------------------------|
| responce  | unique categories allowed                  |
| responded | number of categories observed in iteration |
| length    | length of returned vector                  |

**Examples**

```
generate_multiple_responce_vector(responce=1:4,responded=1:4,length=10)
```

---

|                 |                                |
|-----------------|--------------------------------|
| generate_string | <i>Generate random strings</i> |
|-----------------|--------------------------------|

---

**Description**

Generate random strings

**Usage**

```
generate_string(  
  vector = c(LETTERS, letters, 0:9),  
  vector_length = 1,  
  nchar = 5  
)
```

**Arguments**

|               |                               |
|---------------|-------------------------------|
| vector        | character pool                |
| vector_length | number of strings to generate |
| nchar         | Length of generated strings   |

**Examples**

```
generate_string(nchar=10)  
generate_string(nchar=10,vector_length=10)
```

---

|                                   |                                              |
|-----------------------------------|----------------------------------------------|
| generate_unique_comparisons_index | <i>Generate index for unique comparisons</i> |
|-----------------------------------|----------------------------------------------|

---

**Description**

Generate index for unique comparisons

**Usage**

```
generate_unique_comparisons_index(items)
```

**Arguments**

|       |                 |
|-------|-----------------|
| items | number of items |
|-------|-----------------|

**Examples**

```
generate_unique_comparisons_index(1)
generate_unique_comparisons_index(2)
generate_unique_comparisons_index(3)
generate_unique_comparisons_index(4)
generate_unique_comparisons_index(5)
generate_unique_comparisons_index(6)
```

---

`getfwp`*Get the file path of the currently running script*

---

**Description**

Returns the normalised absolute path of the R script that is currently executing. The function tries the following methods in order:

1. The `--file=` command-line argument (set when running via `Rscript script.R`).
2. The `fileName` variable in the first call-stack frame (set by some IDEs).
3. The `ofile` variable in the first call-stack frame (set when the script is loaded with `source()`).
4. The active document path from `rstudioapi` (RStudio only).
5. The source editor path from `rstudioapi` as a fallback.

**Usage**

```
getfwp()
```

**Value**

A character string containing the normalised absolute path of the current script, or an empty string ("" ) if the path cannot be determined.

**Examples**

```
#getfwp()
```

---

`get_script_directory`    *Get script directory*

---

### Description

Returns the directory of the currently active script as a string with a trailing slash. Works across multiple environments: RStudio, command line execution, and generic R sessions.

### Usage

```
get_script_directory()
```

### Details

The function tries three approaches in order:

1. If RStudio is available, uses `rstudioapi` to get the active document path
2. If running from the command line via `Rscript --file=`, parses the file argument
3. Falls back to `getwd()` as a last resort

### Value

A character string with the directory path, always ending with `"/"`

### Note

The fallback to `getwd()` may not reflect the script's actual location if the working directory has been changed during the session.

### Examples

```
# Returns the directory of the active script in RStudio
directory <- get_script_directory()
directory
```

---

`hinvert_title_grob`    *Invert a title grob horizontally*

---

### Description

Helper used by `duplicate_y_axis` to mirror a grob's width layout and text alignment so it renders correctly on the right-hand side of a plot.

### Usage

```
hinvert_title_grob(grob)
```

**Arguments**

grob                      A grob object (titleGrob or similar) to invert.

**Value**

The modified grob.

---

|         |                                            |
|---------|--------------------------------------------|
| icc_cfa | <i>Select responses for each dimension</i> |
|---------|--------------------------------------------|

---

**Description**

Select responses for each dimension

**Usage**

```
icc_cfa(eta, gamma, lambda, psi)
```

**Arguments**

eta                      eta or ability  
gamma                    gamma or threshold  
lambda                   lambda or loading  
psi                      psi or error

**Examples**

```
icc_cfa(seq(-6,6,.1),1,1,1)
```

---

|                |                              |
|----------------|------------------------------|
| increase_index | <i>index dataframe picks</i> |
|----------------|------------------------------|

---

**Description**

index dataframe picks

**Usage**

```
increase_index(blocks, items)
```

**Arguments**

blocks                   number of blocks  
items                    number of items per block

**Examples**

```
increase_index(3,3)
```

---

|                      |                                          |
|----------------------|------------------------------------------|
| install_all_packages | <i>Install all missing CRAN packages</i> |
|----------------------|------------------------------------------|

---

**Description**

Compares the set of currently installed packages against the full list of packages available on CRAN and installs any that are missing. Already-installed packages are not re-downloaded or updated. Note that CRAN contains thousands of packages, so this function can take a very long time and requires a large amount of disk space.

**Usage**

```
install_all_packages()
```

**Value**

Invisibly returns NULL. Called for its side effect of installing packages.

---

|              |                                           |
|--------------|-------------------------------------------|
| install_load | <i>Install and load multiple packages</i> |
|--------------|-------------------------------------------|

---

**Description**

Checks whether each package in package is already installed. Missing packages are downloaded from CRAN with dependencies = TRUE and then loaded. Packages that are already installed are loaded directly without reinstalling.

**Usage**

```
install_load(package)
```

**Arguments**

|         |                                                                    |
|---------|--------------------------------------------------------------------|
| package | Character vector of package names to install (if needed) and load. |
|---------|--------------------------------------------------------------------|

**Value**

A named logical vector (one element per package) indicating whether each package was successfully attached: TRUE if loaded, FALSE if loading failed.

**Author(s)**

Steven Worthington

**Examples**

```
install_load("car")
install_load(c("car", "ggplot2"))
```

---

|                  |                                                        |
|------------------|--------------------------------------------------------|
| key_to_cfa_model | <i>Convert a key list to a lavaan CFA model string</i> |
|------------------|--------------------------------------------------------|

---

### Description

Converts the named list key format used by [report\\_alpha](#) into a lavaan model syntax string suitable for passing directly to `lavaan::cfa()`. Each list element becomes one factor definition line of the form `factorname =~ item1+item2+...`

### Usage

```
key_to_cfa_model(key)
```

### Arguments

|     |                                                                                                                                                                                                                           |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| key | A named list where each name is a factor (trait) label and each element is a character vector of item column names belonging to that factor, e.g. <code>list(f1 = c("x1", "x2", "x3"), f2 = c("x4", "x5", "x6"))</code> . |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### Value

A single character string containing the full lavaan model specification with one factor definition per line.

### Examples

```
population_model<- 't1=~x1+.5*x2+.5*x3
                    t2=~x4+.5*x5+.5*x6
                    t3=~x7+.5*x8+.5*x9'
model_data<-lavaan::simulateData(population_model,sample.nobs=1000)
key<-list(f1=paste0("x",1:3),f2=paste0("x",4:6),f3=paste0("x",7:9))
model<-key_to_cfa_model(key)
fit<-lavaan::cfa(model,model_data)
```

---

|        |                                   |
|--------|-----------------------------------|
| k_fold | <i>K-Fold train test sampling</i> |
|--------|-----------------------------------|

---

### Description

Splits a dataframe into train and test dataframes for model evaluation. Prepared data include data objects for `xgboost`.

### Usage

```
k_fold(df, model_formula, k = 10)
```

**Arguments**

|               |                                                                 |
|---------------|-----------------------------------------------------------------|
| df            | Dataframe containing the dataset to be split.                   |
| model_formula | Model formula specifying the predictors and outcome variable.   |
| k             | Integer value representing the number of folds. Defaults to 10. |

**Details**

This function performs k-fold cross-validation by splitting the input dataframe into k folds. Each fold serves as a test set once, while the remaining k-1 folds form the training set.

The function prepares data objects for xgboost model training and evaluation, including train/test datasets and xgboost DMatrix objects.

The output is a list containing the following elements: -'f': List of train and test datasets for each fold. -'index': Vector of fold indices. -'model\_formula': Model formula used for generating the datasets. -'variables': Names of the variables in the model formula. -'predictors': Names of the predictor variables. -'outcome': Name of the outcome variable. -'xgb': List of xgboost DMatrix objects for training and testing.

**Examples**

```
# Example with the 'infert' dataset
infert_formula<-formula(case~education+spontaneous+induced)
result<-k_fold(infert,k=10,model_formula=infert_formula)

# Example with the 'mtcars' dataset
model_formula<-as.formula(mpg~cyl+disp+hp+drat+wt+qsec+vs+am+gear+carb)
result<-k_fold(mtcars,k=2,model_formula=model_formula)
```

---

|          |                            |
|----------|----------------------------|
| k_sample | <i>Train test sampling</i> |
|----------|----------------------------|

---

**Description**

Splits a dataframe into train and test dataframes for model evaluation. Prepared data include data objects for xgboost.

**Usage**

```
k_sample(df, model_formula, k = 1)
```

**Arguments**

|               |                                                                                   |
|---------------|-----------------------------------------------------------------------------------|
| df            | Dataframe containing the dataset to be split.                                     |
| model_formula | Model formula specifying the predictors and outcome variable.                     |
| k             | Integer value representing the number of folds. Defaults to 1 (train-test split). |



Details

This function performs k-fold cross-validation or a simple train-test split (if k=1) by splitting the input dataframe into k folds. Each fold serves as a test set once, while the remaining k-1 folds form the training set.

The function prepares data objects for xgboost model training and evaluation, including train, test, and validation datasets and xgboost DMatrix objects.

The output is a list containing the following elements: -'f': List of train, test, and validation datasets for each fold. -'index': Vector of fold indices. -'model\_formula': Model formula used for generating the datasets. -'variables': Names of the variables in the model formula. -'predictors': Names of the predictor variables. -'outcome': Name of the outcome variable. -'xgb': List of xgboost DMatrix objects for training, testing, and validation.

Examples

```
# Example with the 'infert' dataset
infert_formula<-formula(case~education+spontaneous+induced)
result<-k_sample(df=infert,k=10,model_formula=infert_formula)

# Example with the 'mtcars' dataset
model_formula<-formula(mpg~cyl+disp+hp+drat+wt+qsec+vs+am+gear+carb)
result<-k_sample(df=mtcars,k=10,model_formula=model_formula)
```

---

|                 |                                              |
|-----------------|----------------------------------------------|
| matrix_triangle | <i>Return upper or lower matrix triangle</i> |
|-----------------|----------------------------------------------|

---

Description

Return upper or lower matrix triangle

Usage

```
matrix_triangle(m, off_diagonal = NA, diagonal = NULL, type = "lower")
```

Arguments

|              |                                                                     |
|--------------|---------------------------------------------------------------------|
| m            | matrix                                                              |
| off_diagonal | off diagonal value                                                  |
| diagonal     | diagonal value. If NULL it returns the diagonal of the input matrix |
| type         | "upper" displays upper triangle, "lower" displays lower triangle    |

Examples

```
m<-matrix(1:9,nrow=3,ncol=3)
matrix_triangle(m=m)
matrix_triangle(m=m,diagonal=NA,type="lower")
matrix_triangle(m=m,diagonal=NULL,type="lower")
matrix_triangle(m=m,diagonal=NA,type="upper")
matrix_triangle(m=m,diagonal=NULL,type="upper")
```

---

|               |                                    |
|---------------|------------------------------------|
| mean_sd_alpha | <i>Mean and SD of scale scores</i> |
|---------------|------------------------------------|

---

### Description

Computes the mean and standard deviation of respondent scale scores. When divisor is NULL the scale score is the row mean across items. When divisor is supplied the scale score is the row sum divided by divisor, which is useful when scores need to be expressed on a custom metric (e.g. dividing a sum score by the maximum possible score to obtain a proportion).

### Usage

```
mean_sd_alpha(df, divisor = NULL)
```

### Arguments

|         |                                                                                                                             |
|---------|-----------------------------------------------------------------------------------------------------------------------------|
| df      | A data frame whose columns are the items of a single scale. All columns must be numeric.                                    |
| divisor | Numeric scalar used to divide the row sum before computing the mean and SD. When NULL (default) row means are used instead. |

### Value

A one-row data frame with columns MEAN and SD containing the mean and standard deviation of the scale scores across respondents.

### Examples

```
set.seed(12345)
df<-data.frame(matrix(.5,ncol=6,nrow=6))
correlation_matrix<-as.matrix(df)
diag(correlation_matrix)<-1
df<-round(generate_correlation_matrix(correlation_matrix,nrows=1000),0)+5
mean_sd_alpha(df)
mean_sd_alpha(df,divisor=100)
```

---

|       |                                  |
|-------|----------------------------------|
| mgsub | <i>Sub for multiple patterns</i> |
|-------|----------------------------------|

---

### Description

Sub for multiple patterns

### Usage

```
mgsub(mydata, pattern, replacement, ...)
```

**Arguments**

|             |                          |
|-------------|--------------------------|
| mydata      | Character                |
| pattern     | Character to search for  |
| replacement | Replacement character    |
| ...         | arguments passed to gsub |

**Examples**

```
mgsub(mydata="##$^&*_+",pattern=c("%","*"),"REPLACE",fixed=TRUE)
```

---

|               |                                                              |
|---------------|--------------------------------------------------------------|
| min_max_index | <i>Indices of the minimum and maximum values in a vector</i> |
|---------------|--------------------------------------------------------------|

---

**Description**

Returns the positions of the minimum and maximum values in a vector. When there are ties all tied positions are returned.

**Usage**

```
min_max_index(vector)
```

**Arguments**

|        |                   |
|--------|-------------------|
| vector | A numeric vector. |
|--------|-------------------|

**Value**

A named list with two elements:

**max\_index** Integer vector of positions where the maximum value occurs.

**min\_index** Integer vector of positions where the minimum value occurs.

**Examples**

```
vector1<-c(1,2,3,4,5,4,3,2,1)
vector2<-c(1,2,3,4,5,5,3,2,1)
vector3<-c(1,2,3,5,5,4,3,2,1)
vector4<-c(1,2,3,4,6,4,3,2,1)
vector5<-c(1,6,3,4,6,4,3,2,1)
vector<-vector1
which(vector==max(vector),arr.ind=TRUE)
which(vector==min(vector),arr.ind=TRUE)
min_max_index(vector1)
min_max_index(vector2)
min_max_index(vector3)
min_max_index(vector4)
min_max_index(vector5)
```

---

|                |                                     |
|----------------|-------------------------------------|
| model_loadings | <i>Pattern and structure matrix</i> |
|----------------|-------------------------------------|

---

### Description

Pattern and structure matrix

### Usage

```
model_loadings(model, cut = NULL, matrix_type = "pattern", sort = TRUE, ...)
```

### Arguments

|             |                                    |
|-------------|------------------------------------|
| model       | psych EFA model                    |
| cut         | cut point for loadings             |
| matrix_type | "pattern" "structure" "all"        |
| sort        | if TRUE it will sort loadings      |
| ...         | arguments passed to psych::fa.sort |

### Note

Check to see if you have multicollinearity values above .8 in the matrix are problematic  
 Structure matrix represents Loadings after rotation  
 Pattern matrix represents Loadings before rotation

### Examples

```
model<-psych::fa(mtcars,nfactors=2,rotate="oblimin",fm="pa",oblique.scores=TRUE)
model_loadings(model=model,cut=NULL,matrix_type="pattern")
model_loadings(model=model,cut=0.4,matrix_type="structure")
model_loadings(model=model,cut=0.4,matrix_type="all",sort=FALSE)
```

---

|                    |                                                              |
|--------------------|--------------------------------------------------------------|
| name_triplet_pairs | <i>Create Pair Labels from Consecutive Triplets of Items</i> |
|--------------------|--------------------------------------------------------------|

---

### Description

Builds pair labels from items grouped in triplets.

In simple terms: items are taken 3 at a time, and for each triplet the function creates the three pair combinations: (1,2), (1,3), and (2,3).

Labels are returned as strings such as "i1i2", "i1i3", "i2i3" (or with your chosen separator/prefix).

**Usage**

```
name_triplet_pairs(n, prefix = "i", sep = "", strict = TRUE)
```

**Arguments**

|        |                                                                                                                                                                             |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| n      | Either: <ul style="list-style-type: none"><li>• A single integer (total number of items, e.g. 15).</li><li>• A vector of item indices (e.g. 4:18).</li></ul>                |
| prefix | Character prefix added before each item index. Default is "i".                                                                                                              |
| sep    | Character separator inserted between the two item labels. Default is "".                                                                                                    |
| strict | Logical. If TRUE (default), stop with an error when the number of items is not a multiple of 3. If FALSE, silently drops leftover items so only complete triplets are used. |

**Details**

If there are  $T$  triplets, output length is  $3T$ , because each triplet contributes exactly 3 pairs.

For one triplet (a, b, c), the generated labels are: ab, ac, bc (with chosen prefix and sep).

**Value**

A character vector of pair labels.

**Examples**

```
# 15 items -> 5 triplets -> 15 pair labels
name_triplet_pairs(15)

# Custom separator
name_triplet_pairs(6, prefix = "i", sep = "_")

# Start from specific indices
name_triplet_pairs(4:9)
# triplets are (4,5,6) and (7,8,9)

# Non-multiple of 3 with strict=FALSE -> trims extras
name_triplet_pairs(10, strict = FALSE)

# Vector input with trimming when needed
name_triplet_pairs(4:18, strict = FALSE)
```

---

|                    |                              |
|--------------------|------------------------------|
| off_diagonal_index | <i>index of off diagonal</i> |
|--------------------|------------------------------|

---

**Description**

index of off diagonal

**Usage**

```
off_diagonal_index(length)
```

**Arguments**

|        |                    |
|--------|--------------------|
| length | length of diagonal |
|--------|--------------------|

**Examples**

```
off_diagonal_index(length=6)
```

---

|                 |                                      |
|-----------------|--------------------------------------|
| outlier_summary | <i>Percent of outliers in vector</i> |
|-----------------|--------------------------------------|

---

**Description**

Percent of outliers in vector

**Usage**

```
outlier_summary(vector)
```

**Arguments**

|        |                |
|--------|----------------|
| vector | numeric vector |
|--------|----------------|

**Details**

returns dataframe

**Examples**

```
vector<-generate_missing(rnorm(1000))
df<-generate_missing(mtcars[,1:2])
outlier_summary(vector)
data.frame(sapply(mtcars,outlier_summary))
```

---

```
output_compare_model_logistic
      Compare logistic regression models models
```

---

**Description**

Compare logistic regression models models

**Usage**

```
output_compare_model_logistic(model1, model2)
```

**Arguments**

|        |                  |
|--------|------------------|
| model1 | object glm model |
| model2 | object glm model |

**Examples**

```
modelcategoricalpredictor<-glm(case~education,data=infert,family=binomial)
modelcontinuouspredictor<-glm(case~age,data=infert,family=binomial)
modeltwopredictors<-glm(case~education*age,data=infert,family=binomial)
modelmultiple<-glm(case~education*age*parity,data=infert,family=binomial)
anova(modelcategoricalpredictor,modelcontinuouspredictor)
output_compare_model_logistic(model1=modelcategoricalpredictor,
                              model2=modeltwopredictors)
output_compare_model_logistic(model1=modelcontinuouspredictor,
                              model2=modeltwopredictors)
output_compare_model_logistic(model1=modelcontinuouspredictor,
                              model2=modelcategoricalpredictor)
```

---

```
output_separator      Output separator
```

---

**Description**

Heading, main output, and instructions for output for the console environment

**Usage**

```
output_separator(
  string,
  output = NULL,
  instruction = NULL,
  length = getOption("width")/2
)
```

**Arguments**

|             |                                                              |
|-------------|--------------------------------------------------------------|
| string      | Title of output                                              |
| output      | object to print                                              |
| instruction | Character provided instructions regarding the output         |
| length      | Numeric Length of separator measured in number of characters |

**Examples**

```
output_separator(string="TEST",output="TEST",instruction="TEST",length=100)
output_separator(string="TEST",instruction="TEST",length=100)
output_separator(string="TEST",output="TEST",length=100)
output_separator(string="TEST")
```

---

|       |                                                             |
|-------|-------------------------------------------------------------|
| padNA | <i>Pad a data frame to a target number of rows with NAs</i> |
|-------|-------------------------------------------------------------|

---

**Description**

Extends a data frame to rowsneeded rows by appending (or prepending) NA-filled rows. Internal helper used by [c\\_bind](#).

**Usage**

```
padNA(df, rowsneeded, first = TRUE)
```

**Arguments**

|            |                                                                                                            |
|------------|------------------------------------------------------------------------------------------------------------|
| df         | A data frame to pad.                                                                                       |
| rowsneeded | Integer target row count. Must be greater than or equal to nrow(df).                                       |
| first      | Logical. When TRUE (default) NA rows are appended at the bottom; when FALSE they are prepended at the top. |

**Value**

A data frame with rowsneeded rows and the same columns as df.

**Author(s)**

Ananda Mahto



## Description

Produces a faceted ggplot2 chart showing the autocorrelation function (ACF), autocovariance function, and partial autocorrelation function (PACF) of a time series side by side. Each facet includes dashed 95% confidence interval lines computed from the distribution of the ACF values, making it easy to identify significant lags and seasonal patterns. Missing values are excluded via `na.action = stats::na.exclude`.

## Usage

```
plot_acf(df, lag.max = length(df), base_size = 10, title = "")
```

## Arguments

|                        |                                                                                                                   |
|------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>df</code>        | A ts object containing the time series to analyse.                                                                |
| <code>lag.max</code>   | Integer specifying the maximum number of lags to compute. Default is <code>length(df)</code> (all possible lags). |
| <code>base_size</code> | Base font size passed to <code>theme_bw()</code> . Default is 10.                                                 |
| <code>title</code>     | Character string used as the plot title. Default is "".                                                           |

## Value

A ggplot object with three free-scale facets:

**Correlation** Autocorrelation function — measures linear dependence between the series and its own lagged values.

**Covariance** Autocovariance function — the un-normalised version of the ACF.

**Partial.Correlation** Partial autocorrelation function — correlation at each lag after removing the effect of shorter lags, useful for identifying AR model order.

Each facet includes dashed horizontal lines for the 95% CI lower bound (blue), mean (black), and upper bound (blue).

## Examples

```
ts_data<-ts(UKDriverDeaths,start=1969,end=1984,frequency=12)
plot_acf(df=ts_data,base_size=20)
```

---

|              |                |
|--------------|----------------|
| plot_boxplot | <i>Boxplot</i> |
|--------------|----------------|

---

**Description**

Boxplot

**Usage**

```
plot_boxplot(df, title = "", base_size = 10)
```

**Arguments**

|           |                                                    |
|-----------|----------------------------------------------------|
| df        | dataframe or vector with continous or ordinal data |
| title     | Plot title                                         |
| base_size | numeric base font size                             |

**Details**

uses ggplot

**Examples**

```
vector<-generate_missing(rnorm(1000))
df<-generate_missing(mtcars[,1:2])
plot_boxplot(df=vector)
plot_boxplot(df=generate_missing(vector))
plot_boxplot(df=df)
```

---

|          |                                                        |
|----------|--------------------------------------------------------|
| plot_cfa | <i>Batch-plot CFA across layouts and display modes</i> |
|----------|--------------------------------------------------------|

---

**Description**

Drop-in replacement for plot\_cfa() — same signature, returns a named list of ggplot objects instead of base-graphics recordings.

**Usage**

```
plot_cfa(model, ...)
```

**Arguments**

|       |                                            |
|-------|--------------------------------------------|
| model | A fitted lavaan object                     |
| ...   | Extra arguments forwarded to plot_cfa_gg() |

Value

Named list of ggplot objects (same keys as the original plot\_cfa)

---

|             |                                      |
|-------------|--------------------------------------|
| plot_cfa_gg | <i>Plot CFA model (semPlot-free)</i> |
|-------------|--------------------------------------|

---

Description

Plot CFA model (semPlot-free)

Usage

```
plot_cfa_gg(  
  model,  
  what = c("std", "est", "eq"),  
  layout = c("tree", "circle", "spring"),  
  label_size = 3.2,  
  edge_label_size = 2.6,  
  color_latent = "#4f8ef7",  
  color_observed = "#e8eaf0",  
  ...  
)
```

Arguments

|                 |                                                                                 |
|-----------------|---------------------------------------------------------------------------------|
| model           | A fitted lavaan object                                                          |
| what            | One of "std" (standardized), "est" (unstandardized), or "eq" (parameter labels) |
| layout          | One of "tree", "circle", or "spring"                                            |
| label_size      | Size of node labels                                                             |
| edge_label_size | Size of path coefficient labels                                                 |
| color_latent    | Fill colour for latent variable nodes                                           |
| color_observed  | Fill colour for observed variable nodes                                         |
| ...             | Ignored (kept for API compatibility with plot_cfa)                              |

Value

A named list of ggplot objects

Examples

```
model='LATENT1=~X1+X2+X3
      LATENT2=~X4+X5+X6'
df<-lavaan::simulateData(model=model,model.type="cfa",
                        return.type="data.frame",sample.nobs=100)

df<-generate_missing(df)
fit<-lavaan::cfa(model,data=df,missing="ML")
plot_cfa_gg(fit,what="std")
plot_cfa_gg(fit,what="std",layout="tree")
plot_cfa_gg(fit,what="std",layout="circle")
plot_cfa_gg(fit,what="std",layout="spring")
```

---

|                |                              |
|----------------|------------------------------|
| plot_confusion | <i>Plot confusion matrix</i> |
|----------------|------------------------------|

---

Description

This function creates a confusion matrix plot with observed and predicted outcomes,including row and column percentages,and various accuracy metrics.

Usage

```
plot_confusion(observed, predicted, base_size = 10, title = "")
```

Arguments

|           |                                                                                                                                 |
|-----------|---------------------------------------------------------------------------------------------------------------------------------|
| observed  | Vector of observed outcomes. This can be numeric or factor values representing the true class labels.                           |
| predicted | Vector of predicted outcomes. This should have the same length as the observed vector and represent the predicted class labels. |
| base_size | Integer value representing the base font size for the plot. Defaults to 10.                                                     |
| title     | String representing the title of the plot. Defaults to an empty string.                                                         |

Details

This function generates a confusion matrix plot using ggplot2. It provides a visual representation of the confusion matrix with observed outcomes on the x-axis and predicted outcomes on the y-axis. The cells of the matrix are filled with the count of observations and annotated with the corresponding values.

The plot also includes various accuracy metrics in the caption,such as: -Overall Accuracy: Proportion of correctly classified observations (diagonal elements). -Off-diagonal Accuracy: Proportion of misclassified observations (off-diagonal elements). -Cohen’s Kappa (Unweighted,Linear,and Squared): Measures the agreement between observed and predicted outcomes.

**Examples**

```
# Example with numeric class labels
plot_confusion(observed=c(1,2,3,1,2,3),predicted=c(1,2,3,1,2,3))

# Example with factor class labels
observed<-c(rep("male",10),rep("female",10),"male","male")
predicted<-c(rep("male",10),rep("female",10),"female","female")
plot_confusion(observed=observed,predicted=predicted)
```

---

|               |                                 |
|---------------|---------------------------------|
| plot_corrplot | <i>Correlation matrix plots</i> |
|---------------|---------------------------------|

---

**Description**

Correlation matrix plots

**Usage**

```
plot_corrplot(mydata, title = "", base_size = 10, fill_limits = c(-1, 0, 1))
```

**Arguments**

|             |                                |
|-------------|--------------------------------|
| mydata      | correlation matrix             |
| title       | plot title                     |
| base_size   | base font size                 |
| fill_limits | lower and upper limit for fill |

**Examples**

```
plot_corrplot(stats::cor(mtcars),title="Correlation")
plot_corrplot(stats::cor(mtcars),base_size=20)
```

---

|                 |                                                    |
|-----------------|----------------------------------------------------|
| plot_crosstable | <i>Bubble plots for pairwise cross-tabulations</i> |
|-----------------|----------------------------------------------------|

---

**Description**

Creates a bubble (point) plot for each pair of categorical variables where point size encodes cell frequency and point colour encodes the levels of the first variable. Variable pairs can be supplied explicitly via combinations, or generated automatically from all unique pairs within factor\_index. A progress bar is displayed during computation.

**Usage**

```
plot_crosstable(
  df,
  factor_index,
  combinations = NULL,
  shape = 16,
  angle = 0,
  base_size = 10,
  title = ""
)
```

**Arguments**

|                           |                                                                                                                                                                                                              |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>df</code>           | A data frame containing the variables to plot.                                                                                                                                                               |
| <code>factor_index</code> | Integer vector of column indices. When <code>combinations</code> is <code>NULL</code> , all unique pair-wise combinations of the selected columns are plotted (self-pairs and duplicate pairs are excluded). |
| <code>combinations</code> | A data frame with two character columns named <code>index1</code> and <code>index2</code> , each row specifying one variable pair to plot. Takes precedence over <code>factor_index</code> .                 |
| <code>shape</code>        | Integer specifying the ggplot2 point shape. Default is 16 (filled circle).                                                                                                                                   |
| <code>angle</code>        | Numeric angle (in degrees) for x-axis tick labels. Default is 0.                                                                                                                                             |
| <code>base_size</code>    | Base font size passed to <code>theme_bw()</code> . Default is 10.                                                                                                                                            |
| <code>title</code>        | Character string used as the plot title. Default is "".                                                                                                                                                      |

**Value**

A named list of ggplot objects, one per variable pair. Each element is named "var1\_var2" and shows a bubble chart with cell frequency as the point size, frequency counts as text labels, and total observations in the caption. Variable pairs with zero total observations are silently dropped.

**Examples**

```
combinations<-data.frame(index1=c("vs","am","gear"),index2=c("cyl","cyl","cyl"))
plot_crosstable(df=mtcars,factor_index=8:9)
plot_crosstable(df=mtcars,combinations=combinations)
```

---

plot\_histogram

*Histograms with density function*


---

**Description**

Histograms with density function

**Usage**

```
plot_histogram(
  df,
  bins = 30,
  title = "",
  base_size = 10,
  xlims = NULL,
  fill = "gray25",
  color = "gray50",
  ylab = "Count"
)
```

**Arguments**

|           |                                                    |
|-----------|----------------------------------------------------|
| df        | dataframe or vector with continous or ordinal data |
| bins      | number of bars to display                          |
| title     | plot title                                         |
| base_size | numeric base font size                             |
| xlims     | x axis limits                                      |
| fill      | color of bar                                       |
| color     | color of bar outline                               |
| ylab      | y label                                            |

**Details**

uses ggplot

**Examples**

```
vector<-generate_missing(rnorm(1000))
df<-generate_missing(mtcars[,1:2])
plot_histogram(df=vector)
plot_histogram(df=df,xlims=c(0,50))
plot_histogram(df=df)
plot_mplot(plotlist=plot_histogram(df=mtcars),cols=4)
```

---

plot\_icc\_thurstonian    *Plot thurstonian icc*

---

**Description**

Plot icc curves for binary thurstonian coded items for a single dimension using the compute\_icc\_thurstonian function

**Usage**

```
plot_icc_thurstonian(mydata, title = "Item Characteristic Curve")
```

**Arguments**

|        |                                                 |
|--------|-------------------------------------------------|
| mydata | dataframe from compute_icc_thurstonian function |
| title  | plot title                                      |

**Examples**

```
gamma<-c(0.556,-1.253,-1.729,0.618,0.937,0.295,-0.672,-1.127,-0.446,0.632,1.147,0.498)
psi<-c(2.172,1.883,2.055,1.869,2.231,2.100,1.762,1.803,1.565,1.892,1.794,1.686)
lambda<-c(1.082,1.082,-1.297,-1.297,0.802,0.802,1.083,1.083)
gamma<-gamma[response_dimension(c(1:12),3,c(1,2))]
```

```
psi<-psi[response_dimension(c(1:12),3,c(1,2))]
```

```
eta<-seq(-6,6,by=1)
```

```
result<-compute_icc_thurstonian(eta=eta,gamma=gamma,lambda=lambda,psi=psi,plot=TRUE)
```

```
plot_icc_thurstonian(result$icc)
```

---

|                  |                                                                            |
|------------------|----------------------------------------------------------------------------|
| plot_interaction | <i>Plot two-way interaction graphs for all IV pair and DV combinations</i> |
|------------------|----------------------------------------------------------------------------|

---

**Description**

For every unique pair of independent variables (IV1 x IV2) and every dependent variable (DV), produces a line-and-point interaction plot with group means on the y-axis. IV1 levels appear on the x-axis (flipped) and IV2 levels are represented by colour and line group. Optional error bars and per-group sample size annotations are included.

When the number of combinations exceeds four times the available CPU cores the plots are produced in parallel via `future.apply`, otherwise sequentially.

**Usage**

```
plot_interaction(
  df,
  dv,
  iv,
  base_size = 20,
  type = "se",
  order_factor = TRUE,
  title = "",
  note = ""
)
```



**Arguments**

|                           |                                                                                                                                                             |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>df</code>           | A data frame containing both the independent and dependent variables.                                                                                       |
| <code>dv</code>           | Integer vector of column indices for the continuous dependent variables.                                                                                    |
| <code>iv</code>           | Integer vector of column indices for the categorical independent variables. Columns are coerced to factors automatically.                                   |
| <code>base_size</code>    | Base font size in pt passed to <code>theme_bw</code> . Default 20.                                                                                          |
| <code>type</code>         | Type of error bar to display. One of "se" (standard error), "ci" (95% confidence interval), "sd" (standard deviation), or "" (no error bars). Default "se". |
| <code>order_factor</code> | Logical. If TRUE factor levels on the x-axis are sorted by the group mean of the DV (descending). Default TRUE.                                             |
| <code>title</code>        | Character. Plot title applied to every panel. Default "".                                                                                                   |
| <code>note</code>         | Character. Caption / footnote appended to every panel. Default "".                                                                                          |

**Value**

A named list with three elements:

- `plot_data` — named list of summary data frames (one per IV1-IV2-DV combination) as returned by `Rmisc::summarySE`.
- `plot_data_df` — single data frame combining all summary data frames row-wise.
- `plots` — named list of ggplot objects (one per combination).

All list elements are named "iv1\_iv2\_dv".

**Examples**

```
# Single DV, two IVs
plot_interaction(df=mtcars, dv=2, iv=8:9, base_size=20, type="se")

# Multiple DVs, two IVs
plot_interaction(df=mtcars, dv=2:3, iv=8:9, base_size=20, type="se")
plot_interaction(df=mtcars, dv=2:3, iv=8:9, base_size=20, type="ci")
plot_interaction(df=mtcars, dv=2:3, iv=9:10, base_size=20, type="sd")

# No error bars, unordered factor axis
plot_interaction(df=mtcars, dv=2, iv=9:10, base_size=20, type="", order_factor=FALSE)
```

---

`plot_irt_onefactor`      *Return data for irt plots*

---

**Description**

Return data for irt plots

Usage

```
plot_irt_onefactor(model, theta = seq(-6, 6, 0.1), title = "", base_size = 10)
```

Arguments

|           |             |
|-----------|-------------|
| model     | object mirt |
| theta     | theta       |
| title     | plot title  |
| base_size | base size   |

Examples

```
cormatrix<-psych::sim.rasch(nvar=5,n=50000,low=-4,high=4,d=NULL,a=1,mu=0,sd=1)$items
model<-mirt::mirt(cormatrix,1,empiricalhist=TRUE,calcNull=TRUE)
plot_irt_onefactor(model=model,base_size=10,title="Normal Test")
cormatrix<-psych::sim.rasch(nvar=5,n=50000,low=-6,high=-4,d=NULL,a=1,mu=0,sd=1)$items
model<-mirt::mirt(cormatrix,1,empiricalhist=TRUE,calcNull=TRUE)
plot_irt_onefactor(model=model,base_size=10,title="Easy Items")
cormatrix<-psych::sim.rasch(nvar=5,n=50000,low=4,high=6,d=NULL,a=1,mu=0,sd=1)$items
model<-mirt::mirt(cormatrix,1,empiricalhist=TRUE,calcNull=TRUE)
plot_irt_onefactor(model=model,base_size=10,title="Difficult Items")
cormatrix<-psych::sim.rasch(nvar=5,n=50000,low=-4,high=-4,d=NULL,a=0.01,mu=0,sd=1)$items
model<-mirt::mirt(cormatrix,1,empiricalhist=TRUE,calcNull=TRUE)
plot_irt_onefactor(model=model,base_size=10,title="Low Discrimination")
cormatrix<-psych::sim.poly(nvar=5,n=50000,low=-4,high=4,a=1,c=0,z=1,d=NULL,
                           mu=0,sd=1,cat=5,mod="logistic",theta=NULL)$items
model<-mirt::mirt(cormatrix,1,itemtype="graded")
plot_irt_onefactor(model=model,base_size=10,title="graded response")
```

---

|               |                      |
|---------------|----------------------|
| plot_loadings | <i>Plot loadings</i> |
|---------------|----------------------|

---

Description

Plot loadings

Usage

```
plot_loadings(
  model,
  matrix_type = NULL,
  title = "",
  base_size = 10,
  color = c("#5E912C", "white", "#5F2C91"),
  sort = TRUE
)
```

**Arguments**

|             |                             |
|-------------|-----------------------------|
| model       | psych EFA model             |
| matrix_type | "pattern" "structure"       |
| title       | plot title                  |
| base_size   | base font size              |
| color       | color ranges for heatmap    |
| sort        | TRUE or FALSE sort loadings |

**Examples**

```

model<-psych::fa(mtcars,nfactors=2,rotate="oblimin",fm="pa",oblique.scores=TRUE)
plot_loadings(model=model,matrix_type="structure")
plot_loadings(model=model,matrix_type="pattern")
cm<-matrix(c(1,.8,.8,.1,.1,.1,
             .8,1,.8,.1,.1,.1,
             .8,.8,1,.1,.1,.1,
             .1,.1,.1,1,.8,.8,
             .1,.1,.1,.8,1,.8,
             .1,.1,.1,.8,.8,1),
           ncol=6,nrow=6)

df1<-generate_correlation_matrix(cm,nrows=10000)
model1<-psych::fa(df1,nfactors=2,rotate="oblimin",fm="pa",oblique.scores=TRUE)
plot_loadings(model=model1,matrix_type="pattern",base_size=30)
cm<-matrix(c(1,.1,.1,.1,.1,.1,
             .1,1,.1,.1,.1,.1,
             .1,.1,1,.1,.1,.1,
             .1,.1,.1,1,.8,.8,
             .1,.1,.1,.8,1,.8,
             .1,.1,.1,.8,.8,1),
           ncol=6,nrow=6)

df1<-generate_correlation_matrix(cm,nrows=10000)
model2<-psych::fa(df1,nfactors=2,rotate="oblimin",fm="pa",oblique.scores=TRUE)
plot_loadings(model=model2,matrix_type="pattern",base_size=30)
cm<-matrix(c(1,.01,.01,.01,.01,.01,
             .01,1,.01,.01,.01,.01,
             .01,.01,1,.01,.01,.01,
             .01,.01,.01,1,.01,.01,
             .01,.01,.01,.01,1,.01,
             .01,.01,.01,.01,.01,1),
           ncol=6,nrow=6)

df1<-generate_correlation_matrix(cm,nrows=10000)
model3<-psych::fa(df1,nfactors=2,rotate="oblimin",fm="pa",oblique.scores=TRUE)
plot_loadings(model=model3,matrix_type="pattern",base_size=10)

```

**Description**

Logistic model plot

**Usage**

```
plot_logistic_model(df, outcome = "outcome", title = "", base_size = 10)
```

**Arguments**

|           |                                                             |
|-----------|-------------------------------------------------------------|
| df        | dataframe with predictor and outcome outcome should be last |
| outcome   | name of outcome variable                                    |
| title     | Character plot title                                        |
| base_size | base font size                                              |

**Examples**

```
df<-data.frame(outcome=c(rep(1,10),rep(0,10)),
               pd1=c(rep(1,11),rep(0,9)),
               pd2=c(rep(1,9),rep(0,11)),
               pc1=c(rnorm(10,mean=5),rnorm(10,mean=10)),
               pc2=c(rnorm(10,mean=5),rnorm(10,mean=20)))
plot_logistic_model(df=df,base_size=15)
```

---

|             |                                                        |
|-------------|--------------------------------------------------------|
| plot_mosaic | <i>Mosaic plots for pairwise categorical variables</i> |
|-------------|--------------------------------------------------------|

---

**Description**

Creates a mosaic plot for every ordered pair of categorical variables within factor\_index. In each plot, bar widths represent the marginal proportion of the first variable and bar heights represent the conditional proportion of the second variable given the first, making it straightforward to assess both marginal distributions and conditional relationships simultaneously. Rows with missing values are excluded pair-wise. A progress bar is displayed during computation.

**Usage**

```
plot_mosaic(df, factor_index, base_size = 10, title = "")
```

**Arguments**

|              |                                                                                                                            |
|--------------|----------------------------------------------------------------------------------------------------------------------------|
| df           | A data frame containing the variables to plot.                                                                             |
| factor_index | Integer vector of column indices identifying the categorical variables. All ordered pairs of distinct columns are plotted. |
| base_size    | Base font size passed to theme_bw(). Default is 10.                                                                        |
| title        | Character string prepended to each plot title. Default is "".                                                              |

**Value**

A named list of ggplot objects, one per ordered variable pair. Each element is named "var1 var2" and shows a mosaic chart with bar widths proportional to the marginal distribution of var1, bar heights proportional to the conditional distribution of var2 given var1, and total complete-case observations in the caption. Variables with fewer than two observed levels are handled gracefully by adding a placeholder level.

**Examples**

```
plot_mosaic(df=mtcars, factor_index=8:9)
plot_mosaic(df=mtcars, factor_index=9:10)
```

---

|           |                                                  |
|-----------|--------------------------------------------------|
| plot_mtmm | <i>Multitrait-multimethod (MTMM) matrix plot</i> |
|-----------|--------------------------------------------------|

---

**Description**

Visualises a Campbell-Fiske multitrait-multimethod matrix as a faceted tile plot. For each trait-method combination the function computes a scale score (row mean of items), calculates Cronbach's alpha (shown on the diagonal), and correlates all scale scores across traits and methods. Each cell is colour-coded by its MTMM classification, making it easy to evaluate convergent and discriminant validity at a glance.

**Usage**

```
plot_mtmm(df, key, method, subject, title = "")
```

**Arguments**

|         |                                                                                                                                                           |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| df      | A data frame in long format where each row corresponds to one subject under one method. Item response columns must be present for all traits and methods. |
| key     | A named list mapping trait names to character vectors of item column names, e.g. <code>list(t1 = c("x1", "x2", "x3"), t2 = c("x4", "x5", "x6"))</code> .  |
| method  | Character string giving the name of the column that identifies the measurement method for each row.                                                       |
| subject | Character string giving the name of the column that identifies the subject (used to align scores across methods).                                         |
| title   | Character string appended to the plot title. Default is "".                                                                                               |

**Value**

A ggplot object showing a faceted tile plot where rows and columns correspond to traits, facets correspond to method pairs, cell values are correlations (or alpha on the diagonal), and fill colour encodes one of four MTMM relationship types:

**monotrait-monomethod (reliability)** Same trait, same method — Cronbach's alpha.

**monotrait-heteromethod (validity)** Same trait, different methods — convergent validity.

**heterotrait-monomethod** Different traits, same method — discriminant validity within method.

**heterotrait-heteromethod** Different traits, different methods — discriminant validity across methods.

### Examples

```
population_model<- 't1=~x1+.9*x2+.9*x3
                    t2=~x4+.9*x5+.9*x6
                    t3=~x7+.9*x8+.9*x9'
model_data<-lavaan::simulateData(population_model,sample.nobs=1000)
model_data<-model_data[sample(1:1000,1000,TRUE),]
model_data<-rbind(model_data,model_data,model_data)
model_data$method<-c(rep("m1",1000),rep("m2",1000),rep("m3",1000))
model_data$id<-rep(1:1000,3)
key<-list(t1=paste0("x",1:3),t2=paste0("x",4:6),t3=paste0("x",7:9))
plot_mtm(df=model_data,key=key,method="method",subject="id")
```

---

plot\_multiplot

*Multiple ggplot plots in one graph*

---

### Description

Multiple ggplot plots in one graph

### Usage

```
plot_multiplot(..., plotlist = NULL, cols = 2, layout = NULL)
```

### Arguments

|          |                                                               |
|----------|---------------------------------------------------------------|
| ...      | plot objects                                                  |
| plotlist | a list of plots                                               |
| cols     | number of columns in layout                                   |
| layout   | a matrix specifying the layout. If present, 'cols' is ignored |

### Examples

```
p1<-ggplot(ChickWeight,aes(x=Time,y=weight,colour=Diet,group=Chick))+
  geom_line()+
  ggtitle("Growth curve for individual chicks")+
  theme_bw()
p2<-ggplot(ChickWeight,aes(x=Time,y=weight,colour=Diet))+
  geom_point(alpha=.3)+
  geom_smooth(alpha=.2,size=1,method="loess",formula="y~x")+
  ggtitle("Fitted growth curve per diet")+
  theme_bw()
p3<-ggplot(subset(ChickWeight,Time==21),aes(x=weight,colour=Diet))+
```

```

      geom_density()+
      ggtitle("Final weight, by diet")+theme_bw()
p4<-ggplot(subset(ChickWeight,Time==21),aes(x=weight,fill=Diet))+
      geom_histogram(colour="black",binwidth=50)+facet_grid(Diet~.)+
      ggtitle("Final weight, by diet")+theme_bw()
cars_plot<-plot_histogram(mtcars)
plot_multiplot(p1,p2,p3,p4,cols=2)
plot_multiplot(plotlist=plot_histogram(mtcars[,1:4]),cols=2)
plot_multiplot(plotlist=plot_histogram(mtcars),layout=matrix(1:4,ncol=2,byrow=TRUE))
plot_multiplot(plotlist=plot_scatterplot(mtcars[,1:4]),cols=2)
plot_multiplot(plotlist=cars_plot,layout=matrix(1:4,ncol=2,byrow=TRUE))
plot_multiplot(plotlist=cars_plot,cols=3)

```

---

plot\_normality\_diagnostics

*Normality plots*


---

## Description

plot histogram density boxplot qq plot

## Usage

```

plot_normality_diagnostics(
  df,
  breaks = NULL,
  title = "",
  file = NULL,
  w = 10,
  h = 10
)

```

## Arguments

|        |                                                    |
|--------|----------------------------------------------------|
| df     | dataframe or vector with continous or ordinal data |
| breaks | number of bars to display                          |
| title  | plot title                                         |
| file   | output filename                                    |
| w      | width of pdf file                                  |
| h      | height of pdf file                                 |

## Details

uses plot base

## Examples

```
vector<-generate_missing(rnorm(1000))
df<-generate_missing(mtcars[,1:2])
plot_normality_diagnostics(df=vector,title="",file="rnorm",breaks=30)
plot_normality_diagnostics(df=vector,title="")
plot_normality_diagnostics(df=df,title="mtcars")
plot_normality_diagnostics(df=df,title="mtcars",file="rnorm")
```

---

plot\_oneway

---

*Plot group means with error bars for all IV-DV combinations*


---

## Description

For every combination of independent variable (IV) and dependent variable (DV) supplied, produces a horizontal dot plot of group means with optional error bars (standard error, confidence interval, or standard deviation). Sample size per group is annotated on each panel.

When the number of IV-DV combinations exceeds four times the available CPU cores the plots are produced in parallel via `future.apply`, otherwise sequentially.

## Usage

```
plot_oneway(
  df,
  dv,
  iv,
  base_size = 20,
  type = "se",
  order_factor = TRUE,
  title = "",
  note = "",
  width = 60
)
```

## Arguments

|              |                                                                                                                                                             |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| df           | A data frame containing both the independent and dependent variables.                                                                                       |
| dv           | Integer vector of column indices for the continuous dependent variables.                                                                                    |
| iv           | Integer vector of column indices for the categorical independent variables. Columns are coerced to factors automatically.                                   |
| base_size    | Base font size in pt passed to <code>theme_bw</code> . Default 20.                                                                                          |
| type         | Type of error bar to display. One of "se" (standard error), "ci" (95% confidence interval), "sd" (standard deviation), or "" (no error bars). Default "se". |
| order_factor | Logical. If TRUE factor levels on the x-axis are sorted by the group mean of the DV (descending). Default TRUE.                                             |
| title        | Character. Plot title applied to every panel. Default "".                                                                                                   |
| note         | Character. Caption / footnote appended to every panel. Default "".                                                                                          |
| width        | Integer. Character width at which long axis labels are wrapped. Default 60.                                                                                 |



**Value**

A named list with three elements:

- `plot_data` — named list of summary data frames (one per IV-DV pair) as returned by `Rmisc::summarySE`.
- `plot_data_df` — single data frame combining all summary data frames row-wise.
- `plots` — named list of ggplot objects (one per IV-DV pair).

All list elements are named "iv\_dv".

**Examples**

```
nrows <- 1000
df <- data.frame(generate_factor(vector=LETTERS[1:5], nrows=nrows, ncols=10, type="random"),
                 generate_data(nrows=nrows, ncols=5, type="normal"))
result <- plot_oneway(df=df, dv=11:15, iv=1:10)

# Single IV, single DV
plot_oneway(df=mtcars, dv=2, iv=9)

# Multiple IVs and DVs
plot_oneway(df=mtcars, dv=2:3, iv=9:10)

# Error bar types
plot_oneway(df=mtcars, dv=2:3, iv=9:10, type="se")
plot_oneway(df=mtcars, dv=2:3, iv=9:10, type="ci")
plot_oneway(df=mtcars, dv=2:3, iv=9:10, type="sd")
plot_oneway(df=mtcars, dv=2:3, iv=9:10, type="")

# Factor ordering
plot_oneway(df=mtcars, dv=2:3, iv=9:10, type="", order_factor=FALSE)
plot_oneway(df=mtcars, dv=2:3, iv=9:10, type="", order_factor=TRUE)
```

---

plot\_oneway\_diagnostics

*Diagnostic plots for one-way ANOVA models*

---

**Description**

For every combination of independent variable (IV) and dependent variable (DV), fits a linear model and produces a 6-panel diagnostic plot via `ggfortify::autoplot`: Residuals vs Fitted, Normal Q-Q, Scale-Location, Cook's Distance, Residuals vs Leverage, and Cook's Distance vs Leverage.

**Interpretation:**

- *Residuals vs Fitted* — points should be randomly scattered with no pattern; a funnel shape indicates heteroscedasticity.
- *Normal Q-Q* — points should follow the diagonal; large deviations indicate non-normality.

When the number of IV-DV combinations exceeds four times the available CPU cores the plots are produced in parallel via `future.apply`, otherwise sequentially.

**Usage**

```
plot_oneway_diagnostics(df, dv, iv, base_size = 10)
```

**Arguments**

- df                    A data frame containing both the independent and dependent variables.
- dv                    Integer vector of column indices for the continuous dependent variables.
- iv                    Integer vector of column indices for the categorical independent variables.
- base\_size            Base font size in pt passed to theme\_bw. Default 10.

**Value**

A named list of ggplot objects (one 6-panel plot per IV-DV pair), named "iv\_dv".

**Examples**

```
nrows <- 1000
df <- data.frame(generate_factor(vector=LETTERS[1:5], nrows=nrows, ncols=10, type="random"),
                 generate_data(nrows=nrows, ncols=5, type="normal"))
result <- plot_oneway_diagnostics(df=df, dv=11:15, iv=1:10)

# Single DV, multiple IVs
plot_oneway_diagnostics(df=mtcars, dv=1, iv=9:10)

# Multiple DVs and IVs
plot_oneway_diagnostics(df=mtcars, dv=1:2, iv=9:10)
```

---

|              |                                                               |
|--------------|---------------------------------------------------------------|
| plot_outlier | <i>Outlier graph using mean median and boxplot algorithms</i> |
|--------------|---------------------------------------------------------------|

---

**Description**

Outlier graph using mean median and boxplot algorithms

**Usage**

```
plot_outlier(df, method = "mean", title = "", base_size = 10)
```

**Arguments**

- df                    dataframe or vector with continous or ordinal data
- method                "mean" "median" "boxplot"
- title                 plot title
- base\_size            base font size

**Author(s)**

unknown

**Examples**

```
vector<-generate_missing(rnorm(1000))
df<-generate_missing(mtcars[,1:2])
plot_outlier(df=vector,method="mean",title="random vector")
plot_outlier(df=vector,method="median")
plot_outlier(df=vector,method="boxplot")
plot_outlier(df=df,method="mean",title="random vector")
plot_outlier(df=df,method="median")
plot_outlier(df=df,method="boxplot")
plot_mplot(plotlist=plot_outlier(df=mtcars[,2:5],method="mean"),cols=2)
```

---

plot\_qq*qq plots*

---

**Description**

qq plots

**Usage**

```
plot_qq(df, title = "", base_size = 10)
```

**Arguments**

|           |                                                    |
|-----------|----------------------------------------------------|
| df        | dataframe or vector with continous or ordinal data |
| title     | plot title                                         |
| base_size | numeric base font size                             |

**Details**

uses ggplot

**Examples**

```
vector<-generate_missing(rnorm(1000))
df<-generate_missing(mtcars[,1:2])
plot_qq(df=vector)
plot_qq(df=df)
plot_mplot(plotlist=plot_qq(df=mtcars),cols=4)
```

---

plot\_response\_frequencies

*Horizontal bar charts of response frequencies*


---

### Description

Creates one horizontal bar chart per variable showing the frequency count of each observed level. Missing values are excluded before tabulation. Variables with no valid observations are silently dropped from the output.

### Usage

```
plot_response_frequencies(
  df,
  factor_index,
  base_size = 10,
  title = "",
  width = 100,
  reorder = FALSE
)
```

### Arguments

|              |                                                                                                                                                           |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| df           | A data frame containing the variables to plot.                                                                                                            |
| factor_index | Integer vector of column indices identifying the variables to plot.                                                                                       |
| base_size    | Base font size passed to theme_bw(). Default is 10.                                                                                                       |
| title        | Character string prepended to each plot title. Default is "".                                                                                             |
| width        | Integer controlling the character wrap width applied to the variable name in the plot title. Default is 100.                                              |
| reorder      | Logical. When TRUE bars are ordered by frequency in ascending order (longest bar at the top). When FALSE (default) the original level order is preserved. |

### Value

A named list of ggplot objects, one per variable, named by the column name. Each plot is a horizontal bar chart with counts on the x-axis and total observations shown in the caption.

### Examples

```
plot_response_frequencies(df=mtcars, factor_index=1:10)
```

plot\_roc

*Plot Receiver Operating Characteristic (ROC) curve***Description**

Generates a ROC curve from observed outcomes and predicted probabilities.

**Usage**

```
plot_roc(observed, predicted, base_size = 10, title = "")
```

**Arguments**

|           |                                                                                                          |
|-----------|----------------------------------------------------------------------------------------------------------|
| observed  | Vector of observed outcomes. These are the true class labels.                                            |
| predicted | Vector of predicted outcome probabilities. These are the predicted probabilities for the positive class. |
| base_size | Integer value representing the base font size for the plot. Defaults to 10.                              |
| title     | String representing the title of the plot. Defaults to an empty string.                                  |

**Details**

This function generates a ROC curve to evaluate the performance of a binary classification model. The ROC curve is a plot of the true positive rate (TPR) against the false positive rate (FPR) at various threshold settings.

The function performs the following steps: 1. Computes the ROC curve and its confidence interval using 'pROC::roc'. 2. Generates ROC plots for both reversed and non-reversed order of class levels. 3. Creates a list of ROC plots, each with an AUC value, control level, and direction.

The output is a list of ggplot objects representing the ROC curves for different class level orders.

**Examples**

```
# Example with random observed and predicted values
observed<-round(abs(rnorm(100,m=0,sd=0.5)))
predicted<-abs(rnorm(100,m=0,sd=0.5))
plot_roc(observed=observed,predicted=predicted)

# Example with generated correlation matrix
df1<-data.frame(matrix(0.999,ncol=2,nrow=2))
correlation_matrix<-as.matrix(df1)
diag(correlation_matrix)<-1
df1<-generate_correlation_matrix(correlation_matrix,nrows=1000)
df1$X1<-ifelse(abs(df1$X1) < 1,0,1)
df1$X2<-abs(df1$X2)
df1$X2<-(df1$X2-min(df1$X2))/(max(df1$X2)-min(df1$X2))
plot_roc(observed=round(abs(df1$X1),0),predicted=abs(df1$X2))
```

---

plot\_scatterplot

*Scatter plots for all variable pairs in a data frame*


---

### Description

Generates a scatter plot with a smoothing line and marginal histograms for every pair of numeric columns in `df`. Each plot includes Pearson  $r$ , explained variance, the regression equation, and the regression angle in its caption (when the default  $y \sim x$  formula is used).

When the number of pairs exceeds four times the available CPU cores the plots are produced in parallel via `future.apply`, otherwise sequentially.

### Usage

```
plot_scatterplot(
  df,
  method = lm,
  formula = y ~ x,
  base_size = 10,
  coord_equal = FALSE,
  all_orders = FALSE,
  title = "",
  combinations = NULL,
  string_aes = TRUE
)
```

### Arguments

|                           |                                                                                                                                                                                                          |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>df</code>           | A data frame of numeric variables. When <code>df</code> has exactly two columns the first is treated as the predictor and the second as the outcome.                                                     |
| <code>method</code>       | Smoothing method passed to <code>geom_smooth</code> . Accepts "lm", "glm", "gam", "loess", or a function such as <code>MASS::rlm</code> . Default <code>lm</code> .                                      |
| <code>formula</code>      | Formula passed to <code>geom_smooth</code> . Default $y \sim x$ . When a non-default formula is supplied the caption shows only pairwise $n$ .                                                           |
| <code>base_size</code>    | Base font size in pt passed to <code>theme_bw</code> . Default 10.                                                                                                                                       |
| <code>coord_equal</code>  | Logical. If TRUE both axes share the same scale and limits. Default FALSE.                                                                                                                               |
| <code>all_orders</code>   | Logical. If TRUE both (X, Y) and (Y, X) orderings are plotted for each pair. Default FALSE.                                                                                                              |
| <code>title</code>        | Character. Plot title applied to every panel. Default "".                                                                                                                                                |
| <code>combinations</code> | A two-column data frame specifying which pairs to plot. Column 1 is the x-variable name, column 2 is the y-variable name. When NULL (default) all pairs are derived automatically from <code>df</code> . |
| <code>string_aes</code>   | Logical. If TRUE variable names are passed through <code>string_aes()</code> to clean axis labels. Default TRUE.                                                                                         |

**Value**

A named list of ggplot objects, one per variable pair. Names follow the pattern "x\_y".

**Examples**

```
result <- plot_scatterplot(df=mtcars, title="", coord_equal=TRUE, base_size=10)
plot_multiplot(plotlist=result[1:12], cols=4)

# Two-column data frame: first column = predictor, second = outcome
plot_scatterplot(df=mtcars[,1:2], base_size=10, coord_equal=TRUE, all_orders=FALSE)

# Custom variable pairs
plot_scatterplot(df=mtcars, base_size=10, coord_equal=TRUE,
  combinations=data.frame(x=c("mpg", "mpg", "mpg"),
    y=c("cyl", "hp", "disp")))

# Simulated near-perfect correlation
x <- rnorm(1000)
y <- x + rnorm(x, sd=.1)
plot_scatterplot(df=data.frame(x,y), title="Random Simulation", coord_equal=TRUE)
```

---

|            |                                                                                   |
|------------|-----------------------------------------------------------------------------------|
| plot_scree | <i>Scree plot displaying the Kaiser and Jolife criteria for factor extraction</i> |
|------------|-----------------------------------------------------------------------------------|

---

**Description**

Scree plot displaying the Kaiser and Jolife criteria for factor extraction

**Usage**

```
plot_scree(df, base_size = 15, title = "", color = c("#5F2C91", "#5E912C"))
```

**Arguments**

|           |                                 |
|-----------|---------------------------------|
| df        | dataframe                       |
| base_size | base font size                  |
| title     | plot title                      |
| color     | color of line and point outline |

**Examples**

```
plot_scree(df=mtcars, title="", base_size=15)
```

---

|                   |                          |
|-------------------|--------------------------|
| plot_separability | <i>Plot separability</i> |
|-------------------|--------------------------|

---

## Description

This function creates a separability plot showing the density distribution of predicted probabilities for different observed categories.

## Usage

```
plot_separability(observed, predicted, base_size = 10, title = "")
```

## Arguments

|           |                                                                                                                                               |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| observed  | Vector of observed outcomes. This can be numeric or factor values representing the true class labels.                                         |
| predicted | Vector of predicted outcome probabilities. This should have the same length as the observed vector and represent the predicted probabilities. |
| base_size | Integer value representing the base font size for the plot. Defaults to 10.                                                                   |
| title     | String representing the title of the plot. Defaults to an empty string.                                                                       |

## Details

This function generates a separability plot using ggplot2. It shows the density distribution of predicted probabilities for different observed categories. The plot helps to visualize how well the predicted probabilities separate the different observed categories.

The plot includes the following components: -Density curves for each observed category, representing the distribution of predicted probabilities. -A legend indicating the observed categories. -The total number of observations is included in the plot caption.

## Examples

```
# Example with numeric class labels
df1<-data.frame(matrix(.999,ncol=2,nrow=2))
correlation_matrix<-as.matrix(df1)
diag(correlation_matrix)<-1
df1<-generate_correlation_matrix(correlation_matrix,nrows=1000)
df1$X1<-ifelse(abs(df1$X1) < 1,0,1)
df1$X2<-abs(df1$X2)
df1$X2<-(df1$X2-min(df1$X2))/(max(df1$X2)-min(df1$X2))
plot_separability(observed=round(abs(df1$X1),0),predicted=abs(df1$X2))
```



---

|                    |                                          |
|--------------------|------------------------------------------|
| plot_trees_xgboost | <i>Plot trees for xgboost::xgb.train</i> |
|--------------------|------------------------------------------|

---

## Description

Plot trees for xgboost::xgb.train

## Usage

```
plot_trees_xgboost(model, train, file = "xgboost")
```

## Arguments

|       |                                |
|-------|--------------------------------|
| model | object from xgboost::xgb.train |
| train | Train dataset                  |
| file  | output filename                |

## Examples

```
infern_formula<-formula(case~education+spontaneous+induced)
boston_formula<-formula(medv~crim+zn+indus+chas+nox+rm+age+dis+rad+tax+ptratio+black+lstat)
train_test_classification<-k_fold(df=infern,model_formula=infern_formula)
train_test_regression<-k_fold(df=MASS::Boston,model_formula=boston_formula)
xgb_classification<-xgboost::xgb.train(
  data=train_test_classification$xgb$f1$train,
  watchlist=train_test_classification$xgb$f1$watchlist,
  eta=.1,
  nthread=8,
  nround=20,
  objective="binary:logistic")
xgb_regression<-xgboost::xgb.train(
  data=train_test_regression$xgb$f1$train,
  watchlist=train_test_regression$xgb$f1$watchlist,
  eta=.3,
  nthread=8,
  nround=20)
# xgboost::xgb.plot.multi.trees(model=xgb_classification,features_keep=2)
# plot_trees_xgboost(model=xgb_classification,
#   train=train_test_classification$xgb$f1,
#   file="Classification")
# plot_trees_xgboost(model=xgb_regression,
#   train=train_test_regression$xgb$f1,
#   file="Regression")
```

plot\_ts

*Line plot for a time series***Description**

Converts a ts object into a ggplot2 line chart with semi-transparent points and an overlaid linear trend line. The returned ggplot object can be extended with additional layers (e.g. geom\_vline() to mark events).

**Usage**

```
plot_ts(df, base_size = 10, ylab = "Count", title = "")
```

**Arguments**

|           |                                                            |
|-----------|------------------------------------------------------------|
| df        | A ts object containing the time series to plot.            |
| base_size | Base font size passed to theme_bw(). Default is 10.        |
| ylab      | Character string for the y-axis label. Default is "Count". |
| title     | Character string used as the plot title. Default is "".    |

**Value**

A ggplot object showing the time series as a line with points, a linear trend line fitted via lm, and a caption reporting the total number of observations.

**Examples**

```
ts_data<-ts(UKDriverDeaths,start=1969,end=1984,frequency=12)
result<-plot_ts(ts_data,title="UK driver deaths")
for(i in 1969:1984)
  result<-result+geom_vline(xintercept=i,color="blue",size=1,alpha=.5)
result
autoplot(stl(ts_data,s.window='periodic'))+
  theme_bw(base_size=10)+
  labs(title="UK driver deaths")
forecast::gglagplot(data.frame(ts_data),do.lines=FALSE,lags=100)+
  theme_bw(base_size=10)+labs(title="UK driver deaths",y="count")
```

---

|        |                                                          |
|--------|----------------------------------------------------------|
| proper | <i>Capitalize first character and lowercase the rest</i> |
|--------|----------------------------------------------------------|

---

**Description**

Capitalize first character and lowercase the rest

**Usage**

```
proper(x)
```

**Arguments**

|   |           |
|---|-----------|
| x | Character |
|---|-----------|

**Examples**

```
x<-generate_string(nchar=10,vector=LETTERS,vector_length=10)
proper(x)
```

---

|                     |                                                          |
|---------------------|----------------------------------------------------------|
| proportion_accurate | <i>Proportion overall accuracy of a confusion matrix</i> |
|---------------------|----------------------------------------------------------|

---

**Description**

Calculates the overall accuracy and Cohen's kappa statistics of a confusion matrix.

**Usage**

```
proportion_accurate(observed, predicted)
```

**Arguments**

|           |                                                                      |
|-----------|----------------------------------------------------------------------|
| observed  | Vector of observed variables. These are the true class labels.       |
| predicted | Vector of predicted variables. These are the predicted class labels. |

**Details**

This function evaluates the performance of a confusion matrix by calculating the overall accuracy and Cohen's kappa statistics.

The function performs the following steps: 1. Computes the confusion matrix from the observed and predicted values. 2. Calculates the diagonal proportion (overall accuracy) and the off-diagonal proportion. 3. Computes Cohen's kappa statistics (unweighted, linear, and squared weights).

The output is a data.frame containing the following metrics: -'cm\_diagonal': Proportion of correct classifications (diagonal elements). -'cm\_off\_diagonal': Proportion of misclassified observations (off-diagonal elements). -'kappa\_unweighted': Cohen's kappa statistic with no weights. -'kappa\_linear': Cohen's kappa statistic with linear weights. -'kappa\_squared': Cohen's kappa statistic with squared weights.

Examples

```
# Example with numeric observed and predicted values
proportion_accurate(observed=c(1,2,3,4,5,10),predicted=c(1,2,3,4,5,11))
```

---

|                   |                                  |
|-------------------|----------------------------------|
| questions_by_keys | <i>Convert key to index list</i> |
|-------------------|----------------------------------|

---

Description

Convert key to index list

Usage

```
questions_by_keys(key)
```

Arguments

key                      a vector indicating the dimension of each question. The order of the elements in the key represents the order of the questions, the numeric values represent the dimension the question belongs to

Examples

```
key<-c(1,2,3,4,5,1,2,3,4,5)
questions_by_keys(key)
```

---

|                                |                                 |
|--------------------------------|---------------------------------|
| questions_dimensions_dataframe | <i>Question dimension table</i> |
|--------------------------------|---------------------------------|

---

Description

Return a dataframe with the order of the questions, their respective dimensions, and the description of the dimensions

Usage

```
questions_dimensions_dataframe(
  key,
  dimensions,
  elaborate_dimensions,
  questions
)
```

Arguments

|                      |                                                                                                                                                                                                    |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| key                  | a vector indicating the dimension of each question. The order of the elements in the key represents the order of the questions, the numeric values represent the dimension the question belongs to |
| dimensions           | dimension names                                                                                                                                                                                    |
| elaborate_dimensions | full dimension names                                                                                                                                                                               |
| questions            | question names                                                                                                                                                                                     |

Examples

```
key<-c(1,2,3,4,5,1,2,3,4,5)
dimensions<-paste0("Dimension",1:10)
elaborate_dimensions<-paste0("Elaborated_Dimension",1:10)
questions<-paste0("Question",1:65)
questions_dimensions_dataframe(key,dimensions,elaborate_dimensions,questions)
```

---

|         |                                   |
|---------|-----------------------------------|
| rad2deg | <i>Convert radians to degrees</i> |
|---------|-----------------------------------|

---

Description

Convert radians to degrees

Usage

```
rad2deg(radians)
```

Arguments

|         |         |
|---------|---------|
| radians | radians |
|---------|---------|

Examples

```
rad2deg(pi)
```

---

|                   |                                                     |
|-------------------|-----------------------------------------------------|
| rank3_to_triplets | <i>Convert thurstonian binary triplets to scale</i> |
|-------------------|-----------------------------------------------------|

---

**Description**

Convert thurstonian binary triplets to scale

**Usage**

```
rank3_to_triplets(mydata)
```

**Arguments**

mydata                  dataframe

**Examples**

```
set.seed(12345)
mydata<-data.frame(i1=rnorm(10,mean=2,sd=.5),
                   i2=rnorm(10,mean=2,sd=.5),
                   i3=rnorm(10,mean=2,sd=.5),
                   i4=rnorm(10,mean=2,sd=.5),
                   i5=rnorm(10,mean=2,sd=.5),
                   i6=rnorm(10,mean=2,sd=.5))
result<-rank_to_binary(mydata[,1:3])
rank3_to_triplets(result)
```

---

|                   |                                                                                |
|-------------------|--------------------------------------------------------------------------------|
| rank_df_to_binary | <i>Convert scale to thurstonian binary with n items per block and n blocks</i> |
|-------------------|--------------------------------------------------------------------------------|

---

**Description**

Convert scale to thurstonian binary with n items per block and n blocks

**Usage**

```
rank_df_to_binary(mydata, items, reverse = TRUE)
```

**Arguments**

mydata                  dataframe  
items                    number of items in block  
reverse                  if TRUE assumes that the highest value is first item in rank if FALSE the lowest value is the first item in rank

**Examples**

```
set.seed(12345)
mydata<-data.frame(i1=rnorm(10,mean=2,sd=.5),
                   i2=rnorm(10,mean=2,sd=.5),
                   i3=rnorm(10,mean=2,sd=.5),
                   i4=rnorm(10,mean=2,sd=.5),
                   i5=rnorm(10,mean=2,sd=.5),
                   i6=rnorm(10,mean=2,sd=.5))
rank_df_to_binary(mydata[,c("i1","i2","i3","i4")],4)
rank_df_to_binary(mydata,3)
```

---

|                |                                                                           |
|----------------|---------------------------------------------------------------------------|
| rank_to_binary | <i>Convert scale to thurstonian binary with n items per ranking block</i> |
|----------------|---------------------------------------------------------------------------|

---

**Description**

Convert scale to thurstonian binary with n items per ranking block

**Usage**

```
rank_to_binary(mydata, items, reverse = TRUE)
```

**Arguments**

|         |                                                                                                                  |
|---------|------------------------------------------------------------------------------------------------------------------|
| mydata  | dataframe                                                                                                        |
| items   | number of items in block                                                                                         |
| reverse | if TRUE assumes that the highest value is first item in rank if FALSE the lowest value is the first item in rank |

**Examples**

```
set.seed(12345)
mydata<-data.frame(i1=round(rnorm(10,mean=2,sd=1),2),
                   i2=round(rnorm(10,mean=2,sd=1),2),
                   i3=round(rnorm(10,mean=2,sd=1),2),
                   i4=round(rnorm(10,mean=2,sd=1),2),
                   i5=round(rnorm(10,mean=2,sd=1),2),
                   i6=round(rnorm(10,mean=2,sd=1),2))
rank_to_binary(mydata[,c("i1","i2","i3")],items=3)
rank_to_binary(mydata[,c("i1","i2","i3")],items=3,reverse=FALSE)
rank_to_binary(mydata,items=3)
```

---

|           |                               |
|-----------|-------------------------------|
| raw_alpha | <i>Cronbach's alpha (raw)</i> |
|-----------|-------------------------------|

---

### Description

Computes Cronbach's alpha directly from the covariance matrix of a unidimensional set of items using the formula  $\alpha = \frac{k}{k-1} \left( 1 - \frac{\sum \text{diag}(\Sigma)}{\sum \Sigma} \right)$ , where  $k$  is the number of items and  $\Sigma$  is the covariance matrix. Pairwise complete observations are used so that missing data on individual items does not discard entire rows.

### Usage

```
raw_alpha(df)
```

### Arguments

|    |                                                                                          |
|----|------------------------------------------------------------------------------------------|
| df | A data frame whose columns are the items of a single scale. All columns must be numeric. |
|----|------------------------------------------------------------------------------------------|

### Value

A single numeric value: Cronbach's alpha for the scale. Values above 0.70 are generally considered acceptable for research purposes.

### Examples

```
set.seed(12345)
df<-data.frame(matrix(.5,ncol=6,nrow=6))
correlation_matrix<-as.matrix(df)
diag(correlation_matrix)<-1
df<-round(generate_correlation_matrix(correlation_matrix,nrows=1000),0)+5
psych::alpha(df)
raw_alpha(df=df)
```

---

|           |                                                            |
|-----------|------------------------------------------------------------|
| rbind_all | <i>Row-bind two data frames with different column sets</i> |
|-----------|------------------------------------------------------------|

---

### Description

Combines two data frames or matrices by rows even when they do not share the same columns. Columns present in one input but absent in the other are added and filled with NA before binding. Row names from both inputs are preserved unless they would produce duplicates, in which case default integer row names are used.

### Usage

```
rbind_all(df1, df2)
```



**Arguments**

df1                    A data frame or matrix.  
df2                    A data frame or matrix.

**Value**

A data frame containing all rows from df1 followed by all rows from df2, with the union of both column sets. Cells where a column did not exist in the original input are NA.

**Examples**

```
df1<-generate_correlation_matrix(n=10)
df2<-generate_correlation_matrix(n=10)
names(df2)[4]<-"X11"
rbind_all(df1=df1,df2=df2)
row.names(df1)<-21:30
rbind_all(df1=df1,df2=df2)
```

---

|                    |                             |
|--------------------|-----------------------------|
| recode_scale_dummy | <i>Scale and dummy code</i> |
|--------------------|-----------------------------|

---

**Description**

Scales numeric variables between 0 and 1 and creates dummy coding for character and factor variables.

**Usage**

```
recode_scale_dummy(df, categories = 10)
```

**Arguments**

df                    Dataframe containing the dataset to be scaled and dummy coded.  
categories            Numeric value representing the number of unique values a vector must have to perform dummy coding. Defaults to 10.

**Details**

This function processes a dataframe by scaling numeric variables and creating dummy codes for character and factor variables. The numeric variables are scaled between 0 and 1, while the character and factor variables are converted to dummy variables if they have fewer unique values than the specified 'categories' parameter.

The function performs the following steps: 1. Identifies numeric variables in the dataframe and scales them. 2. Identifies character and factor variables and creates dummy variables if they meet the criteria. 3. Combines the scaled numeric variables and dummy variables into a single dataframe.

The output is a dataframe with scaled numeric variables and dummy-coded character/factor variables.

## Examples

```
# Example with the 'infert' dataset
recode_scale_dummy(infert)

# Example with a custom dataframe
df<-data.frame(numeric_var=c(1,2,3,4,5),
               factor_var=factor(c('A','B','A','B','C')))
recode_scale_dummy(df)
```

---

remove\_nc

*Replace and remove non-computable values*

---

## Description

Cleans a data frame by replacing non-computable values (NA, NaN, Inf, -Inf, and empty strings) with a chosen replacement, then optionally drops rows or columns that still contain missing values or have zero variance.

## Usage

```
remove_nc(
  df,
  value = NA,
  remove_rows = FALSE,
  aggressive = FALSE,
  remove_cols = FALSE,
  remove_zero_variance = FALSE
)
```

## Arguments

|                      |                                                                                                                                                                                                                                    |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| df                   | A data frame to clean.                                                                                                                                                                                                             |
| value                | The replacement value for all non-computable entries. Default is NA.                                                                                                                                                               |
| remove_rows          | Logical. When TRUE, rows containing NA after replacement are removed according to the aggressive setting. Default is FALSE.                                                                                                        |
| aggressive           | Logical. Only used when remove_rows = TRUE. <ul style="list-style-type: none"> <li>• TRUE — remove a row if <i>any</i> value is NA.</li> <li>• FALSE — remove a row only if <i>all</i> values are NA.</li> </ul> Default is FALSE. |
| remove_cols          | Logical. When TRUE, columns where <i>all</i> values are NA are dropped. Default is FALSE.                                                                                                                                          |
| remove_zero_variance | Logical. Only used when remove_cols = TRUE. When TRUE, columns with only one unique non-missing value (zero variance) are also dropped. Default is FALSE.                                                                          |

**Value**

A data frame with non-computable values replaced and, depending on the flags, rows and/or columns removed.

**Examples**

```
df<-mtcars
df[1,]<-as.numeric(NaN)
df[2,]<-as.numeric(Inf)
df[3,]<-as.numeric(-Inf)
df[4,]<-as.numeric(NA)
df[5,]<-""
remove_nc(df=df,value=NA)
cdf(remove_nc(df=df,value=NA))
df<-generate_missing(mtcars,missing=5)
remove_nc(df,remove_rows=TRUE,aggressive=FALSE)
remove_nc(df,remove_rows=TRUE,aggressive=TRUE)
df<-generate_missing(generate_correlation_matrix(nrows=5),missing=2)
df$X2<-NA
df$X3<-1
remove_nc(df,remove_cols=TRUE,remove_zero_variance=FALSE)
remove_nc(df,remove_cols=TRUE,remove_zero_variance=TRUE)
```

---

|                 |                        |
|-----------------|------------------------|
| remove_outliers | <i>Remove outliers</i> |
|-----------------|------------------------|

---

**Description**

Remove outliers

**Usage**

```
remove_outliers(vector, probs = c(0.25, 0.75), na.rm = TRUE, ...)
```

**Arguments**

|        |                                                  |
|--------|--------------------------------------------------|
| vector | numeric                                          |
| probs  | numeric vector with lowest and highest quantiles |
| na.rm  | if TRUE removes NA values                        |
| ...    | arguments passed to quantile                     |

**Examples**

```
vector<-generate_missing(rnorm(1000))
df<-generate_missing(mtcars[,1:2])
remove_outliers(vector)
data.frame(sapply(df,remove_outliers))
```

---

|                      |                                           |
|----------------------|-------------------------------------------|
| remove_user_packages | <i>Remove all user-installed packages</i> |
|----------------------|-------------------------------------------|

---

### Description

Uninstalls every package that is not part of the R base or recommended distribution. Packages installed in Microsoft R Open (MRO) library paths are also preserved. Only packages with no Priority field (i.e. neither "base" nor "recommended") are removed. **Warning:** this operation is irreversible. All third-party packages will need to be reinstalled afterwards.

### Usage

```
remove_user_packages()
```

### Value

Invisibly returns a named list with one element per removed package (the result of `remove.packages()`). Called primarily for its side effect of uninstalling packages.

---

|                          |                                                           |
|--------------------------|-----------------------------------------------------------|
| replace_na_with_previous | <i>Last observation carried forward (LOCF) imputation</i> |
|--------------------------|-----------------------------------------------------------|

---

### Description

Replaces each NA in a vector with the most recent preceding non-NA value (last observation carried forward, LOCF). If the first element is NA, it is replaced with the first non-NA value found anywhere in the vector. To apply LOCF to every column of a data frame use `df[] <- lapply(df, replace_na_with_previous)`.

### Usage

```
replace_na_with_previous(vector)
```

### Arguments

|        |                                                  |
|--------|--------------------------------------------------|
| vector | A vector of any type that may contain NA values. |
|--------|--------------------------------------------------|

### Value

A vector of the same length and type as vector with NA values replaced by the preceding non-NA element. Returns the original vector unchanged if it contains no NA values.

**Examples**

```
df1<-generate_missing(rnorm(10),missing=5)
df2<-generate_missing(rnorm(10),missing=5)
df3<-generate_missing(rnorm(10),missing=5)
df4<-generate_missing(rnorm(10),missing=5)
df5<-generate_missing(rnorm(10),missing=5)
df<-data.frame(df1,df2,df3,df4,df5)
row.names(df)<-paste0("A",row.names(df))
replace_na_with_previous(df1)
df[]<-lapply(df,replace_na_with_previous)
```

report\_alpha

*Cronbach's alpha reliability report for multiple scales***Description**

Computes Cronbach's alpha and a comprehensive set of item-level reliability statistics for one or more scales using `psych::alpha()`. Supports item reversal, bootstrap confidence intervals, and optional Excel export. Scales are classified by their alpha value (Unacceptable < 0.60, Acceptable 0.60–0.70, Good and Acceptable 0.70–0.80, Good 0.80–0.90, Excellent > 0.90).

**Usage**

```
report_alpha(
  df,
  key = NULL,
  questions = NULL,
  reverse = NULL,
  mini = NULL,
  maxi = NULL,
  file = NULL,
  ...
)
```

**Arguments**

|           |                                                                                                                                                                                                                                                                              |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| df        | A data frame containing all item columns.                                                                                                                                                                                                                                    |
| key       | A named list where each name is a scale label and each element is a character vector of item column names belonging to that scale, e.g. <code>list(f1 = c("X1", "X2", "X3"))</code> . When NULL (default) all columns in df are treated as a single scale named "dimension". |
| questions | A named list (same structure as key) of question label strings to append to item names in the output tables. When NULL (default) only column names are used.                                                                                                                 |
| reverse   | A named list (same structure as key) of numeric sign vectors (1 = keep, -1 = reverse) passed to <code>psych::reverse.code()</code> . When NULL (default) no reversal is applied.                                                                                             |

|      |                                                                                                                                                                                                                                           |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| mini | Numeric scalar specifying the minimum possible scale rating used for item reversal. When NULL (default) the empirical minimum is used.                                                                                                    |
| maxi | Numeric scalar specifying the maximum possible scale rating used for item reversal. When NULL (default) the empirical maximum is used.                                                                                                    |
| file | Character string naming the output Excel file (without extension). When NULL (default) no file is written. The workbook contains four sheets: total statistics, bootstrap CIs (if requested), item statistics, and alpha-if-item-removed. |
| ...  | Additional arguments passed to psych::alpha(), such as cumulative, n.iter (bootstrap iterations), or check.keys.                                                                                                                          |

Value

- A named list with the following elements:
- result\_total** Scale-level statistics including raw alpha, standardised alpha, Guttman’s Lambda 6, average inter-item correlation, signal-to-noise ratio, mean, SD, Kaiser criterion, and alpha classification.
  - result\_boot** Bootstrap confidence intervals for alpha (only populated when n.iter > 1 is passed via ...).
  - result\_item\_statistics** Item-level statistics including corrected and uncorrected item-total correlations, item mean and SD, and response frequencies.
  - result\_dropped** Alpha-if-item-removed statistics for each item within each scale.

Examples

```
set.seed(12345)
df<-data.frame(matrix(.5,ncol=6,nrow=6))
correlation_martix<-as.matrix(df)
diag(correlation_martix)<-1
df<-round(generate_correlation_matrix(correlation_martix,nrows=1000),0)+5
key<-list(f1=c("X1","X2","X3"),
          f2=c("X4","X5","X6"))
reverse<-list(f1=c(1,1,1),
              f2=c(1,1,1))
report_alpha(df=df,key=key,cumulative=TRUE,n.iter=1)
report_alpha(df=df,key=key,reverse=reverse,check.keys=FALSE,n.iter=2)
report_alpha(df=df,key=key,check.keys=FALSE,n.iter=2,file="alpha")
```

---

|            |               |
|------------|---------------|
| report_cfa | <i>Report</i> |
|------------|---------------|

---

Description

Report

Usage

```
report_cfa(model, file = NULL, w = 10, h = 10)
```

**Arguments**

|       |                    |
|-------|--------------------|
| model | lavaan object      |
| file  | output filename    |
| w     | width of pdf file  |
| h     | height of pdf file |

**Examples**

```

model='LATENT=~ITEM1+ITEM2+ITEM3+ITEM4+ITEM5'
df<-lavaan::simulateData(model=model,model.type="cfa",
                          return.type="data.frame",sample.nobs=100)
df<-generate_missing(df)
fit<-lavaan::cfa(model,data=df,missing="ML")
report_cfa(fit)
report_cfa(fit,file="cfa")

```

---

report\_choric\_serial    *Report polychoric tetrachoric polyserial biserial correlation*

---

**Description**

Report polychoric tetrachoric polyserial biserial correlation

**Usage**

```

report_choric_serial(
  x,
  y = NULL,
  file = NULL,
  w = 10,
  h = 10,
  type = "tetrachoric",
  ...
)

```

**Arguments**

- x                      The input may be in one of four forms:
- a) a data frame or matrix of dichotomous data (e.g., the lsat6 from the bock data set) or discrete numerical (i.e., not too many levels, e.g., the big 5 data set, bfi) for polychoric, or continuous for the case of biserial and polyserial
  - b) a 2 x 2 table of cell counts or cell frequencies (for tetrachoric) or an n x m table of cell counts (for both tetrachoric and polychoric)
  - c) a vector with elements corresponding to the four cell frequencies (for tetrachoric)
  - d) a vector with elements of the two marginal frequencies (row and column) and the comorbidity (for tetrachoric)

|      |                                                                                                                                                                                                                                      |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| y    | matrix or dataframe of discrete scores. In the case of tetrachoric, these should be dichotomous, for polychoric not too many levels, for biserial they should be discrete (e.g., item responses) with not too many (<10?) categories |
| file | output filename                                                                                                                                                                                                                      |
| w    | width of pdf file                                                                                                                                                                                                                    |
| h    | height of pdf file                                                                                                                                                                                                                   |
| type | "tetrachoric" "polychoric" "polyserial" "biserial"                                                                                                                                                                                   |
| ...  | arguments passed to psych::polychoric                                                                                                                                                                                                |

### Examples

```
report_choric_serial(generate_data(min=0,max=1,type="uniform"),
                     type="tetrachoric",file="tetrachoric")
report_choric_serial(generate_data(min=1,max=5,type="uniform"),
                     type="polychoric")
report_choric_serial(x=psych::lsat6,y=psych::lsat6,
                     type="polyserial",file="polyserial")
report_choric_serial(x=psych::lsat6,y=psych::lsat6,
                     type="biserial",file="biserial")
```

---

|                    |                                  |
|--------------------|----------------------------------|
| report_correlation | <i>Report correlation matrix</i> |
|--------------------|----------------------------------|

---

### Description

Report correlation matrix

### Usage

```
report_correlation(
  x,
  y = NULL,
  use = "pairwise",
  method = "pearson",
  adjust = "holm",
  alpha = 0.05,
  ci = TRUE,
  file = NULL,
  w = 10,
  h = 10,
  base_size = 20,
  scatterplot = TRUE
)
```



**Arguments**

|             |                                                                                                                                            |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| x           | matrix or dataframe                                                                                                                        |
| y           | a second matrix or dataframe with the same number of rows as x                                                                             |
| use         | "pairwise" is the default value and will do pairwise deletion of cases. "complete" will select just complete cases                         |
| method      | "pearson" "spearman" "kendall"                                                                                                             |
| adjust      | "holm", "hochberg", "hommel", "bonferroni", "BH", "BY", "fdr", "none"                                                                      |
| alpha       | alpha level of confidence intervals                                                                                                        |
| ci          | By default, confidence intervals are found. However, this leads to a great slowdown of speed. So, for just the rs, ts and ps, set ci=FALSE |
| file        | output filename                                                                                                                            |
| w           | width of pdf file                                                                                                                          |
| h           | height of pdf file                                                                                                                         |
| base_size   | base font size                                                                                                                             |
| scatterplot | if TRUE it will output scatterplots                                                                                                        |

**Examples**

```
report_correlation(x=generate_missing(mtcars[,1:3],10))
report_correlation(x=generate_missing(mtcars[,1:3],10),
                    file="correlation",scatterplot=TRUE)
report_correlation(x=mtcars[,1:3],file="correlation")
```

---

|                  |                                                 |
|------------------|-------------------------------------------------|
| report_dataframe | <i>Write matrix or dataframe to excel sheet</i> |
|------------------|-------------------------------------------------|

---

**Description**

Usefull for generic data where conditional formatting of a spesific collumn is required

**Usage**

```
report_dataframe(df, file = NULL, type = "critical_value", ...)
```

**Arguments**

|      |                                                             |
|------|-------------------------------------------------------------|
| df   | dataframe or matrix                                         |
| file | output filename of excel file                               |
| type | "critical_value" "matrix"                                   |
| ...  | arguments passed to excel_critical_value or to excel_matrix |

Examples

```
comment<-list(mpg="Miles/(US) gallon",
              cyl="Number of cylinders",
              disp="Displacement (cu.in.)",
              hp="Gross horsepower",
              drat="Rear axle ratio",
              wt="Weight (1000 lbs)",
              qsec="1/4 mile time",
              vs="Engine (0=V-shaped,1=straight)",
              am="Transmission (0=automatic,1>manual)",
              gear="Number of forward gears",
              carb="Number of carburetors")
report_dataframe(mtcars,sheet="report",file="mtcars",comment=comment,numFmt="#0.00",
                critical=list(am="<0.05"))
report_dataframe(mtcars,sheet="report",file=NULL,comment=comment,numFmt="#0.00",
                critical=list(am="<0.05"))
```

---

|            |                         |
|------------|-------------------------|
| report_efa | <i>Output EFA model</i> |
|------------|-------------------------|

---

Description

Output EFA model

Usage

```
report_efa(
  model,
  df,
  file = NULL,
  w = 10,
  h = 5,
  cut = 0,
  base_size = 10,
  scores = FALSE
)
```

Arguments

|           |                                                    |
|-----------|----------------------------------------------------|
| model     | psych EFA model                                    |
| df        | dataframe                                          |
| file      | output filename                                    |
| w         | width of pdf file                                  |
| h         | height of pdf file                                 |
| cut       | cut point for loadings                             |
| base_size | base font size                                     |
| scores    | if TRUE it will output factor scores in excel file |

**Note**

Orthogonal=varimax, Oblique=oblimin

**Examples**

```
model<-psych::fa(mtcars,nfactors=2,rotate="oblimin",fm="minres",oblique.scores=TRUE)
report_efa(model=model,df=mtcars,file="efa")
model<-psych::fa(mtcars,nfactors=2,rotate="oblimin",fm="uls",oblique.scores=TRUE)
report_efa(model=model,df=mtcars)
model<-psych::fa(mtcars,nfactors=2,rotate="oblimin",fm="ols",oblique.scores=TRUE)
report_efa(model=model,df=mtcars)
model<-psych::fa(mtcars,nfactors=2,rotate="oblimin",fm="wls",oblique.scores=TRUE)
report_efa(model=model,df=mtcars)
model<-psych::fa(mtcars,nfactors=2,rotate="oblimin",fm="gls",oblique.scores=TRUE)
report_efa(model=model,df=mtcars)
model<-psych::fa(mtcars,nfactors=2,rotate="oblimin",fm="pa",oblique.scores=TRUE)
report_efa(model=model,df=mtcars)
model<-psych::fa(mtcars,nfactors=2,rotate="oblimin",fm="ml",oblique.scores=TRUE)
report_efa(model=model,df=mtcars)
model<-psych::fa(mtcars,nfactors=2,rotate="oblimin",fm="minchi",oblique.scores=TRUE)
report_efa(model=model,df=mtcars)
model<-psych::fa(mtcars,nfactors=2,rotate="oblimin",fm="minrank",oblique.scores=TRUE)
report_efa(model=model,df=mtcars)
model<-psych::fa(mtcars,nfactors=2,rotate="oblimin",fm="old.min",oblique.scores=TRUE)
report_efa(model=model,df=mtcars)
model<-psych::fa(mtcars,nfactors=2,rotate="oblimin",fm="alpha",oblique.scores=TRUE)
#report_efa(model=model,df=mtcars)
```

---

report\_factorial\_anova

*Plot means with standard error for every level in a dataframe*

---

**Description**

Plot means with standard error for every level in a dataframe

**Usage**

```
report_factorial_anova(
  df,
  dv,
  wid,
  within = NULL,
  within_full = NULL,
  between = NULL,
  within_covariates = NULL,
  between_covariates = NULL,
  observed = NULL,
```

```

diff = NULL,
reverse_diff = FALSE,
type = 3,
white.adjust = TRUE,
detailed = TRUE,
return_aov = TRUE,
file = NULL,
post_hoc_test = TRUE,
base_size = 15
)

```

### Arguments

|                    |                                                                                                                                                     |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| df                 | dataframe                                                                                                                                           |
| dv                 | names of dependent variables                                                                                                                        |
| wid                | names of                                                                                                                                            |
| within             | names of within factors                                                                                                                             |
| within_full        | names of within factors after data are collapsed to means per condition                                                                             |
| between            | names of between factors                                                                                                                            |
| within_covariates  | names of within covariates                                                                                                                          |
| between_covariates | names of between covariates                                                                                                                         |
| observed           | names in data that are already specified in either within or between that contain predictor variables that are observed variables (not manipulated) |
| diff               | names of variables to collapse in a different score                                                                                                 |
| reverse_diff       | If TRUE, triggers reversal of the difference collapse requested by diff                                                                             |
| type               | sum of squares 1 2 3                                                                                                                                |
| white.adjust       | if TRUE corrects for heteroscedasticity                                                                                                             |
| detailed           | if TRUE returns detailed information                                                                                                                |
| return_aov         | if TRUE returns aov object                                                                                                                          |
| file               | output filename                                                                                                                                     |
| post_hoc_test      | if TRUE outputs post hoc in file                                                                                                                    |
| base_size          | base font size                                                                                                                                      |

### Examples

```

set.seed(12345)
df<-data.frame(id=rep(seq(1,80),each=81,1),
               IV1=rep(LETTERS[1:3],each=1,2160),
               IV2=rep(LETTERS[4:6],each=3,720),
               IV3=rep(LETTERS[7:9],each=9,240),
               IV4=rep(LETTERS[10:12],each=27,80),
               stringsAsFactors=FALSE)
cdf<-data.frame(matrix(.01,ncol=4,nrow=4))

```

```

correlation_matrix<-as.matrix(cdf)
diag(correlation_matrix)<-1
cdf<-generate_correlation_matrix(correlation_matrix,nrows=nrow(df))+10
names(cdf)<-paste0("DV",1:4)
df<-data.frame(df,cdf)
df$DV2<-df$DV2+10
df$DV3<-df$DV3+20
df$DV4<-df$DV4+30
df[df$IV1=="A",]$DV1<-df[df$IV1=="A",]$DV1+1
df[df$IV1=="B",]$DV1<-df[df$IV1=="B",]$DV1+2
df[df$IV1=="C",]$DV1<-df[df$IV1=="C",]$DV1+3
cdf(df)
r1<-report_factorial_anova(df=df,wid="id",dv=c("DV1","DV2"),
                           within=c("IV1","IV2"),within_full=c("IV1","IV2"),
                           between=NULL,
                           within_covariates=NULL,between_covariates=NULL,
                           file="anova_within",
                           post_hoc=TRUE)
r2<-report_factorial_anova(df=df,wid="id",dv=c("DV1","DV2"),
                           within=NULL,within_full=NULL,
                           between=c("IV1","IV2"),
                           within_covariates=NULL,between_covariates=NULL,
                           file="anova_between",
                           post_hoc=TRUE)
r3<-report_factorial_anova(df=df,wid="id",dv=c("DV1","DV2"),
                           within=c("IV3","IV4"),within_full=c("IV3","IV4"),
                           between=c("IV1","IV2"),
                           within_covariates=NULL,between_covariates=NULL,
                           file="anova_mixed",
                           post_hoc=FALSE)
r4<-report_factorial_anova(df=df,wid="id",dv=c("DV1","DV2"),
                           within=c("IV1","IV2"),within_full=c("IV1","IV2"),
                           between=NULL,
                           within_covariates=c("DV3","DV4"),between_covariates=NULL,
                           file="anova_within_cov",
                           post_hoc=TRUE)

```

---

report\_hlr

*Report HLR*


---

## Description

Report HLR

## Usage

```

report_hlr(
  df,
  corlist,
  factorlist,

```

```
    predictor,  
    random_effect,  
    file = NULL,  
    sheet = "report"  
  )
```

**Arguments**

|               |                              |
|---------------|------------------------------|
| df            | dataframe                    |
| corlist       | Numeric outcome index        |
| factorlist    | Numeric predictor index      |
| predictor     | Character predictor name     |
| random_effect | Character random effect name |
| file          | Character file               |
| sheet         | Character sheet              |

**Examples**

```
report_hlr(df=infert,corlist=8,factorlist=1,  
           predictor="case",random_effect="case")
```

---

|            |                             |
|------------|-----------------------------|
| report_irt | <i>Output for irt model</i> |
|------------|-----------------------------|

---

**Description**

Output for irt model

**Usage**

```
report_irt(model, m2 = TRUE, file = NULL)
```

**Arguments**

|       |                              |
|-------|------------------------------|
| model | object mirt                  |
| m2    | if TRUE report m2 statistics |
| file  | output filename              |

**Examples**

```
set.seed(12345)  
cormatrix<-psych::sim.rasch(nvar=5,n=50000,low=-4,high=4,d=NULL,a=1,mu=0,sd=1)$items  
irt_onefactor<-mirt::mirt(cormatrix,1,empiricalhist=TRUE,calcNull=TRUE)  
irt_twofactor<-mirt::mirt(cormatrix,2,empiricalhist=TRUE,calcNull=TRUE)  
irt_threefactor<-mirt::mirt(cormatrix,3,empiricalhist=TRUE,calcNull=TRUE)  
report_irt(model=irt_onefactor,file="one_factor")  
report_irt(model=irt_twofactor,file="two_factors")  
report_irt(model=irt_threefactor,file="three_factors")
```

---

|            |                             |
|------------|-----------------------------|
| report_lda | <i>Report for MASS::lda</i> |
|------------|-----------------------------|

---

**Description**

Report for MASS::lda

**Usage**

```
report_lda(model, file = NULL, w = 10, h = 10, base_size = 10, title = "")
```

**Arguments**

|           |                       |
|-----------|-----------------------|
| model     | object from MASS::lda |
| file      | output filename       |
| w         | width of pdf file     |
| h         | height of pdf file    |
| base_size | base font size        |
| title     | plot title            |

**Examples**

```
model<-MASS::lda(case~.,data=infert)
result<-report_lda(model=model)
result<-report_lda(model=model,file="lda")
model<-MASS::lda(Species~.,data=iris)
result<-report_lda(model=model,file="lda")
```

---

|                 |                                   |
|-----------------|-----------------------------------|
| report_logistic | <i>Report logistic regression</i> |
|-----------------|-----------------------------------|

---

**Description**

Report logistic regression

**Usage**

```
report_logistic(
  model,
  validation_data = NULL,
  file = NULL,
  title = "",
  w = 10,
  h = 10,
  base_size = 10,
  fast = FALSE
)
```

**Arguments**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| model           | object glm                                                      |
| validation_data | validation data                                                 |
| file            | output filename                                                 |
| title           | plot title                                                      |
| w               | width of pdf file. Relevant only when file string is not empty  |
| h               | height of pdf file. Relevant only when file string is not empty |
| base_size       | base font size                                                  |
| fast            | if TRUE it will not output individual scores and residuals      |

**Note**

(1) Problematic values for standardized residuals  $> \pm 1.96$

Standardized residuals are residuals divided by an estimated standard deviation and they can be interpreted as z scores in that:

95 99 99.99 (2) Problematic values for  $dfBeta \geq 1$

$dfBeta$  estimates coefficients if the respective case is removed from the dataset

(3) Problematic values for Hat values (leverage) 2 or 3 times the average  $(k+1)/n$

Hat values (leverage), gauge the influence of the observed value of the outcome variable over the predicted values

The average leverage value is defined as  $(k+1)/n$ ,  $k$ =number of predictors,  $n$ =number of participants. Leverage values lie between 0 (no influence) and 1 (complete influence over prediction)

If no cases exert undue influence over the model then all leverage values should be close to  $(k+1)/n$  Hoaglin and Welsch (1978) recommends investigating cases with values greater than twice the average  $2(k+1)/n$

Stevens (2002) recommends investigating cases with values greater than three times the average  $3(k+1)/n$

(4) Problematic values for VIFs  $> 10$

**ASSUMPTIONS**

(1) Linearity between continous predictors and the logit (test wether the interaction term between the predictor and its log transformation is significant)

(2) Independence of errors

(3) No multicollinearity

**Examples**

```
modelcategoricalpredictor0<-glm(case~education,data=infert,family=binomial)
modelcategoricalpredictor1<-glm(case~education,data=infert,family=gaussian)
#modelcategoricalpredictor2<-glm(case~education,data=infert,family=Gamma)
#modelcategoricalpredictor3<-glm(case~education,data=infert,family=inverse.gaussian)
modelcategoricalpredictor4<-glm(case~education,data=infert,family=poisson)
modelcategoricalpredictor5<-glm(case~education,data=infert,family=quasi)
modelcategoricalpredictor6<-glm(case~education,data=infert,family=quasibinomial)
modelcategoricalpredictor7<-glm(case~education,data=infert,family=quasipoisson)
modelcontinuouspredictor0<-glm(case~stratum,data=infert,family=binomial)
```



```

modeltwopredictors0<-glm(case~education+stratum,data=infert,family=binomial)
modeltwopredictors1<-glm(case~education+stratum,data=infert,family=gaussian)
#modeltwopredictors2<-glm(case~education+stratum,data=infert,family=Gamma)
#modeltwopredictors3<-glm(case~education+stratum,data=infert,family=inverse.gaussian)
modeltwopredictors4<-glm(case~education+stratum,data=infert,family=poisson)
modeltwopredictors5<-glm(case~education+stratum,data=infert,family=quasi)
modeltwopredictors6<-glm(case~education+stratum,data=infert,family=quasibinomial)
modeltwopredictors7<-glm(case~education+stratum,data=infert,family=quasipoisson)
report_logistic(model=modelcategoricalpredictor0)
report_logistic(model=modelcategoricalpredictor1)
#report_logistic(model=modelcategoricalpredictor2)
#report_logistic(model=modelcategoricalpredictor3)
report_logistic(model=modelcategoricalpredictor4)
report_logistic(model=modelcategoricalpredictor5)
report_logistic(model=modelcategoricalpredictor6)
report_logistic(model=modelcategoricalpredictor7)
report_logistic(model=modelcontinuouspredictor0)
report_logistic(model=modeltwopredictors0)
report_logistic(model=modelcategoricalpredictor0,
                 file="logistic_categorical_predictor",
                 validation_data=infert)
report_logistic(model=modelcontinuouspredictor0,
                 file="logistic_continuous_predictor",
                 validation_data=infert)
report_logistic(model=modeltwopredictors0,
                 file="logistic_two_predictors",
                 validation_data=infert[1:10,])

```

---

report\_manova

*Manova result*


---

## Description

Manova result

## Usage

```
report_manova(model, file = NULL)
```

## Arguments

|       |                        |
|-------|------------------------|
| model | object of manova model |
| file  | output filename        |

## Note

Pillai-Bartlett trace (V): Represents the sum of the proportion of explained variance on the discriminant functions. As such, it is similar to the ratio of  $SS_M / SS_T$ , which is known as  $R^2$ .

Hotelling's  $T^2$ : Represents the sum of the eigenvalues for each variate it compares directly to the

F-ratio in ANOVA

Wilks-s lambda (L): Represents the ratio of error variance to total variance ( $SS_R / SS_T$ ) for each variate.

Roy-s largest root: Represents the proportion of explained variance to unexplained variance ( $SS_M / SS_R$ ) for the first discriminant function.

#### ASSUMPTIONS

Independence: Observations should be statistically independent.

Random sampling: Data should be randomly sampled from the population of interest and measured at an interval level.

Multivariate normality: In ANOVA, we assume that our dependent variable is normally distributed within each group. In the case of MANOVA, we assume that the dependent variables (collectively) have multivariate normality within groups.

Homogeneity of covariance matrices: In ANOVA, it is assumed that the variances in each group are roughly equal (homogeneity of variance). In MANOVA we must assume that this is true for each dependent variable, but also that the correlation between any two dependent variables is the same in all groups. This assumption is examined by testing whether the population variance-covariance matrices of the different groups in the analysis are equal.

### Examples

```
## Set orthogonal contrasts.
op<-options(contrasts=c("contr.helmert","contr.poly"))
model_mixed<-manova(cbind(yield,foo)~N*P*K,within(npk,foo<-rnorm(24)))
model_between<-manova(cbind(rnorm(24),rnorm(24))~round(rnorm(24),0)*round(rnorm(24),0))
report_manova(model=model_mixed)
report_manova(model=model_between)
```

---

```
report_normality_tests
```

*Normality tests*

---

### Description

Shapiro-Wilk Anderson-Darling Cramer-von-Mises Shapiro-Francia  
Jarque-Bera Kolmogorov-Smirnov Lilliefors Pearson X2

### Usage

```
report_normality_tests(df, file = NULL)
```

### Arguments

|      |                                          |
|------|------------------------------------------|
| df   | dataframe with continous or ordinal data |
| file | output filename                          |

### Details

returns xlsx file

**Examples**

```
vector<-generate_missing(rnorm(1000))
df<-generate_missing(mtcars[,1:2])
report_normality_tests(df=df)
report_normality_tests(df=vector,file="normality_tests")
```

---

|               |                |
|---------------|----------------|
| report_oneway | <i>One way</i> |
|---------------|----------------|

---

**Description**

One way

**Usage**

```
report_oneway(
  df,
  dv,
  iv,
  file = NULL,
  w = 10,
  h = 10,
  base_size = 10,
  note = "",
  title = "",
  type = "ci",
  plot_means = FALSE,
  plot_diagnostics = FALSE
)
```

**Arguments**

|                  |                                                                             |
|------------------|-----------------------------------------------------------------------------|
| df               | A data frame containing both the independent and dependent variables.       |
| dv               | Integer vector of column indices for the continuous dependent variables.    |
| iv               | Integer vector of column indices for the categorical independent variables. |
| file             | output filename                                                             |
| w                | width of pdf file                                                           |
| h                | height of pdf file                                                          |
| base_size        | base font size                                                              |
| note             | text for footnote                                                           |
| title            | plot title                                                                  |
| type             | type of bar to display "se" "ci" "sd" ""                                    |
| plot_means       | if TRUE it will output mean plots and descriptives for plots                |
| plot_diagnostics | if TRUE it will output ANOVA diagnostics plots                              |

**Note**

- (1) The Fisher procedure assumes homoscedasticity
- (2) The Welch procedure does not assume homoscedasticity
- (3) The Kruskal Wallis procedure does not assume normality but it is not an alternative for violations of homoscedasticity
- (4) Posthoc Tuckey: not good for unequal sample sizes or homoscedasticity
- (5) Posthoc Games Howell: good for unequal sample sizes and homoscedasticity

**Examples**

```
report_oneway(df=df_blood_pressure,
              dv=c(which("bp_before"==names(df_blood_pressure)),
                    which("bp_after"==names(df_blood_pressure))),
              iv=c(which("sex"==names(df_blood_pressure)),
                    which("agegrp"==names(df_blood_pressure))),
              file="anova",
              plot_diagnostics=FALSE,
              plot_means=FALSE)
report_oneway(df=mtcars,dv=2:4,iv=9:10,file="anova_oneway_two_factor")
report_oneway(df=mtcars,dv=2:4,iv=9,file="anova_oneway_one_factor")
report_oneway(df=mtcars,dv=2:4,iv=9,file="anova_oneway_one_factor",
              plot_means=TRUE,plot_diagnostics=TRUE)
```

report\_pdf

*Report pdf***Description**

Report pdf

**Usage**

```
report_pdf(
  ...,
  plotlist = NULL,
  file = NULL,
  title = NULL,
  w = 10,
  h = 10,
  print_plot = TRUE
)
```

**Arguments**

|          |                      |
|----------|----------------------|
| ...      | plot objects         |
| plotlist | list of plot objects |
| file     | output filename      |

|            |                                           |
|------------|-------------------------------------------|
| title      | output filename                           |
| w          | width of pdf file                         |
| h          | height of pdf file                        |
| print_plot | if TRUE it prints plot on graphics device |

### Examples

```
p1<-ggplot(ChickWeight,aes(x=Time,y=weight,colour=Diet,group=Chick))+
  geom_line()+
  ggtitle("Growth curve for individual chicks")+
  theme_bw()
p2<-ggplot(ChickWeight,aes(x=Time,y=weight,colour=Diet))+
  geom_point(alpha=.3)+
  geom_smooth(alpha=.2,size=1,method="loess",formula="y~x")+
  ggtitle("Fitted growth curve per diet")+theme_bw()
cars_plot_multiplot<-plot_multiplot(plotlist=plot_histogram(mtcars[,1:4]),cols=2)
cars_plot_base<-plot_normality_diagnostics(mtcars)
report_pdf(p1,p2,print_plot=TRUE)
report_pdf(p1,p2,file="report",print_plot=FALSE)
report_pdf(plotlist=cars_plot_multiplot,print_plot=TRUE)
report_pdf(plotlist=cars_plot_multiplot,file="report",print_plot=FALSE)
report_pdf(plotlist=cars_plot_base,print_plot=TRUE)
report_pdf(plotlist=cars_plot_base,file="report",print_plot=FALSE)
```

---

|                   |                   |
|-------------------|-------------------|
| report_regression | <i>Regression</i> |
|-------------------|-------------------|

---

### Description

Regression

### Usage

```
report_regression(
  model,
  base_size = 10,
  title = "",
  file = NULL,
  w = 10,
  h = 10,
  plot_diagnostics = TRUE
)
```

### Arguments

|           |                |
|-----------|----------------|
| model     | object ml      |
| base_size | base font size |

|                  |                                                                 |
|------------------|-----------------------------------------------------------------|
| title            | plot title                                                      |
| file             | output filename                                                 |
| w                | width of pdf file. Relevant only when file string is not empty  |
| h                | height of pdf file. Relevant only when file string is not empty |
| plot_diagnostics | if TRUE it will output linear model diagnostics plots           |

### Note

(1) Problematic values for standardized residuals  $> \pm 1.96$

**\*\*Standardized residuals\*\*** are residuals divided by an estimated standard deviation and they can be interpreted as z scores in that:

- 95.00 - 99.00 - 99.99 (2) **\*\*Studentized residuals\*\*** indicate the the ability of the model to predict that case. They follow a t distribution

(3) **\*\*DFFits\*\*** indicate the difference between the adjusted predicted value and the original predicted value. Adjusted predicted value for a case refers to the predicted value of that case, when that case is excluded from model fit.

(4) **\*\*Cook's distance\*\*** indicates leverage. Problematic values for cook's distance  $> 1$  Cook and Weisberg (1982).

(5) **\*\*Hat values\*\*** indicate leverage. Problematic values for Hat values 2 or 3 times the average  $(k+1)/n$

The average leverage value is defined as  $(k+1)/n$ ,  $k$ =number of predictors,  $n$ =number of participants. Leverage values lie between 0 (no influence) and 1 (complete influence over prediction)

- Hoaglin and Welsch (1978) recommends investigating cases with values greater than twice the average  $2(k+1)/n$

- Stevens (2002) recommends investigating cases with values greater than three times the average  $3(k+1)/n$

**\*\*T-tests\*\*** test the hypothesis that b's are different from 0

**\*\*Multiple R<sup>2</sup>\*\***: Variance Explained

**\*\*Adjusted R<sup>2</sup>\*\***: Indicates how much variance in Y would be accounted for if the model is derived from the population from which the sample was taken. Ideally,  $R^2 = \text{Adjusted } R^2$

**\*\*F-Statistic\*\*** tests the null hypothesis is that the overall model has no effect

**\*\*Covariance ratios\*\*** critical values  $\text{CVR} > 1 + [3(k+1)/n]$   $\text{CRV} < 1 - [3(k+1)/n]$ . In general we should obtain small values or we may have to remove cases

**\*\*ASSUMPTIONS\*\***

(1) variable types: All predictors must be quantitative or categorical (with two levels), and the outcome variable must be quantitative (interval data), continuous and unbounded (no constraints on the variability of the outcome) (2) Non-zero variance

(3) No perfect multicollinearity

(4) Predictors are uncorrelated with -external variables-

(5) Homoscedasticity: At each level of the predictor variable(s), the variance of the residual terms should be constant. Residuals at each level of the predictor(s) should have similar variance (homoscedasticity)

(6) Independent errors: For any two observations the residual terms should be uncorrelated (or independent)

This eventuality is sometimes described as a lack of autocorrelation. This assumption can be tested with the Durbin-Watson test, which tests for serial correlations between errors. Specifically, it tests whether adjacent residuals are correlated The size of the Durbin-Watson statistic depends upon the

number of predictors in the model and the number of observations. As a very conservative rule of thumb, values less than 1 or greater than 3 are definitely cause for concern; however, values closer to 2 may still be problematic depending on your sample and model. R also provides a p-value of the autocorrelation. Be very careful with the Durbin-Watson test, though, as it depends on the order of the data: if you reorder your data, you'll get a different value.

(7) Normally distributed errors: It is assumed that the residuals in the model are random, normally distributed variables with a mean of 0.

(8) Independence: It is assumed that all of the values of the outcome variable are independent (in other words, each value of the outcome variable comes from a separate entity).

(9) Linearity: The mean values of the outcome variable for each increment of the predictor(s) lie along a straight line.

## Examples

```
form<-formula(mpg~qsec)
regressionmodel<-lm(form,data=mtcars)
multipleregressionmodel<-lm(mpg~qsec*hp*wt*drat,data=mtcars)
res<-report_regression(model=regressionmodel,plot_diagnostics=TRUE)
res<-report_regression(model=multipleregressionmodel)
res<-report_regression(model=regressionmodel,file="regression")
res<-report_regression(model=multipleregressionmodel,
                        file="regression",
                        plot_diagnostics=TRUE)
```

---

report\_ttests

*Run Pairwise t-tests and Return a Reporting Table*

---

## Description

Performs t-tests for each selected dependent variable against each selected independent variable, across all pairwise level combinations of the independent variable. Also computes descriptive statistics, effect sizes, Bartlett homogeneity test results, and Bonferroni adjustment.

In simple terms: this function creates a full t-test report table you can export or use in downstream summaries.

## Usage

```
report_ttests(df, dv, iv, file = NULL, ...)
```

## Arguments

|      |                                                                             |
|------|-----------------------------------------------------------------------------|
| df   | A data frame containing both the independent and dependent variables.       |
| dv   | Integer vector of column indices for the continuous dependent variables.    |
| iv   | Integer vector of column indices for the categorical independent variables. |
| file | output filename                                                             |
| ...  | Arguments passed on to <code>stats::t.test</code>                           |

`x` a (non-empty) numeric vector of data values.  
`y` an optional (non-empty) numeric vector of data values.  
`alternative` a character string specifying the alternative hypothesis, must be one of "two.sided" (default), "greater" or "less". You can specify just the initial letter.  
`mu` a number indicating the true value of the mean (or difference in means if you are performing a two sample test).  
`paired` a logical indicating whether you want a paired t-test.  
`var.equal` a logical variable indicating whether to treat the two variances as being equal. If TRUE then the pooled variance is used to estimate the variance otherwise the Welch (or Satterthwaite) approximation to the degrees of freedom is used.  
`conf.level` confidence level of the interval.  
`formula` a formula of the form `lhs ~ rhs` where `lhs` is a numeric variable giving the data values and `rhs` either 1 for a one-sample or paired test or a factor with two levels giving the corresponding groups. If `lhs` is of class "Pair" and `rhs` is 1, a paired test is done, see Examples.  
`data` an optional matrix or data frame (or similar: see [model.frame](#)) containing the variables in the formula `formula`. By default the variables are taken from `environment(formula)`.  
`subset` an optional vector specifying a subset of observations to be used.  
`na.action` a function which indicates what should happen when the data contain NAs.

## Details

Missing values are removed per analysis pair using complete cases on the current dependent and independent variables.

For each independent variable, all pairwise level combinations are tested using `utils::combn`.

The function also calls `report_dataframe` to generate a formatted report.

## Value

A data frame where each row is one pairwise group comparison for one dependent-independent variable combination. Returned columns mean:

- DV: Name stored in the DV column by the current implementation. Note: this currently contains the independent variable name.
- IV: Name stored in the IV column by the current implementation. Note: this currently contains the dependent variable name.
- level1: First group level being compared.
- level2: Second group level being compared.
- n1: Sample size in level1.
- n2: Sample size in level2.
- t: t statistic from t.test.



- df: Degrees of freedom for the t statistic.
- p: p-value from t.test.
- CI\_l: Lower confidence interval bound for the mean difference.
- CI\_u: Upper confidence interval bound for the mean difference.
- alternative: Alternative hypothesis used by t.test.
- method: Test label from t.test (for example Welch Two Sample t-test).
- mean1: Mean of the dependent variable in level1.
- mean2: Mean of the dependent variable in level2.
- sd1: Standard deviation in level1.
- sd2: Standard deviation in level2.
- sd\_pooled: Pooled standard deviation,  $\sqrt{(sd1^2 + sd2^2) / 2}$ .
- d: Cohen d effect size,  $\text{abs}(\text{mean2} - \text{mean1}) / \text{sd\_pooled}$ .
- r: Effect-size r derived from d using the function formula.
- k\_squared[bartlett]: Bartlett test statistic for equal variances.
- df[bartlett]: Degrees of freedom of Bartlett test.
- p[bartlett]: p-value of Bartlett test. Small values suggest heteroscedasticity.
- bonferroni\_p: Bonferroni-adjusted alpha threshold computed for the number of tests in the output table.
- significant: Logical-like character flag (TRUE/FALSE) indicating whether p is below bonferroni\_p.

## Examples

```
report_ttests(df=df_insurance,
              dv=which("charges"==names(df_insurance)),
              iv=c(2))
report_ttests(df=df_insurance,
              dv=which("charges"==names(df_insurance)),
              iv=c(4))
report_ttests(df=df_insurance,
              dv=which("charges"==names(df_insurance)),
              iv=c(2,4))
report_ttests(df=df_insurance,
              dv=which("charges"==names(df_insurance)),
              iv=c(2,4),
              alternative="two.sided")
report_ttests(df=df_insurance,
              dv=which("charges"==names(df_insurance)),
              iv=c(2,4),
              alternative="less")
report_ttests(df=df_insurance,
              dv=which("charges"==names(df_insurance)),
              iv=c(2,4),
              alternative="greater")
report_ttests(df=df_insurance,
```

```

      dv=which("charges"==names(df_insurance)),
      iv=c(2,4),
      var.equal=TRUE)
report_ttests(df=mtcars,dv=1:7,iv=8:10,var.equal=TRUE,file="ttest")

```

report\_wtests

*Run Pairwise Wilcoxon Tests and Return a Reporting Table***Description**

Performs Wilcoxon rank-sum tests for each selected dependent variable against each selected independent variable, across all pairwise level combinations of the independent variable. Also computes descriptive statistics, effect sizes, Bartlett homogeneity results, and Bonferroni adjustment.

In simple terms: this function builds a full nonparametric comparison table, similar to `report_ttests`, but using `wilcox.test` for group differences.

**Usage**

```
report_wtests(df, dv, iv, file = NULL, ...)
```

**Arguments**

|      |                                                                             |
|------|-----------------------------------------------------------------------------|
| df   | A data frame containing both the independent and dependent variables.       |
| dv   | Integer vector of column indices for the continuous dependent variables.    |
| iv   | Integer vector of column indices for the categorical independent variables. |
| file | output filename                                                             |
| ...  | Arguments passed on to <code>stats::wilcox.test</code>                      |

x numeric vector of data values. Non-finite (e.g., infinite or missing) values will be omitted.  
 y an optional numeric vector of data values: as with x non-finite values will be omitted.  
 alternative a character string specifying the alternative hypothesis, must be one of "two.sided" (default), "greater" or "less". You can specify just the initial letter.  
 mu a number specifying an optional parameter used to form the null hypothesis. See 'Details'.  
 paired a logical indicating whether you want a paired test.  
 exact a logical indicating whether an exact p-value should be computed.  
 correct a logical indicating whether to apply continuity correction in the normal approximation for the p-value, or an integer *k* between 0 and 3 giving the number of correction terms to use from the Edgeworth series for the normal approximation.  
 conf.int a logical indicating whether a confidence interval should be computed.

`conf.level` confidence level of the interval.

`tol.root` (when `conf.int` is true:) a positive numeric tolerance, used in `uniroot`(\*, `tol=tol.root`) calls.

`digits.rank` a number; if finite, `rank(signif(r, digits.rank))` will be used to compute ranks for the test statistic instead of (the default) `rank(r)`.

`formula` a formula of the form `lhs ~ rhs` where `lhs` is a numeric variable giving the data values and `rhs` either 1 for a one-sample or paired test or a factor with two levels giving the corresponding groups. If `lhs` is of class `"Pair"` and `rhs` is 1, a paired test is done, see Examples.

`data` an optional matrix or data frame (or similar: see `model.frame`) containing the variables in the formula `formula`. By default the variables are taken from `environment(formula)`.

`subset` an optional vector specifying a subset of observations to be used.

`na.action` a function which indicates what should happen when the data contain `NA`s.

## Details

Missing values are removed per analysis pair using complete cases on the current dependent and independent variables.

For each independent variable, all pairwise level combinations are tested using `utils::combn`.

The function calls `stats::wilcox.test` with `conf.int = TRUE` and forwards additional arguments through ....

The function also calls `report_dataframe` to generate a formatted report.

## Value

A data frame where each row is one pairwise level comparison for one dependent-independent variable combination. Returned columns mean:

- `DV`: Name stored in the `DV` column by the current implementation. Note: this currently contains the independent variable name.
- `IV`: Name stored in the `IV` column by the current implementation. Note: this currently contains the dependent variable name.
- `level1`: First group level being compared.
- `level2`: Second group level being compared.
- `n1`: Sample size in `level1`.
- `n2`: Sample size in `level2`.
- `W`: Wilcoxon test statistic from `wilcox.test`.
- `p`: p-value from `wilcox.test`.
- `CI_l`: Lower confidence interval bound from `wilcox.test`.
- `CI_u`: Upper confidence interval bound from `wilcox.test`.
- `alternative`: Alternative hypothesis used by `wilcox.test`.
- `method`: Test label from `wilcox.test`.

- mean1: Mean of the dependent variable in level1.
- mean2: Mean of the dependent variable in level2.
- sd1: Standard deviation in level1.
- sd2: Standard deviation in level2.
- sd\_pooled: Pooled standard deviation,  $\sqrt{(sd1^2 + sd2^2) / 2}$ .
- d: Cohen d effect size,  $\text{abs}(\text{mean2} - \text{mean1}) / \text{sd\_pooled}$ .
- r: Wilcoxon effect size from `rstatix::wilcox_effsize`.
- k\_squared[bartlett]: Bartlett test statistic for equal variances.
- df[bartlett]: Degrees of freedom of Bartlett test.
- p[bartlett]: p-value of Bartlett test. Small values suggest heteroscedasticity.
- bonferroni\_p: Bonferroni-adjusted alpha threshold computed for the number of tests in the output table.
- significant: Logical-like character flag (TRUE/FALSE) indicating whether p is below bonferroni\_p.

### Examples

```
report_wtests(df=df_insurance,
              dv=which("charges"==names(df_insurance)),
              iv=c(2))
report_wtests(df=df_insurance,
              dv=which("charges"==names(df_insurance)),
              iv=c(4))
report_wtests(df=df_insurance,
              dv=which("charges"==names(df_insurance)),
              iv=c(2,4))
report_wtests(df=df_insurance,
              dv=which("charges"==names(df_insurance)),
              iv=c(2,4),
              alternative="two.sided")
report_wtests(df=df_insurance,
              dv=which("charges"==names(df_insurance)),
              iv=c(2,4),
              alternative="less")
report_wtests(df=df_insurance,
              dv=which("charges"==names(df_insurance)),
              iv=c(2,4),
              alternative="greater")
report_wtests(df=df_insurance,
              dv=which("charges"==names(df_insurance)),
              iv=c(2,4),
              var.equal=TRUE)
report_wtests(df=df_insurance,
              dv=which("charges"==names(df_insurance)),
              iv=c(2,4),
              var.equal=TRUE,
              file="wilcoxontest")
```

---

|                |                                      |
|----------------|--------------------------------------|
| report_xgboost | <i>Report for xgboost::xgb.train</i> |
|----------------|--------------------------------------|

---

## Description

Report for xgboost::xgb.train

## Usage

```
report_xgboost(
  model,
  validation_data = NULL,
  label = NULL,
  file = "xgboost",
  w = 10,
  h = 10,
  base_size = 10,
  title = "",
  fast = FALSE
)
```

## Arguments

|                 |                                              |
|-----------------|----------------------------------------------|
| model           | object from xgboost::xgb.train               |
| validation_data | validation data                              |
| label           | outcome variable name                        |
| file            | output filename                              |
| w               | width of pdf file                            |
| h               | height of pdf file                           |
| base_size       | base font size                               |
| title           | plot title                                   |
| fast            | if TRUE error values are not saved in output |

## Examples

```
infern_formula<-formula(case~education+spontaneous+induced)
boston_formula<-formula(medv~crim+zn+indus+chas+nox+rm+age+dis+rad+tax+ptratio+black+lstat)
train_test_classification<-k_fold(df=infern,model_formula=infern_formula)
train_test_regression<-k_fold(df=MASS::Boston,model_formula=boston_formula)
xgb_classification<-xgboost::xgb.train(
  params=xgboost::xgb.params(objective="binary:logistic"),
  data=train_test_classification$xgb$f1$train,
  evals=train_test_classification$xgb$f1$watchlist,
  nround=20)
xgb_regression<-xgboost::xgb.train(
```

```

        data=train_test_regression$rgb$f1$train,
        evals=train_test_regression$rgb$f1$watchlist,
        nround=20)
## Not run:
report_xgboost(model=xgb_classification,
               validation_data=train_test_classification$f$test$f1,
               label=train_test_classification$outcome,
               file="Classification")
report_xgboost(model=xgb_regression,
               validation_data=train_test_regression$f$test$f1,
               label=train_test_regression$outcome,
               file="Regression")

## End(Not run)
```

---

|                    |                                                               |
|--------------------|---------------------------------------------------------------|
| response_dimension | <i>index parameter and items relative to their dimensions</i> |
|--------------------|---------------------------------------------------------------|

---

**Description**

index parameter and items relative to their dimensions

**Usage**

```
response_dimension(response, dimensions, items)
```

**Arguments**

|            |                               |
|------------|-------------------------------|
| response   | vector one to number of items |
| dimensions | number of dimensions          |
| items      | item comparisons              |

**Examples**

```
response_dimension(c(1:18),3,c(1,2))
response_dimension(c(1:18),3,c(1,3))
response_dimension(c(1:18),3,c(2,3))
```

---

|                    |                                                                       |
|--------------------|-----------------------------------------------------------------------|
| response_frequency | <i>Response frequency table for ordinal or Likert-scale variables</i> |
|--------------------|-----------------------------------------------------------------------|

---

## Description

Tabulates how often each response category was chosen for one or more ordinal variables (e.g. Likert scale items). For each variable the function returns the count, proportion, or percentage of respondents who selected each response option, along with the number of missing or out-of-range responses. The function is a guard against accidental use on continuous variables: if the number of unique values exceeds max the function returns NULL.

## Usage

```
response_frequency(
  df,
  max = 10,
  uniqueitems = NULL,
  type = "percent",
  file = NULL
)
```

## Arguments

|             |                                                                                                                                                                                                                                        |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| df          | A data frame whose columns are the ordinal variables to tabulate.                                                                                                                                                                      |
| max         | Integer. Maximum number of unique response options allowed before the function returns NULL. Use this to prevent accidentally tabulating continuous variables. Default is 10.                                                          |
| uniqueitems | Vector of all valid response values (e.g. 1:5 for a 5-point Likert scale). When NULL (default) the unique values observed in df are used.                                                                                              |
| type        | Character string controlling the metric returned. One of "frequency" (raw counts), "proportion" (counts divided by valid responses), "percent" (proportion multiplied by 100, default), or "all" (all three metrics stacked row-wise). |
| file        | Character string naming the output Excel file (without extension). When NULL (default) no file is written.                                                                                                                             |

## Value

A data frame with one row per variable (three rows per variable when type = "all") containing the following columns:

**type** "Frequency", "Proportion", or "Percent".

**variable** Name of the column from df.

**(response columns)** One column per value in uniqueitems, named by the response value, containing the frequency, proportion, or percent of respondents who chose that category.

**miss** Observations with values outside uniqueitems. For proportions this is the missing rate; for percent it is the missing percentage.

**responses** Total number of valid (non-missing) responses.

Returns NULL if the number of unique values exceeds max.

### Examples

```
response_frequency(mtcars[,c("gear", "carb")], uniqueitems=1:8, type="frequency")
response_frequency(mtcars[,c("gear")], uniqueitems=1:8, type="proportion")
response_frequency(mtcars[,c("gear", "carb")], uniqueitems=1:8, type="percent")
response_frequency(mtcars[,c("gear", "carb")], uniqueitems=1:8, type="all")
response_frequency(mtcars[,c("gear", "carb")], uniqueitems=1:8, type="all",
                  file="descriptives")
```

---

result\_confusion\_performance

*Plot performance of confusion matrix for different cut off points*

---

### Description

This function generates a plot to visualize the performance of a confusion matrix at various cut-off points. It evaluates the proportion of correct classifications and identifies the optimal cut-off point.

### Usage

```
result_confusion_performance(
  observed,
  predicted,
  step = 0.1,
  base_size = 10,
  title = ""
)
```

### Arguments

|           |                                                                                                          |
|-----------|----------------------------------------------------------------------------------------------------------|
| observed  | Vector of observed outcomes. These are the true class labels.                                            |
| predicted | Vector of predicted outcome probabilities. These are the predicted probabilities for the positive class. |
| step      | Numeric value representing the stepping for tested cut values. Defaults to 0.1.                          |
| base_size | Integer value representing the base font size for the plot. Defaults to 10.                              |
| title     | String representing the title of the plot. Defaults to an empty string.                                  |



## Details

This function evaluates the performance of a confusion matrix at different cut-off points. It iterates through a range of cut-off points, calculates the confusion matrix, and evaluates the proportion of correct classifications for each cut-off.

The function generates a plot that includes: -The proportion of correct classifications for different cut-off points. -Vertical lines indicating the optimal cut-off point. -A legend representing different performance metrics. -A caption showing the number of observations and the optimal cut-off point.

The function returns a list containing the plot, the data frame with cut-off performance, the optimal cut-off point, and the confusion matrix at the optimal cut-off.

## Examples

```
# Example with numeric class labels
df<-data.frame(matrix(.999,ncol=2,nrow=2))
correlation_matrix<-as.matrix(df)
diag(correlation_matrix)<-1
df<-generate_correlation_matrix(correlation_matrix,nrows=1000)
df$X1<-ifelse(abs(df$X1) < 1,0,1)
df$X2<-abs(df$X2)
df$X2<-(df$X2-min(df$X2))/(max(df$X2)-min(df$X2))
result_confusion_performance(observed=round(abs(df$X1),0),
                             predicted=abs(df$X2),
                             step=0.01)
result_confusion_performance(observed=c(1,2,3,1,2,3),
                             predicted=abs(rnorm(6,0,sd=0.1)))
```

---

|                 |                                              |
|-----------------|----------------------------------------------|
| round_dataframe | <i>Round numeric columns in a data frame</i> |
|-----------------|----------------------------------------------|

---

## Description

Applies a rounding or transformation function to every numeric column in a data frame, leaving non-numeric columns (factor, character, etc.) unchanged.

## Usage

```
round_dataframe(df, digits = 0, type = "round")
```

## Arguments

|        |                                                                                                                                                                                                              |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| df     | A data frame containing a mix of numeric and non-numeric columns.                                                                                                                                            |
| digits | Integer number of decimal places. Only used with type = "round" and type = "tenth". Default is 0.                                                                                                            |
| type   | Character string specifying the transformation to apply to numeric columns:<br>"round" Round to digits decimal places using round() (default).<br>"ceiling" Round up to the nearest integer using ceiling(). |

"floor" Round down to the nearest integer using floor().

"tenth" Divide each value by 10 then round to digits decimal places — useful for rescaling values that were multiplied by 10 (e.g. converting tenths back to units).

**Value**

A data frame with the same structure as df where all numeric columns have been rounded or transformed according to type.

**Examples**

```
round_dataframe(df=change_data_type(df=mtcars, type="factor"), digits=0)
round_dataframe(df=change_data_type(df=mtcars, type="character"), digits=0)
round_dataframe(df=mtcars, digits=0)
round_dataframe(df=mtcars, digits=0, type="ceiling")
round_dataframe(df=mtcars, digits=0, type="floor")
round_dataframe(df=mtcars*100, digits=2, type="tenth")
```

---

|            |                                                                     |
|------------|---------------------------------------------------------------------|
| score_tirt | <i>Score Multiple Thurstonian IRT Response Patterns (MAP / EBM)</i> |
|------------|---------------------------------------------------------------------|

---

**Description**

Scores many respondents at once using Thurstonian IRT parameters and returns MAP (empirical Bayes modal) latent trait estimates for each row in patterns.

In simple terms: this function applies score\_tirt\_pattern to every respondent, after first checking and aligning item columns so they match lambda row order.

**Usage**

```
score_tirt(patterns, lambda, theta_diag, tau, Psi, nu = NULL)
```

**Arguments**

|            |                                                                                                                           |
|------------|---------------------------------------------------------------------------------------------------------------------------|
| patterns   | A matrix or data.frame of response patterns (rows = respondents, columns = pair/items).                                   |
| lambda     | Loading matrix (rows = pair/items, columns = latent traits).                                                              |
| theta_diag | Numeric vector of residual variances aligned to rows of lambda.                                                           |
| tau        | Numeric vector of thresholds aligned to rows of lambda.                                                                   |
| Psi        | Latent covariance matrix (traits x traits).                                                                               |
| nu         | Optional numeric vector of indicator intercepts aligned to rows of lambda. If NULL, zeros are used in score_tirt_pattern. |

## Details

Name alignment is the key safeguard: if both `rownames(lambda)` and `colnames(patterns)` are present, `patterns` is reordered to match `lambda` row order before scoring.

If required `lambda` names are missing from `patterns`, the function stops with an informative error. If names are unavailable, positional alignment is assumed and a warning is issued.

## Value

A numeric matrix of latent scores: rows correspond to respondents in `patterns`, columns correspond to traits in `colnames(lambda)`.

## Examples

```
library(thurstonianIRT)
data("triplets")
# define the blocks of items
blocks <-
  set_block(c("i1", "i2", "i3"), traits = c("t1", "t2", "t3"),
            signs = c(1, 1, 1)) +
  set_block(c("i4", "i5", "i6"), traits = c("t1", "t2", "t3"),
            signs = c(-1, 1, 1)) +
  set_block(c("i7", "i8", "i9"), traits = c("t1", "t2", "t3"),
            signs = c(1, 1, -1)) +
  set_block(c("i10", "i11", "i12"), traits = c("t1", "t2", "t3"),
            signs = c(1, -1, 1))
# generate the data to be understood by 'thurstonianIRT'
triplets_long <- make_TIRT_data(
  data = triplets, blocks = blocks, direction = "larger",
  format = "pairwise", family = "bernoulli", range = c(0, 1)
)
# fit the data using lavaan
fit <- fit_TIRT_lavaan(triplets_long)
pars <- extract_tirt_params(fit)
patterns <- as.matrix(triplets)
score_tirt(patterns, lambda = pars$lambda, theta_diag = pars$theta_diag,
            tau = pars$tau, Psi = pars$Psi, nu = NULL)
# Check same scores from thurstonianIRT package
triplets_long <- make_TIRT_data(data = triplets,
                                blocks = blocks,
                                direction = "larger",
                                format = "pairwise",
                                family = "bernoulli",
                                range = c(0, 1))
scores_thurstonianIRT <- predict(fit)
scores <- score_tirt(patterns, lambda = pars$lambda, theta_diag = pars$theta_diag,
                    tau = pars$tau, Psi = pars$Psi, nu = NULL)
head(reshape2::recast(scores_thurstonianIRT, formula = id ~ trait, id.var = 1:2))
head(scores)
```

---

|                    |                                                                    |
|--------------------|--------------------------------------------------------------------|
| score_tirt_pattern | <i>Score a Single Thurstonian IRT Response Pattern (MAP / EBM)</i> |
|--------------------|--------------------------------------------------------------------|

---

### Description

Computes the maximum a posteriori (MAP), also called empirical Bayes modal (EBM), estimate of latent traits for one binary/ordinal response pattern under a Thurstonian IRT parameterization.

Missing responses are allowed and are ignored in the likelihood.

### Usage

```
score_tirt_pattern(
  pattern,
  lambda,
  theta_diag,
  tau,
  Psi,
  nu = NULL,
  init = NULL,
  control = list()
)
```

### Arguments

|            |                                                                                                                                                                                                           |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| pattern    | Numeric vector of observed responses for one person, typically coded 0/1, with optional NA values for missing responses. Its order must match the row order of lambda (or be pre-aligned before calling). |
| lambda     | Matrix of factor loadings with rows as observed indicators and columns as latent traits.                                                                                                                  |
| theta_diag | Numeric vector of residual variances (diagonal of theta), aligned to rows of lambda.                                                                                                                      |
| tau        | Numeric vector of thresholds, aligned to rows of lambda.                                                                                                                                                  |
| Psi        | Latent covariance matrix (traits x traits), positive definite.                                                                                                                                            |
| nu         | Optional numeric vector of indicator intercepts aligned to rows of lambda. If NULL, a zero vector is used.                                                                                                |
| init       | Optional numeric vector of starting values for optimization. If NULL, starts at zeros.                                                                                                                    |
| control    | Named list of control arguments passed to optim, merged with defaults reltol = 1e-10 and maxit = 500.                                                                                                     |

### Details

The function maximizes the posterior: likelihood from a probit measurement model plus a multivariate normal prior on traits with covariance Psi. Optimization uses BFGS via optim with an analytic gradient.

**Value**

Named numeric vector of latent trait MAP estimates, with names taken from `colnames(lambda)`.

**Examples**

```
library(thurstonianIRT)
data("triplets")
# define the blocks of items
blocks <-
  set_block(c("i1", "i2", "i3"), traits = c("t1", "t2", "t3"),
            signs = c(1, 1, 1)) +
  set_block(c("i4", "i5", "i6"), traits = c("t1", "t2", "t3"),
            signs = c(-1, 1, 1)) +
  set_block(c("i7", "i8", "i9"), traits = c("t1", "t2", "t3"),
            signs = c(1, 1, -1)) +
  set_block(c("i10", "i11", "i12"), traits = c("t1", "t2", "t3"),
            signs = c(1, -1, 1))
# generate the data to be understood by 'thurstonianIRT'
triplets_long <- make_TIRT_data(
  data = triplets, blocks = blocks, direction = "larger",
  format = "pairwise", family = "bernoulli", range = c(0, 1)
)
# fit the data using lavaan
fit <- fit_TIRT_lavaan(triplets_long)
pars <- extract_tirt_params(fit)
pattern <- as.numeric(triplets[1,])
score_tirt_pattern(pattern, lambda=pars$lambda, theta_diag=pars$theta_diag,
                   tau=pars$tau, Psi=pars$Psi, nu=NULL, init=NULL,
                   control=list())
```

shROUT

*ShROUT-Fleiss reliability coefficients***Description**

Computes five reliability coefficients from the ShROUT and Fleiss (1979) framework using variance components extracted from a person  $\times$  item  $\times$  time mixed model (see [extract\\_components](#)). The coefficients cover between-person and within-person reliability under both fixed and random time designs, and are appropriate for longitudinal or repeated-measures measurement studies.

**Usage**

```
shROUT(spersion, spersionitem, stime, spersontime, sperson, m, k)
```

**Arguments**

|                           |                                                             |
|---------------------------|-------------------------------------------------------------|
| <code>spersion</code>     | Variance component of the person main effect.               |
| <code>spersionitem</code> | Variance component of the person $\times$ item interaction. |

|             |                                                             |
|-------------|-------------------------------------------------------------|
| stime       | Variance component of the time main effect.                 |
| spersontime | Variance component of the person $\times$ time interaction. |
| serror      | Variance component of residual error.                       |
| m           | Number of items (reports) averaged over.                    |
| k           | Number of time points averaged over.                        |

### Value

A data frame with one row per reliability coefficient containing columns measure, result, and description:

**r1f** Between-person reliability of a single measure at fixed time points.

**r1r** Between-person reliability of a single measure at random time points (different people, different days).

**rkf** Between-person reliability of scores averaged over *m* items and *k* fixed time points.

**rkr** Between-person reliability of scores averaged over *k* random time points.

**rc** Within-person reliability of change across time points.

### References

Shrout, P. E., & Fleiss, J. L. (1979). Intraclass correlations: Uses in assessing rater reliability. *Psychological Bulletin*, 86(2), 420–428.

### Examples

```
design<-expand.grid(time=1:3,item=1:2,person=1:10)
design<-change_data_type(design,type="factor")
design$response<-rnorm(30,0,0.1)
model<-mixlm::lm(response~r(time)*r(person)+r(item)*r(person),data=design)
result<-extract_components(model)
vc<-result$components
shrout(spersion=vc[2,3],spersonitem=vc[5,3],stime=vc[1,3],
       spersontime=vc[4,3],serror=vc[6,3],3,3)
```

---

simulate\_cfa\_fit

*Simulate CFA model fit across sample sizes*

---

### Description

Runs a confirmatory factor analysis (CFA) repeatedly across a range of sample sizes and returns fit indices for each. Useful for power analysis and sample size planning in SEM.

Two workflows are supported:

- **Coefficient-based:** supply `model_sim` with fixed loadings — data are generated from those population parameters at each sample size.

- **Correlation-based:** supply `df` — data are bootstrapped from the observed correlation structure of `df` at each sample size.

Iterations run in parallel via `future.apply`. Results are written to a PDF (scatter plots of fit indices vs. sample size) and an Excel file.

### Usage

```
simulate_cfa_fit(
  model_sim = NULL,
  model = NULL,
  df = NULL,
  minnobs = 50,
  maxnobs = 1000,
  stepping = 10,
  file = NULL,
  w = 10,
  h = 10
)
```

### Arguments

|                        |                                                                                                                                                                                   |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>model_sim</code> | A lavaan model string with fixed coefficients (e.g. "F =~ 1*x1 + 0.8*x2"). Used to generate population data. Mutually exclusive with <code>df</code> ; supply one or the other.   |
| <code>model</code>     | A lavaan model string with free coefficients specifying the CFA structure to be estimated at each sample size (e.g. "F =~ x1 + x2").                                              |
| <code>df</code>        | A data frame of observed data. When supplied, each iteration simulates data by sampling from the observed correlation structure. Mutually exclusive with <code>model_sim</code> . |
| <code>minnobs</code>   | Integer. Smallest sample size to evaluate. Default 50.                                                                                                                            |
| <code>maxnobs</code>   | Integer. Largest sample size to evaluate. Default 1000.                                                                                                                           |
| <code>stepping</code>  | Integer. Increment between sample sizes. Default 10.                                                                                                                              |
| <code>file</code>      | Character. Base name (without extension) for the output PDF and Excel files. If NULL no files are written.                                                                        |
| <code>w</code>         | Numeric. Width of the output PDF in inches. Default 10.                                                                                                                           |
| <code>h</code>         | Numeric. Height of the output PDF in inches. Default 10.                                                                                                                          |

### Value

A list of two elements:

- `[[1]]` — a data frame with one row per sample size and one column per lavaan fit index (CFI, RMSEA, SRMR, etc.).
- `[[2]]` — a named list of ggplot scatter plots, one per fit index, showing how the index changes with sample size.

**Examples**

```

model_sim <- 'LATENT =~ 1*X1 + 0.5*X2 + 1.5*X3 + 1.5*X4 + X5'
model      <- 'LATENT =~ X1 + X2 + X3 + X4 + X5'

# Coefficient-based: generate data from known population parameters
result<-simulate_cfa_fit(model_sim=model_sim, model=model,
                        minnobs=50, maxnobs=1000, stepping=100, file="report")
plot_multiplot(plotlist=result[[2]], cols=4)
# Correlation-based: resample from observed data
df <- lavaan::simulateData(model=model_sim, model.type="cfa",
                          return.type="data.frame", sample.nobs=1000)
result<-simulate_cfa_fit(model=model, df=df,
                        minnobs=50, maxnobs=1000, stepping=100, file="report")
plot_multiplot(plotlist=result[[2]], cols=4)

```

---

```
simulate_correlation_from_sample
```

*Generate a dataframe that produces the same correlation matrix as the input dataframe*

---

**Description**

Generate a dataframe that produces the same correlation matrix as the input dataframe

**Usage**

```
simulate_correlation_from_sample(cordata, nrows = 10)
```

**Arguments**

|                      |                            |
|----------------------|----------------------------|
| <code>cordata</code> | dataframe                  |
| <code>nrows</code>   | number of rows to generate |

**Examples**

```

correlation_matrix<-generate_correlation_matrix()
stats::cor(correlation_matrix)
simulate_correlation_from_sample(correlation_matrix,nrows=1000)
stats::cor(simulate_correlation_from_sample(correlation_matrix,nrows=1000))

```



---

|           |                                  |
|-----------|----------------------------------|
| split_str | <i>Split string to dataframe</i> |
|-----------|----------------------------------|

---

**Description**

Split string to dataframe

**Usage**

```
split_str(vector, split = "/", include_original = FALSE)
```

**Arguments**

|                  |                                                        |
|------------------|--------------------------------------------------------|
| vector           | String                                                 |
| split            | Separation character                                   |
| include_original | if TRUE it will return the input on a separate collumn |

**Examples**

```
string<-paste0(1:10,"/",  
              generate_string(nchar=2,vector_length=10),"/",  
              generate_string(nchar=2,vector_length=10),"/",  
              generate_string(nchar=2,vector_length=10))  
split_str(string,split="/")
```

---

|              |                                  |
|--------------|----------------------------------|
| split_str_df | <i>Split string in dataframe</i> |
|--------------|----------------------------------|

---

**Description**

Split string in dataframe

**Usage**

```
split_str_df(df, split = "/", type = "row", index, ...)
```

**Arguments**

|       |                                                                                                                                                                                                                     |
|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| df    | dataframe                                                                                                                                                                                                           |
| split | Separation character                                                                                                                                                                                                |
| type  | "row" "collumn" if "row" it will split the string of row names and it will display it on seperate collumns if "collumn" it will split the string of a spesified collumn and it will display it on separate collumns |
| index | Numeric index of collumn to split. This is only relevant if type="collumn"                                                                                                                                          |
| ...   | arguments passed to split_str                                                                                                                                                                                       |

Examples

```
df<-generate_correlation_matrix()
string<-paste0(1:nrow(df),"/",
               generate_string(nchar=2,vector_length=nrow(df)),"/",
               generate_string(nchar=2,vector_length=nrow(df)),"/",
               generate_string(nchar=2,vector_length=nrow(df)))
row.names(df)<-string
split_str_df(df,split="/",type="row")
df[,1]<-string
split_str_df(df,split="/",type="column",index=1)
```

---

|                |                                 |
|----------------|---------------------------------|
| stat_word_char | <i>Text similarity measures</i> |
|----------------|---------------------------------|

---

Description

Text similarity measures

Usage

```
stat_word_char(text)
```

Arguments

text                      character vector

Examples

```
text<-"There are many variations of passages of Lorem Ipsum available,
but the majority have suffered alteration in some form, by injected humour,
or randomised words which don't look even slightly believable."
stat_word_char(text)
```

---

|            |                                 |
|------------|---------------------------------|
| string_aes | <i>Adjust string aesthetics</i> |
|------------|---------------------------------|

---

Description

Treats spesific characters such as ".", as separating characters and separates strings with space. Trims leading and trailing spaces and capitalizes the first letter of the string and lowers the rest.

Usage

```
string_aes(
  vector,
  characterlist = c(".", "_", "-", " ", "$", "<p>", "</p>", "<br>", "<br/>", "<B>",
    "</B>", "<BR/>", "|", "/", "&nbsp;"),
  proper = TRUE
)
```

**Arguments**

|               |                                                               |
|---------------|---------------------------------------------------------------|
| vector        | Vector                                                        |
| characterlist | List the list of characters to treat as separating characters |
| proper        | Logical TRUE capitalizes the first letter in sentence format  |

**Examples**

```
vector<-c("TES.T", "TES<p>T", "TES&nbsp;T")
string_aes(vector=vector)
string_aes(vector=vector,proper=FALSE)
string_aes(vector=vector,proper=TRUE)
```

---

|           |                                                        |
|-----------|--------------------------------------------------------|
| str_count | <i>Count the number of pattern matches in a string</i> |
|-----------|--------------------------------------------------------|

---

**Description**

Returns the number of times pattern appears in each element of string. Supports both regular expressions and literal string matching via `fixed()`.

Returns the number of times pattern appears in each element of string. Supports both regular expressions and literal string matching via `fixed()`.

**Usage**

```
str_count(string, pattern)
```

```
str_count(string, pattern)
```

**Arguments**

|         |                                                                                   |
|---------|-----------------------------------------------------------------------------------|
| string  | A character vector.                                                               |
| pattern | A regular expression string or a literal string wrapped in <code>fixed()</code> . |

**Value**

An integer vector the same length as `string`.

An integer vector the same length as `string`.

**Examples**

```
# Count vowels
str_count(c("banana", "apple", "cherry"), "[aeiou]")

# Count literal semicolons (useful for delimited data)
str_count(c("a;b;c", "x;y", "z"), fixed(";"))

# Count digits
```

```

str_count(c("abc123", "99bottles", "none"), "[0-9]")
# Count vowels
str_count(c("banana", "apple", "cherry"), "[aeiou]")

# Count literal semicolons (useful for delimited data)
str_count(c("a;b;c", "x;y", "z"), fixed(";"))

# Count digits
str_count(c("abc123", "99bottles", "none"), "[0-9]")

```

---

str\_pad

*Pad a string to a minimum width*


---

## Description

Pads string with pad characters on the left, right, or both sides until it reaches at least width characters. Strings already at or exceeding width are returned unchanged.

Pads string with pad characters on the left, right, or both sides until it reaches at least width characters. Strings already at or exceeding width are returned unchanged.

## Usage

```

str_pad(string, width, side = "right", pad = " ")

str_pad(string, width, side = "right", pad = " ")

```

## Arguments

|        |                                                     |
|--------|-----------------------------------------------------|
| string | A character vector.                                 |
| width  | Integer. Minimum total width of the output string.  |
| side   | One of "right" (default), "left", or "both".        |
| pad    | A single character to use for padding. Default " ". |

## Value

A character vector the same length as string.

A character vector the same length as string.

## Examples

```

# Zero-pad single digit numbers on the left
str_pad(c("1", "10", "100"), width=3, side="left", pad="0")

# Right-pad to align labels
str_pad(c("Name", "Age", "Score"), width=10)

# Pad on both sides (centers the string)

```

```

str_pad("hello", width=11, side="both")
# Zero-pad single digit numbers on the left
str_pad(c("1", "10", "100"), width=3, side="left", pad="0")

# Right-pad to align labels
str_pad(c("Name", "Age", "Score"), width=10)

# Pad on both sides (centers the string)
str_pad("hello", width=11, side="both")

```

---

|             |                                                    |
|-------------|----------------------------------------------------|
| str_replace | <i>Replace the first pattern match in a string</i> |
|-------------|----------------------------------------------------|

---

### Description

Replaces only the first occurrence of pattern in each element of string. For replacing all occurrences use `str_replace_all`.

Replaces only the first occurrence of pattern in each element of string. For replacing all occurrences use `str_replace_all`.

### Usage

```
str_replace(string, pattern, replacement)
```

```
str_replace(string, pattern, replacement)
```

### Arguments

|             |                                                                                   |
|-------------|-----------------------------------------------------------------------------------|
| string      | A character vector.                                                               |
| pattern     | A regular expression string or a literal string wrapped in <code>fixed()</code> . |
| replacement | A character string to replace the first match with.                               |

### Value

A character vector the same length as string.

A character vector the same length as string.

### Examples

```

# Only the first "o" is replaced
str_replace("hello world", "o", "0")

# Remove leading zero (first match only)
str_replace("007 bond", "^0+", "")

# Fixed match: replace first literal dot
str_replace("a.b.c", fixed("."), "-")
# Only the first "o" is replaced

```

```
str_replace("hello world", "o", "0")

# Remove leading zero (first match only)
str_replace("007 bond", "^0+", "")

# Fixed match: replace first literal dot
str_replace("a.b.c", fixed("."), "-")
```

---

|                 |                                                |
|-----------------|------------------------------------------------|
| str_replace_all | <i>Replace all pattern matches in a string</i> |
|-----------------|------------------------------------------------|

---

## Description

Replaces every occurrence of pattern in string with replacement. Supports both regular expressions and literal string matching via `fixed()`.

Replaces every occurrence of pattern in string with replacement. Supports both regular expressions and literal string matching via `fixed()`.

## Usage

```
str_replace_all(string, pattern, replacement)
```

```
str_replace_all(string, pattern, replacement)
```

## Arguments

|             |                                                                                                                                                                                                   |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| string      | A character vector.                                                                                                                                                                               |
| pattern     | A regular expression string, or a literal string wrapped in <code>fixed()</code> , or a named character vector where names are regex patterns and values are replacements (applied sequentially). |
| replacement | A character string to replace each match with. Use <code>""</code> to delete matches.                                                                                                             |

## Value

A character vector the same length as string.

A character vector the same length as string.

## Examples

```
# Regex replacement
str_replace_all("hello world", "o", "0")

# Fixed (literal) replacement
str_replace_all("a.b.c", fixed("."), "-")

# Remove all spaces
str_replace_all("remove all spaces", fixed(" "), "")
```

```
# Named vector: multiple replacements applied in order
str_replace_all("aabbcc", c("a"="X", "b"="Y"))
# Regex replacement
str_replace_all("hello world", "o", "0")

# Fixed (literal) replacement
str_replace_all("a.b.c", fixed("."), "-")

# Remove all spaces
str_replace_all("remove all spaces", fixed(" "), "")

# Named vector: multiple replacements applied in order
str_replace_all("aabbcc", c("a"="X", "b"="Y"))
```

---

|                 |                                                          |
|-----------------|----------------------------------------------------------|
| str_split_fixed | <i>Split strings into a fixed-width matrix of pieces</i> |
|-----------------|----------------------------------------------------------|

---

## Description

Splits each element of string by pattern and returns a character matrix with exactly n columns. If a string produces fewer than n pieces the remaining columns are filled with "".

Splits each element of string by pattern and returns a character matrix with exactly n columns. If a string produces fewer than n pieces the remaining columns are filled with "".

## Usage

```
str_split_fixed(string, pattern, n)
```

```
str_split_fixed(string, pattern, n)
```

## Arguments

|         |                                                                     |
|---------|---------------------------------------------------------------------|
| string  | A character vector.                                                 |
| pattern | A regular expression string or a literal string wrapped in fixed(). |
| n       | Integer. Number of columns in the output matrix.                    |

## Value

A character matrix with length(string) rows and n columns.

A character matrix with length(string) rows and n columns.

## Examples

```
# Split "trait.method" labels into two columns
str_split_fixed(c("speed.run", "height.jump", "weight.lift"), fixed("."), 2)

# Split on a regex pattern
str_split_fixed(c("a1b", "c2d", "e3f"), "[0-9]", 2)
```

```
# Fewer pieces than n: remainder filled with ""
str_split_fixed(c("a.b.c", "x.y"), fixed("."), 3)
# Split "trait.method" labels into two columns
str_split_fixed(c("speed.run", "height.jump", "weight.lift"), fixed("."), 2)

# Split on a regex pattern
str_split_fixed(c("a1b", "c2d", "e3f"), "[0-9]", 2)

# Fewer pieces than n: remainder filled with ""
str_split_fixed(c("a.b.c", "x.y"), fixed("."), 3)
```

---

str\_squish

Remove leading, trailing, and internal extra whitespace

---

## Description

Strips leading and trailing whitespace and collapses any internal sequences of whitespace (spaces, tabs, newlines) down to a single space.

Strips leading and trailing whitespace and collapses any internal sequences of whitespace (spaces, tabs, newlines) down to a single space.

## Usage

```
str_squish(string)
```

```
str_squish(string)
```

## Arguments

string            A character vector.

## Value

A character vector the same length as string.

A character vector the same length as string.

## Examples

```
# Remove extra internal spaces
str_squish(" hello world ")

# Clean up messy column names or labels
str_squish(c(" first name ", "last name", " age"))

# Handles tabs and newlines too
str_squish("line1\n\nline2\t\tword")
# Remove extra internal spaces
str_squish(" hello world ")
```



```
# Clean up messy column names or labels
str_squish(c(" first name ", "last name", " age"))

# Handles tabs and newlines too
str_squish("line1\n\nline2\t\tword")
```

---

str\_wrap

---

*Wrap long strings to a specified line width*


---

## Description

Breaks a character string into multiple lines so that no line exceeds width characters. Words are kept intact; lines are joined with "\n".

Breaks a character string into multiple lines so that no line exceeds width characters. Words are kept intact; lines are joined with "\n".

## Usage

```
str_wrap(string, width = 80)
```

```
str_wrap(string, width = 80)
```

## Arguments

|        |                                                    |
|--------|----------------------------------------------------|
| string | A character vector.                                |
| width  | Maximum number of characters per line. Default 80. |

## Value

A character vector the same length as string, with embedded newlines inserted at word boundaries.

A character vector the same length as string, with embedded newlines inserted at word boundaries.

## Examples

```
# Wrap at 30 characters
cat(str_wrap("The quick brown fox jumped over the lazy dog", width=30))

# Wrap a vector of strings
labels <- c("Short label", "A much longer label that needs wrapping")
str_wrap(labels, width=20)
# Wrap at 30 characters
cat(str_wrap("The quick brown fox jumped over the lazy dog", width=30))

# Wrap a vector of strings
labels <- c("Short label", "A much longer label that needs wrapping")
str_wrap(labels, width=20)
```

---

|         |                                               |
|---------|-----------------------------------------------|
| sub_str | <i>Return n characters from left or right</i> |
|---------|-----------------------------------------------|

---

**Description**

Return n characters from left or right

**Usage**

```
sub_str(x, n = 2, type)
```

**Arguments**

|      |                                |
|------|--------------------------------|
| x    | Character                      |
| n    | Number of characters to return |
| type | "right" "left"                 |

**Examples**

```
sub_str("12345",n=2,type="right")  
sub_str("12345",n=2,type="left")
```

---

|      |                                 |
|------|---------------------------------|
| swap | <i>Reverse a numeric vector</i> |
|------|---------------------------------|

---

**Description**

Reverse a numeric vector

**Usage**

```
swap(vector)
```

**Arguments**

|        |         |
|--------|---------|
| vector | numeric |
|--------|---------|

**Examples**

```
swap(c(1:10,1,2,3))
```

---

|                  |                         |
|------------------|-------------------------|
| symmetric_matrix | <i>Symmetric Matrix</i> |
|------------------|-------------------------|

---

**Description**

Symmetric Matrix

**Usage**

```
symmetric_matrix(matrix, duplicate = "lower", diagonal = NULL)
```

**Arguments**

|           |                                                                     |
|-----------|---------------------------------------------------------------------|
| matrix    | matrix                                                              |
| duplicate | "upper" duplicates upper triangle "lower" duplicates lower triangle |
| diagonal  | diagonal values                                                     |

**Examples**

```
m_lower<-matrix_triangle(matrix(1:9,nrow=3,ncol=3),type="lower",diagonal=NA)
m_upper<-matrix_triangle(matrix(11:19,nrow=3,ncol=3),type="upper",diagonal=NA)
symmetric_matrix(matrix=m_lower,duplicate="lower",diagonal=NA)
symmetric_matrix(matrix=m_upper,duplicate="upper",diagonal=NA)
```

---

|         |                               |
|---------|-------------------------------|
| tag_pos | <i>Part of speech tagging</i> |
|---------|-------------------------------|

---

**Description**

Part of speech tagging

**Usage**

```
tag_pos(text)
```

**Arguments**

|      |                  |
|------|------------------|
| text | character vector |
|------|------------------|

**Examples**

```

text1<-"word_one word_two word_three"
text2<-"word_three word_four word_six"
text3<-"All the Lorem Ipsum generators on the Internet tend to repeat predefined
chunks as necessary, making this the first true generator on the Internet."
text4<-"It uses a dictionary of over 200 Latin words, combined with a handful of
model sentence structures, to generate Lorem Ipsum which looks reasonable."
text5<-"The generated Lorem Ipsum is therefore always free from repetition,
injected humour, or non-characteristic words etc."
text<-c(text1,text2,text3,text4,text5)
tag_pos(text)

```

---

|                 |                                 |
|-----------------|---------------------------------|
| text_similarity | <i>Text similarity measures</i> |
|-----------------|---------------------------------|

---

**Description**

Text similarity measures

**Usage**

```
text_similarity(text1, text2)
```

**Arguments**

|       |                  |
|-------|------------------|
| text1 | character vector |
| text2 | character vector |

**Examples**

```

text1<-"word_one word_two word_three"
text2<-"word_three word_four word_six"
text3<-"All the Lorem Ipsum generators on the Internet tend to repeat predefined
chunks as necessary, making this the first true generator on the Internet."
text4<-"It uses a dictionary of over 200 Latin words, combined with a handful of
model sentence structures, to generate Lorem Ipsum which looks reasonable."
text5<-"The generated Lorem Ipsum is therefore always free from repetition,
injected humour, or non-characteristic words etc."
text<-c(text1,text2,text3,text4,text5)
text<-unlist(strsplit(text,split=" "))
text1<-unlist(strsplit(text1,split=" "))
text2<-unlist(strsplit(text2,split=" "))
text3<-unlist(strsplit(text3,split=" "))
text4<-unlist(strsplit(text4,split=" "))
text5<-unlist(strsplit(text5,split=" "))
text_similarity(text1,text1)
text_similarity(text1,text2)
text_similarity(text1,text3)
text_similarity(text1,text4)

```

---

|         |                                     |
|---------|-------------------------------------|
| trim_df | <i>Trim whitespace in dataframe</i> |
|---------|-------------------------------------|

---

**Description**

Trim whitespace in dataframe

**Usage**

```
trim_df(df)
```

**Arguments**

|    |           |
|----|-----------|
| df | dataframe |
|----|-----------|

**Examples**

```
string<-data.frame(str1=rep(paste0(sample(c(LETTERS,rep(" ",10))),collapse=""),10),
                    str2=rep(paste0(sample(c(LETTERS,rep(" ",10))),collapse=""),10),
                    num1=rnorm(10),
                    stringsAsFactors=FALSE)

trim_df(string)
```

---

|              |                                                             |
|--------------|-------------------------------------------------------------|
| ts_smoothing | <i>Time series smoothing with multiple bandwidth levels</i> |
|--------------|-------------------------------------------------------------|

---

**Description**

Plots a time series and overlays a family of smoothed curves using a chosen smoothing method. For bandwidth-based methods ("kernel", "lowess", "splines", "default"), a sequence of bandwidth or span values from start to stop is swept and each curve is drawn in a different rainbow colour, making it easy to visually select an appropriate smoothing level. "polynomial" and "linear" ignore the bandwidth sequence and fit regression-based trend lines instead. The function uses base R graphics and produces a plot as a side effect.

**Usage**

```
ts_smoothing(
  df,
  start = 0.01,
  stop = 2,
  step = 0.001,
  title = "",
  type = "kernel"
)
```

**Arguments**

|                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>df</code>    | A ts object containing the time series to smooth.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <code>start</code> | Numeric. Starting value of the bandwidth or span sequence. Default is 0.01.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <code>stop</code>  | Numeric. Ending value of the bandwidth or span sequence. Default is 2.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <code>step</code>  | Numeric. Increment between bandwidth values. Smaller values produce more curves. Default is 0.001.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <code>title</code> | Character string appended to the plot title. Default is "".                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <code>type</code>  | Character string specifying the smoothing method. One of:<br>"kernel" Gaussian kernel smoother via <code>ksmooth()</code> — bandwidth controls the smoothing window (default).<br>"lowess" Locally weighted regression via <code>lowess()</code> — <code>f</code> controls the span proportion.<br>"friedman" Friedman's super-smoother via <code>supsmu()</code> — span must be in (0, 1).<br>"splines" Smoothing splines via <code>smooth.spline()</code> — <code>spar</code> controls the penalty.<br>"default" Running mean filter via <code>filter()</code> — bandwidth rounded to an integer window width.<br>"polynomial" Fits a centred cubic polynomial trend with and without seasonal (cos/sin) terms via <code>lm()</code> .<br>"linear" Fits a simple linear trend via <code>lm()</code> and draws the regression line. |

**Value**

Invisibly returns NULL. The function is called for its side effect of producing a base R plot.

**Examples**

```
ts_data<-ts(UKDriverDeaths,start=1969,end=1984,frequency=12)
par(mfrow=c(2,2))
ts_smoothing(ts_data,start=.01,stop=2,step=.01,
             title="Driver Deaths in UK",type="default")
ts_smoothing(ts_data,start=.01,stop=2,step=.01,
             title="Driver Deaths in UK",type="polynomial")
ts_smoothing(ts_data,start=.01,stop=2,step=.01,
             title="Driver Deaths in UK",type="linear")
ts_smoothing(ts_data,start=.01,stop=2,step=.01,
             title="Driver Deaths in UK",type="kernel")
ts_smoothing(ts_data,start=.01,stop=2,step=.01,
             title="Driver Deaths in UK",type="lowess")
ts_smoothing(ts_data,start=.01,stop=2,step=.01,
             title="Driver Deaths in UK",type="friedman")
ts_smoothing(ts_data,start=.01,stop=2,step=.01,
             title="Driver Deaths in UK",type="splines")
```

---

|         |                    |
|---------|--------------------|
| wrapper | <i>Wrap string</i> |
|---------|--------------------|

---

**Description**

Wrap string

**Usage**

```
wrapper(x, ...)
```

**Arguments**

|     |                             |
|-----|-----------------------------|
| x   | title                       |
| ... | arguments passed to strwrap |

**Examples**

```
wrapper(rep("sting",50),30)
```

---

|           |                                                                 |
|-----------|-----------------------------------------------------------------|
| write_txt | <i>Print an object and optionally save output to a log file</i> |
|-----------|-----------------------------------------------------------------|

---

**Description**

Prints input to the console. When file is supplied, the printed output is also captured to a .log file using `sink()`, and the file contents are echoed back to the console after writing, so the output is visible in both places.

**Usage**

```
write_txt(input, file = NULL)
```

**Arguments**

|       |                                                                                                                                                                                              |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| input | Any R object to print.                                                                                                                                                                       |
| file  | Character string naming the output log file without extension. A .log extension is appended automatically. When NULL (default) output is printed to the console only and no file is written. |

**Value**

Invisibly returns NULL. Called for its side effects of printing and optionally writing to a log file.

**Examples**

```
write_txt(mtcars)
write_txt(mtcars,file="mtcars")
```

# Index

## \* ANOVA

- compute\_aov\_es, 18
- compute\_kruskal\_wallis\_test, 26
- compute\_one\_way\_test, 29
- compute\_posthoc, 30
- plot\_interaction, 104
- plot\_oneway, 112
- plot\_oneway\_diagnostics, 113
- report\_factorial\_anova, 139
- report\_manova, 145
- report\_oneway, 147

## \* EFA

- compute\_residual\_stats, 32
- model\_loadings, 92
- plot\_loadings, 106
- plot\_scree, 119
- report\_efa, 138

## \* IRT

- compute\_info\_1pl, 24
- compute\_info\_2pl, 24
- compute\_info\_3pl, 25
- compute\_scores, 32
- compute\_se\_theta, 33
- compute\_unidimensional\_ability, 39
- compute\_unidimensional\_theta, 40
- plot\_irt\_onefactor, 105
- report\_irt, 142

## \* ML

- plot\_trees\_xgboost, 121
- report\_lda, 143
- report\_xgboost, 157

## \* NLP

- clear\_stopwords, 13
- clear\_text, 14
- stat\_word\_char, 170
- tag\_pos, 179
- text\_similarity, 180

## \* SEM

- report\_cfa, 134

- simulate\_cfa\_fit, 166

## \* Thurstonian

- compute\_scores, 32

## \* algebra

- compute\_solve, 34

## \* assumptions

- outlier\_summary, 94
- plot\_boxplot, 98
- plot\_histogram, 102
- plot\_normality\_diagnostics, 111
- plot\_outlier, 114
- plot\_qq, 115
- remove\_outliers, 131
- report\_normality\_tests, 146

## \* cfa

- check\_heywood, 12

## \* correlation

- compute\_power\_r, 30
- compute\_power\_r\_matrix, 31
- deg2rad, 46
- plot\_corrplot, 101
- rad2deg, 125
- report\_choric\_serial, 135
- report\_correlation, 136

## \* dataframe

- cdf, 7
- cdff, 9

## \* datasets

- df\_admission, 47
- df\_automotive\_data, 48
- df\_blood\_pressure, 49
- df\_co2, 50
- df\_crop\_yield, 51
- df\_difficile, 51
- df\_insurance, 52
- df\_ocean, 53
- df\_personality, 56
- df\_responses, 58
- df\_responses\_state, 63



- df\_sexual\_comp, 63
- df\_titanic, 64
- \* **data**
  - generate\_correlation\_matrix, 78
  - generate\_data, 78
  - generate\_factor, 79
  - generate\_missing, 81
  - generate\_multiple\_responce\_vector, 81
  - generate\_string, 82
  - simulate\_correlation\_from\_sample, 168
- \* **descriptives**
  - compute\_aggregate, 16
  - compute\_crosstable, 19
  - compute\_descriptives, 20
  - compute\_frequencies, 22
  - plot\_crosstable, 101
  - plot\_mosaic, 108
  - plot\_response\_frequencies, 116
  - response\_frequency, 159
- \* **diagnostics**
  - cdf, 7
  - cdff, 9
  - check\_heywood, 12
- \* **dimensions**
  - questions\_by\_keys, 124
  - questions\_dimensions\_dataframe, 124
- \* **distance**
  - compute\_tversky\_index, 37
- \* **ebm**
  - score\_tirt, 162
  - score\_tirt\_pattern, 164
- \* **effect-size**
  - report\_ttests, 151
  - report\_wtests, 154
- \* **functions**
  - c\_bind, 44
  - call\_to\_string, 7
  - change\_data\_type, 11
  - comparison\_combinations, 14
  - compute\_adjustment, 16
  - compute\_confidence\_inteval, 18
  - compute\_dissatenuation, 21
  - compute\_kurtosis, 27
  - compute\_skewness, 34
  - compute\_standard, 36
  - compute\_standard\_error, 37
  - confusion, 41
  - confusion\_matrix\_percent, 42
  - data\_frame\_index, 44
  - detach\_package, 46
  - display\_upper\_lower\_triangle, 65
  - dotnames, 66
  - drop\_levels, 67
  - dummy\_arrange, 67
  - duplicate\_y\_axis, 68
  - environment\_options, 69
  - excel\_confusion\_matrix, 69
  - excel\_critical\_value, 70
  - excel\_generic\_format, 71
  - excel\_matrix, 72
  - flatten\_list, 77
  - generate\_correlation\_matrix, 78
  - generate\_data, 78
  - generate\_factor, 79
  - generate\_missing, 81
  - generate\_multiple\_responce\_vector, 81
  - generate\_string, 82
  - get\_script\_directory, 84
  - getfw, 83
  - hinvert\_title\_grob, 84
  - install\_all\_packages, 86
  - install\_load, 86
  - k\_fold, 87
  - k\_sample, 88
  - matrix\_triangle, 89
  - mgs, 90
  - min\_max\_index, 91
  - off\_diagonal\_index, 94
  - output\_separator, 95
  - padNA, 96
  - plot\_confusion, 100
  - plot\_multiplot, 110
  - plot\_roc, 117
  - plot\_separability, 120
  - proper, 123
  - proportion\_accurate, 123
  - questions\_by\_keys, 124
  - questions\_dimensions\_dataframe, 124
  - rbind\_all, 128
  - recode\_scale\_dummy, 129
  - remove\_nc, 130

- remove\_user\_packages, 132
- replace\_na\_with\_previous, 132
- report\_dataframe, 137
- report\_pdf, 148
- result\_confusion\_performance, 160
- round\_dataframe, 161
- simulate\_correlation\_from\_sample, 168
- split\_str, 169
- split\_str\_df, 169
- string\_aes, 170
- sub\_str, 178
- swap, 178
- symmetric\_matrix, 179
- trim\_df, 181
- wrapper, 183
- write\_txt, 183
- \* generate**
  - generate\_correlation\_matrix, 78
  - generate\_data, 78
  - generate\_factor, 79
  - generate\_missing, 81
  - generate\_multiple\_responce\_vector, 81
  - generate\_string, 82
  - simulate\_correlation\_from\_sample, 168
- \* heywood**
  - check\_heywood, 12
- \* inference**
  - report\_ttests, 151
  - report\_wtests, 154
- \* irt**
  - cfa\_icc\_index, 10
  - compute\_ability, 15
  - compute\_dummy\_comparisons, 22
  - compute\_icc\_thurstonian, 23
  - compute\_map, 28
  - extract\_tirt\_params, 75
  - generate\_comparisons\_matrix, 77
  - generate\_matrix\_A, 80
  - generate\_matrix\_lambda\_hat, 80
  - generate\_unique\_comparisons\_index, 82
  - icc\_cfa, 85
  - increase\_index, 85
  - plot\_icc\_thurstonian, 103
  - rank3\_to\_triplets, 126
  - rank\_df\_to\_binary, 126
  - rank\_to\_binary, 127
  - response\_dimension, 158
  - score\_tirt, 162
  - score\_tirt\_pattern, 164
- \* keys**
  - questions\_by\_keys, 124
  - questions\_dimensions\_dataframe, 124
- \* kruskal**
  - compute\_kruskal\_wallis\_test, 26
- \* labels**
  - name\_triplet\_pairs, 92
- \* lavaan**
  - check\_heywood, 12
  - extract\_tirt\_params, 75
- \* linear-equations**
  - compute\_solve, 34
- \* logistic**
  - compute\_y\_logistic, 41
  - output\_compare\_model\_logistic, 95
  - plot\_logistic\_model, 107
  - report\_logistic, 143
- \* map**
  - score\_tirt, 162
  - score\_tirt\_pattern, 164
- \* matrix**
  - compute\_solve, 34
  - display\_upper\_lower\_triangle, 65
  - matrix\_triangle, 89
  - off\_diagonal\_index, 94
  - symmetric\_matrix, 179
- \* missing-values**
  - cdf, 7
  - cdff, 9
- \* nonparametric**
  - compute\_kruskal\_wallis\_test, 26
  - report\_wtests, 154
- \* pairs**
  - name\_triplet\_pairs, 92
- \* pairwise**
  - report\_ttests, 151
  - report\_wtests, 154
- \* plot**
  - duplicate\_y\_axis, 68
  - hinvert\_title\_grob, 84
  - plot\_multiplot, 110
  - report\_pdf, 148

- wrapper, 183
- \* **regression**
  - compute\_y\_logistic, 41
  - output\_compare\_model\_logistic, 95
  - plot\_logistic\_model, 107
  - plot\_scatterplot, 118
  - report\_logistic, 143
  - report\_regression, 149
- \* **reliability**
  - alpha\_diagnostics, 6
  - extract\_components, 74
  - key\_to\_cfa\_model, 87
  - mean\_sd\_alpha, 90
  - plot\_mtm, 109
  - raw\_alpha, 128
  - report\_alpha, 133
  - shrout, 165
- \* **reporting**
  - report\_ttests, 151
  - report\_wtests, 154
- \* **scoring**
  - extract\_tirt\_params, 75
  - score\_tirt, 162
  - score\_tirt\_pattern, 164
- \* **sem**
  - check\_heywood, 12
- \* **series**
  - compute\_moving\_average, 28
  - plot\_acf, 97
  - plot\_ts, 122
  - ts\_smoothing, 181
- \* **set**
  - compute\_tversky\_index, 37
- \* **similarity**
  - compute\_tversky\_index, 37
- \* **statistical**
  - compute\_adjustment, 16
  - compute\_confidence\_inteval, 18
  - compute\_dissatenuation, 21
  - compute\_kurtosis, 27
  - compute\_skewness, 34
  - compute\_standard, 36
  - compute\_standard\_error, 37
- \* **strings**
  - call\_to\_string, 7
  - fixed, 76
  - mgsub, 90
  - output\_separator, 95
  - proper, 123
  - split\_str, 169
  - split\_str\_df, 169
  - str\_count, 171
  - str\_pad, 172
  - str\_replace, 173
  - str\_replace\_all, 174
  - str\_split\_fixed, 175
  - str\_squish, 176
  - str\_wrap, 177
  - string\_aes, 170
  - sub\_str, 178
  - trim\_df, 181
- \* **summary**
  - cdf, 7
  - cdff, 9
- \* **t-test**
  - report\_ttests, 151
- \* **timestamp**
  - convert\_excel\_unix\_timestamp, 43
  - decompose\_datetime, 45
- \* **time**
  - compute\_moving\_average, 28
  - plot\_acf, 97
  - plot\_ts, 122
  - ts\_smoothing, 181
- \* **tirt**
  - cfa\_icc\_index, 10
  - compute\_ability, 15
  - compute\_dummy\_comparisons, 22
  - compute\_icc\_thurstonian, 23
  - compute\_map, 28
  - extract\_tirt\_params, 75
  - generate\_comparisons\_matrix, 77
  - generate\_matrix\_A, 80
  - generate\_matrix\_lambda\_hat, 80
  - generate\_unique\_comparisons\_index, 82
  - icc\_cfa, 85
  - increase\_index, 85
  - plot\_icc\_thurstonian, 103
  - rank3\_to\_triplets, 126
  - rank\_df\_to\_binary, 126
  - rank\_to\_binary, 127
  - response\_dimension, 158
  - score\_tirt, 162
  - score\_tirt\_pattern, 164
- \* **triplets**

- name\_triplet\_pairs, 92
- \* **unidimensional**
  - compute\_info\_1pl, 24
  - compute\_info\_2pl, 24
  - compute\_info\_3pl, 25
  - compute\_se\_theta, 33
  - compute\_unidimensional\_ability, 39
  - compute\_unidimensional\_theta, 40
- \* **utility**
  - name\_triplet\_pairs, 92
- \* **wilcoxon**
  - report\_wtests, 154
- alpha\_diagnostics, 6
- c\_bind, 44, 66, 96
- call\_to\_string, 7
- cdf, 7, 9, 10, 38
- cdff, 9
- cfa\_icc\_index, 10
- change\_data\_type, 11
- check\_heywood, 12
- clear\_stopwords, 13
- clear\_text, 14
- comparison\_combinations, 14
- compute\_ability, 15
- compute\_adjustment, 16
- compute\_aggregate, 16
- compute\_aov\_es, 18
- compute\_confidence\_inteval, 18
- compute\_crosstable, 14, 19
- compute\_descriptives, 16, 20
- compute\_dissatenuation, 21
- compute\_dummy\_comparisons, 22
- compute\_frequencies, 22
- compute\_icc\_thurstonian, 23
- compute\_info\_1pl, 24
- compute\_info\_2pl, 24
- compute\_info\_3pl, 25
- compute\_kruskal\_wallis\_test, 26
- compute\_kurtosis, 27
- compute\_map, 28
- compute\_moving\_average, 28
- compute\_one\_way\_test, 29
- compute\_posthoc, 30
- compute\_power\_r, 30
- compute\_power\_r\_matrix, 31
- compute\_residual\_stats, 32
- compute\_scores, 32
- compute\_se\_theta, 33
- compute\_skewness, 34
- compute\_solve, 34
- compute\_standard, 36
- compute\_standard\_error, 37
- compute\_tversky\_index, 37
- compute\_unidimensional\_ability, 39
- compute\_unidimensional\_theta, 40
- compute\_y\_logistic, 41
- confusion, 41
- confusion\_matrix\_percent, 42
- convert\_excel\_unix\_timestamp, 43
- data\_frame\_index, 44
- decompose\_datetime, 45
- deg2rad, 46
- detach\_package, 46
- df\_admission, 47
- df\_automotive\_data, 48
- df\_blood\_pressure, 49
- df\_co2, 50
- df\_crop\_yield, 51
- df\_difficile, 51
- df\_insurance, 52
- df\_ocean, 53
- df\_personality, 56
- df\_responses, 58
- df\_responses\_state, 63
- df\_sexual\_comp, 63
- df\_titanic, 64
- display\_upper\_lower\_triangle, 65
- dotnames, 66
- drop\_levels, 67
- dummy\_arrange, 67
- duplicate\_y\_axis, 68
- environment\_options, 69
- excel\_confusion\_matrix, 69
- excel\_critical\_value, 70
- excel\_generic\_format, 71
- excel\_matrix, 72
- extract\_components, 74, 165
- extract\_tirt\_params, 75
- fixed, 76
- flatten\_list, 77
- generate\_comparisons\_matrix, 77
- generate\_correlation\_matrix, 78

generate\_data, 78  
generate\_factor, 79  
generate\_matrix\_A, 80  
generate\_matrix\_lambda\_hat, 80  
generate\_missing, 81  
generate\_multiple\_responce\_vector, 81  
generate\_string, 82  
generate\_unique\_comparisons\_index, 82  
get\_script\_directory, 84  
getfw, 83  
  
hinvert\_title\_grob, 84  
  
icc\_cfa, 85  
increase\_index, 85  
install\_all\_packages, 86  
install\_load, 86  
intersect, 38  
  
k\_fold, 87  
k\_sample, 88  
key\_to\_cfa\_model, 87  
  
matrix\_triangle, 89  
mean\_sd\_alpha, 90  
mgsub, 90  
min\_max\_index, 91  
model.frame, 152, 155  
model\_loadings, 92  
  
NA, 152, 155  
name\_triplet\_pairs, 92  
  
off\_diagonal\_index, 94  
outlier\_summary, 94  
output\_compare\_model\_logistic, 95  
output\_separator, 95  
  
padNA, 96  
Pair, 152, 155  
plot\_acf, 97  
plot\_boxplot, 98  
plot\_cfa, 98  
plot\_cfa\_gg, 99  
plot\_confusion, 100  
plot\_corrplot, 101  
plot\_crosstable, 14, 101  
plot\_histogram, 102  
plot\_icc\_thurstonian, 103  
plot\_interaction, 104  
plot\_irt\_onefactor, 105  
plot\_loadings, 106  
plot\_logistic\_model, 107  
plot\_mosaic, 108  
plot\_mtmm, 109  
plot\_multiplot, 110  
plot\_normality\_diagnostics, 111  
plot\_oneway, 112  
plot\_oneway\_diagnostics, 113  
plot\_outlier, 114  
plot\_qq, 115  
plot\_response\_frequencies, 116  
plot\_roc, 117  
plot\_scatterplot, 118  
plot\_scee, 119  
plot\_separability, 120  
plot\_trees\_xgboost, 121  
plot\_ts, 122  
proper, 123  
proportion\_accurate, 123  
  
questions\_by\_keys, 124  
questions\_dimensions\_dataframe, 124  
  
rad2deg, 125  
rank, 155  
rank3\_to\_triplets, 126  
rank\_df\_to\_binary, 126  
rank\_to\_binary, 127  
raw\_alpha, 6, 128  
rbind\_all, 128  
recode\_scale\_dummy, 129  
remove\_nc, 130  
remove\_outliers, 131  
remove\_user\_packages, 132  
replace\_na\_with\_previous, 132  
report\_alpha, 87, 133  
report\_cfa, 134  
report\_choric\_serial, 135  
report\_correlation, 136  
report\_dataframe, 137  
report\_efa, 138  
report\_factorial\_anova, 139  
report\_hlr, 141  
report\_irt, 142  
report\_lda, 143  
report\_logistic, 143  
report\_manova, 145  
report\_normality\_tests, 146

report\_oneway, [147](#)  
report\_pdf, [148](#)  
report\_regression, [149](#)  
report\_ttests, [151](#)  
report\_wtests, [154](#)  
report\_xgboost, [157](#)  
response\_dimension, [158](#)  
response\_frequency, [159](#)  
result\_confusion\_performance, [160](#)  
round\_dataframe, [161](#)

score\_tirt, [162](#)  
score\_tirt\_pattern, [164](#)  
setdiff, [38](#)  
shrout, [165](#)  
signif, [155](#)  
simulate\_cfa\_fit, [166](#)  
simulate\_correlation\_from\_sample, [168](#)  
split\_str, [169](#)  
split\_str\_df, [169](#)  
stat\_word\_char, [170](#)  
stats::t.test, [151](#)  
stats::wilcox.test, [154](#)  
str\_count, [171](#)  
str\_pad, [172](#)  
str\_replace, [173](#)  
str\_replace\_all, [174](#)  
str\_split\_fixed, [175](#)  
str\_squish, [176](#)  
str\_wrap, [177](#)  
string\_aes, [170](#)  
sub\_str, [178](#)  
swap, [178](#)  
symmetric\_matrix, [179](#)

tag\_pos, [179](#)  
text\_similarity, [180](#)  
trim\_df, [181](#)  
ts\_smoothing, [181](#)

uniroot, [155](#)

wrapper, [183](#)  
write\_txt, [183](#)